


**UNIVERSIDAD DE SEVILLA**

**INTERNATIONAL POSTGRADUATE SCHOOL**

MASTER'S THESIS

# Design and Implementation of an Internal Developer Platform (IDP) on OpenShift Governed by AI Agents via the MCP Protocol

*(Agentic Deployer)*

Author

**Jesús Solís Ortega**

Submitted in partial fulfillment of the requirements for the degree of
Master in Software Engineering: Cloud, Data and IT Management

Supervised by

**Manuel Resinas Arias de Reyna**

**November 2026 Examination Period, Academic Year 2025/26**

# Statement of Authorship

Mr. **Jesús Solís Ortega**, author of the Master's Thesis entitled *«Design and Implementation of an Internal Developer Platform (IDP) on OpenShift Governed by AI Agents via the MCP Protocol (Agentic Deployer)»*, submitted in Seville in partial fulfillment of the requirements for the degree of **Master in Software Engineering: Cloud, Data and IT Management** at the University of Seville, hereby declares under his own responsibility that:

1. This work is original and was conducted by the author under the academic supervision and guidance of his advisor.
2. All consulted sources—bibliographic, documentary, and digital—have been properly cited in accordance with academic rigor standards.
3. No content from other authors has been plagiarized, nor have third-party materials been used without due attribution or authorization.
4. Generative Artificial Intelligence tools were utilized strictly in an instrumental and assistive capacity (scaffolding initial code skeletons, exploring libraries, accelerating testing pipelines, and polishing prose style). All source code has been verified, compiled, and evaluated through the automated test suite of the project; all architectural design decisions, critical analyses, empirical evaluations, conclusions, and intellectual accountability rest solely with the author.
5. He grants the University of Seville permission to archive and utilize this work exclusively for academic, assessment, and scientific dissemination purposes, always attributing proper authorship.

Seville, November 2026.

**Signed:** Jesús Solís Ortega

# Acknowledgments

I wish to express my deepest gratitude to my academic advisor, Manuel Resinas Arias de Reyna, for his dedication, insightful critiques, and continuous methodological guidance throughout the development of this Master's Thesis. His feedback and observations have been instrumental in ensuring the technical rigor of this research.

Furthermore, I would like to thank my family and loved ones for their unconditional support, patience, and encouragement throughout this period of graduate studies.

---

> **Attribution Note:**
> The documentary structure, chapter organization, and methodological style guidelines of this thesis build upon the academic Master's Thesis template developed by the **Diverso Lab** research group (University of Seville), distributed under the *Creative Commons Attribution 4.0 International (CC BY 4.0)* license.

# Abstract

The growing complexity of Kubernetes and OpenShift declarative infrastructure poses a significant entry barrier for university researchers, leading to prolonged delivery cycles and costly negotiation processes with IT departments. This Master's Thesis presents the **Agentic Deployer**, an Internal Developer Platform (IDP) that provisions infrastructure across OpenShift and Kubernetes clusters through natural language under strict, deterministic human supervision.

The solution is built upon three pillars: (1) a Hexagonal Architecture that isolates business rules from the stochasticity of Large Language Models (LLMs); (2) the open *Model Context Protocol* (MCP) standard, which prevents vendor lock-in; and (3) a *Human-In-The-Loop* (HITL) pattern governed by an immutable Finite State Machine that ensures no deployment reaches the cluster without technical validation.

Development followed a Domain-First approach, with quality assured through Property-Based and Mutation Testing. Documented case studies reveal that the system eliminates asynchronous bottlenecks, projecting (under standardized operational assumptions) a reduction in Time-to-Market from an order of days to mere minutes. Furthermore, the economic analysis establishes a prospective ROI exceeding 470% over three years (~350% on total cumulative expenditure), demonstrating the high corporate viability of agentic engineering in controlled environments.

**Keywords:** Internal Developer Platform (IDP), OpenShift, Kubernetes, Model Context Protocol (MCP), LLM Agents, Hexagonal Architecture, Human-In-The-Loop.

# Resumen

La creciente complejidad de la infraestructura declarativa de Kubernetes y OpenShift supone una barrera de entrada para investigadores universitarios, forzando ciclos de entrega prolongados y costosos procesos de negociación con los Servicios de Informática (SIC). Este Trabajo de Fin de Máster presenta el **Agentic Deployer**, una Plataforma Interna de Despliegue (IDP) que permite provisionar infraestructura sobre clústeres OpenShift y Kubernetes mediante lenguaje natural bajo estricta supervisión humana determinista.

La solución se articula sobre tres pilares: (1) una Arquitectura Hexagonal que aísla las reglas de negocio de la estocasticidad de los Modelos de Lenguaje (LLM); (2) el estándar abierto *Model Context Protocol* (MCP), que evita el *vendor lock-in*; y (3) un patrón *Human-In-The-Loop* (HITL) gobernado por una Máquina de Estados inmutable que asegura que ningún despliegue impacte en el clúster sin validación técnica.

El desarrollo siguió un enfoque *Domain-First*, asegurando la calidad mediante *Property-Based* y *Mutation Testing*. Los casos de estudio revelan que el sistema elimina los cuellos de botella asíncronos, proyectando (bajo supuestos operativos estandarizados) una reducción del tiempo de entrega de un orden de días a escasos minutos. Adicionalmente, el análisis económico establece un ROI prospectivo superior al 470% a tres años (~350% sobre la inversión total acumulada), demostrando la alta viabilidad corporativa de la ingeniería agéntica en entornos controlados.

**Palabras clave:** Plataforma Interna de Despliegue (IDP), OpenShift, Kubernetes, Model Context Protocol (MCP), Agentes LLM, Arquitectura Hexagonal, Human-In-The-Loop.

# Table of Contents

**Declaration of Authorship** 2

**Acknowledgments** 3

**Abstract and Resumen** 4

**List of Tables** 8

**List of Figures, Algorithms, and Listings** 10

**List of Acronyms and Terms** 12

**Chapter 1. Introduction** 15

1.1. Context and Motivation 15

1.2. Problem Statement 16

1.3. Project Objectives 18

1.4. Scope and Limitations 19

1.5. Formal System Requirements 20

1.6. Structure of the Report 20

**Chapter 2. State of the Art** 22

2.1. Autonomous Agents Based on Large Language Models (LLMs) 22

2.2. Integration Standardization: Model Context Protocol (MCP) 23

2.3. Declarative Infrastructure and Orchestration (Kubernetes and OpenShift) 26

2.4. Hexagonal Architecture and Security in Stochastic Systems 29

2.5. Related Work and Differential Positioning 31

**Chapter 3. Methodology and Technology Stack** 34

3.1. Methodological Approach 34

3.2. Development Phases 34

3.3. Technology Stack and Architectural Justification 37

**Chapter 4. Project Management and Feasibility** 41

4.1. Project Plan — Context and Constraints 41

4.2. Sprint Structure and Effort Estimation 43

4.3. Development Effort and Deviations 46

4.4. Change Control: Evolution and Traceability from the Initial Proposal 47

4.5. Economic Analysis of Development 49

4.6. Corporate Adoption Cost and ROI 50

4.7. Feasibility Analysis Conclusions 56

**Chapter 5. System Design and Architecture** 57

5.1. Requirements Analysis and Architectural Design Criteria 57

5.2. Architectural Topological Modeling (C4 Standard) 60

5.3. Adoption of Hexagonal Architecture (Ports and Adapters) 65

5.4. Institutional Security Validation Algorithmics 68
5.5. Abstraction Pattern: Golden Paths and Syntactic Expansion 72
5.6. End-to-End Data Flow and Lifecycle States 75
**Chapter 6. Cognitive Agent Development and MCP 80**
6.1. Model Context Protocol (MCP) Server Implementation 80
6.2. Multi-Provider Abstraction Design and Data Sovereignty 84
6.3. Cognitive Loop and Stochastic Resilience (ReAct Pattern) 86
**Chapter 7. Human-In-The-Loop (HITL) Implementation and Finite State Management (FSM) 90**
7.1. Foundations of the Human-In-The-Loop (HITL) Pattern 90
7.2. Finite State Machine (FSM) Design 91
7.3. The Asynchronous Operations Dashboard 94
7.4. Feedback Channel to the Researcher: Asynchronous State Notification 98
**Chapter 8. Quality Assurance (QA) and Advanced Testing 101**
8.1. Domain and Integration Testing: Validating the Hexagonal Perimeter 102
8.2. Property-Based Testing: Testing Against Stochastic Behavior 105
8.3. Test Suite Auditing: Mutation Testing 107
8.4. Metamorphic Testing: Methodology and Definition of Cognitive Invariants 111
**Chapter 9. Experimental Results and Evaluation 113**
9.1. Framework and Experimental Environment 113
9.2. Real Case Studies (End-to-End Walkthroughs) 114
9.3. Experimental Evaluation and LLM Comparison 121
9.4. Cognitive Architecture Experiments: Overcoming the Attentional Limit (7B) 127
9.5. Evaluation of Operational Impact on ITSM Processes 129
9.6. Threats to Validity of the Empirical Study 131
**Chapter 10. Conclusions, Limitations, and Future Work 133**
10.1. Conclusions 133
10.2. Prototype Limitations 139
10.3. Future Work 141
**Chapter 11. Bibliography and References 143**
**Annex A. Generated Kubernetes Manifest — Scenario 1 (Happy Path) 149**
A.1. Generation Context 149
A.2. Complete YAML Manifest 150
A.3. Source Code Mapping 151
**Annex B. Detailed Sprint Retrospectives 153**
B.1. Sprint 1 — Hexagonal Core 153
B.2. Sprint 2 — HITL Backend and FSM 153
B.3. Sprint 3 — Golden Paths and FakeK8s 154
B.4. Sprint 4 — ReAct Agent and MCP SDK 154

B.5. Sprint 5 — Advanced QA ..... 154
B.6. Phase 6 — Thesis Drafting and Finalization ..... 155
B.7. Phase 7 — Technical Excellence Consolidation ..... 155
**Annex C. Installation, Deployment, and Operational Manual ..... 156**
C.1. Installation and Deployment Manual ..... 156
C.2. Subsystem Execution ..... 157
C.3. User Manual and HITL Operation ..... 158

# List of Tables

**Table 1:** Multidimensional comparison between proprietary Cloud assistants and Agentic Deployer. *(Chapter 2)* 33
**Table 2:** Development modules and priority hierarchy. *(Chapter 4)* 42
**Table 3:** Net effort estimation per Sprint. *(Chapter 4)* 45
**Table 4:** Summary of schedule deviations per phase. *(Chapter 4)* 46
**Table 5:** Change control matrix and traceability against the initial proposal. *(Chapter 4)* 48
**Table 6:** Human Resources costs (CAPEX equivalent). *(Chapter 4)* 49
**Table 7:** Infrastructure and tooling costs (Development phase). *(Chapter 4)* 50
**Table 8:** Subtotal and total costs of the development phase. *(Chapter 4)* 50
**Table 9:** Adopting organization profiles (Cases A, B, and C). *(Chapter 4)* 51
**Table 10:** Implementation costs (CAPEX - Initial Investment). *(Chapter 4)* 51
**Table 11:** Annual operational infrastructure costs (OPEX). *(Chapter 4)* 52
**Table 12:** Annual operational LLM engine costs (Local vs. Cloud) according to adoption levels. *(Chapter 4)* 52
**Table 13:** LLM engine cost estimation (Cloud API). *(Chapter 4)* 53
**Table 14:** Consolidated total annual OPEX by adoption profile. *(Chapter 4)* 54
**Table 15:** Annual operational savings calculation (Profile B scenario). *(Chapter 4)* 54
**Table 16:** Return on Investment (ROI) over 3 years (Profile B with Ollama). *(Chapter 4)* 55
**Table 17:** Return on Investment (ROI) sensitivity analysis against technician baseline time variations. *(Chapter 4)* 55
**Table 18:** Traceability between Formal Requirements and System Architectural Decisions. *(Chapter 5)* 58
**Table 19:** Comprehensive Traceability Matrix (Requirements ↔ Design ↔ QA Validation). *(Chapter 5)* 59
**Table 20:** FSM state mapping to visual notification indicators for the researcher. *(Chapter 7)* 99
**Table 21:** Test cases for the researcher status consultation layer. *(Chapter 8)* 104
**Table 22:** Invariant properties comprehensively audited via Hypothesis in `test_security.py`. *(Chapter 8)* 105
**Table 23:** Mutation Testing results by module. *(Chapter 8)* 109
**Table 24:** Evaluation environment for practical case studies. *(Chapter 9)* 113
**Table 25:** Execution trace: Validation in the hexagonal core (Step 5). *(Chapter 9)* 116
**Table 26:** Performance metrics of the complete walkthrough (Scenario 1). *(Chapter 9)* 117
**Table 27:** Execution trace: Prompt Injection attempt (Scenario 2). *(Chapter 9)* 119
**Table 28:** Latency breakdown by component across the lifecycle. *(Chapter 9)* 121
**Table 29:** End-to-end comparison of operational metrics (multi-scenario). *(Chapter 9)* 121
**Table 30:** Empirical Metamorphic Testing results by relation against Qwen 2.5 (7B). *(Chapter 9)* 122
**Table 31:** Global metrics of the metamorphic suite. *(Chapter 9)* 123
**Table 32:** Comparative results of the metamorphic stress test (pure Zero-Shot). *(Chapter 9)* 124
**Table 33:** Actual comparative performance of alternative models in the ReAct loop. *(Chapter 9)* 126
**Table 34:** Cognitive Architecture experiment results expanded with Cloud models. *(Chapter 9)* 128
**Table 35:** Operational time impact (traditional ITSM vs. Agentic Deployer). *(Chapter 9)* 130
**Table 36:** Traceability matrix and degree of fulfillment of specific project objectives. *(Chapter 10)* 134
**Table 37:** Cognitive engine ranking and empirical justification for institutional agentic orchestration. *(Chapter 10)* 138

**Table 38:** Generation Context of the YAML Manifest (Scenario 1). *(Annex A)* ..... 149
**Table 39:** Sprint 1 Retrospective (Hexagonal Core). *(Annex B)* ..... 153
**Table 40:** Sprint 2 Retrospective (HITL Backend and FSM). *(Annex B)* ..... 153
**Table 41:** Sprint 3 Retrospective (Golden Paths and FakeK8s). *(Annex B)* ..... 154
**Table 42:** Sprint 4 Retrospective (ReAct Agent and MCP SDK). *(Annex B)* ..... 154
**Table 43:** Sprint 5 Retrospective (Advanced QA). *(Annex B)* ..... 155
**Table 44:** Phase 6 Retrospective (Thesis Drafting and Finalization). *(Annex B)* ..... 155
**Table 45:** Phase 7 Retrospective (Technical Excellence Consolidation). *(Annex B)* ..... 156

# List of Figures, Algorithms, and Listings

## Figures

**Figure 1:** Architectural contrast between the N×M integration problem (proprietary coupling) and the Universal Bus topology proposed by the MCP protocol. *(Chapter 2)* 24

**Figure 2:** Technology stack component diagram highlighting segregation between UI, cognitive reasoning, constrained backend, and continuous quality validation tooling. *(Chapter 3)* 39

**Figure 3:** Gantt chart (Part 1). Tentative scheduling of Sprints 1 through 4 (core and agent). *(Chapter 4)* 44

**Figure 4:** Gantt chart (Part 2). Tentative scheduling of advanced QA and finalization/consolidation phases. *(Chapter 4)* 44

**Figure 5:** System Context Diagram (Level 1) of the C4 Model for Agentic Deployer. *(Chapter 5)* 61

**Figure 6:** Container Diagram (Level 2) of the C4 Model for the Agentic Deployer system. *(Chapter 5)* 62

**Figure 7:** Process-level deployment diagram. Components coexist on the same host machine; the MCP server communicates via `stdio` without exposing network ports. *(Chapter 5)* 63

**Figure 8:** Class diagram (UML) summarizing core logical entities and contracts, highlighting polymorphism for dependency injection (LLMClient, DeployPort, DeploymentRepositoryPort). *(Chapter 5)* 64

**Figure 9:** Hexagonal Architecture topology. Control flow enters from Primary Adapters, while code dependencies point inwards (Dependency Inversion Principle). *(Chapter 5)* 68

**Figure 10:** Simplified overview of `SecurityContextValidator`. Rejection branches trigger errors allowing ReAct self-correction. *(Chapter 5)* 71

**Figure 11:** End-to-end Sequence Diagram (Stage 1). Interaction between Researcher and LLM for structured intent extraction. *(Chapter 5)* 76

**Figure 12:** End-to-end Sequence Diagram (Stage 2). Deterministic validation in the backend: (Top) Nominal compliant path (Stage 2A); (Bottom) Interception of security violations and self-correction (Stage 2B). *(Chapter 5)* 77

**Figure 13:** End-to-end Sequence Diagram (Stage 3). Asynchronous IT technician supervision (HITL) and infrastructure materialization. *(Chapter 5)* 78

**Figure 14:** Complete lifecycle of an MCP message. The JSON-RPC protocol defines three phases: initialization (handshake and tool discovery), execution (invocation and response), and observation (feedback to the LLM). *(Chapter 6)* 82

**Figure 15:** Flowchart of the ReAct (Reasoning and Acting) cognitive loop. *(Chapter 6)* 88

**Figure 16:** State transition graph governing the system's Finite State Machine (FSM). *(Chapter 7)* 92

**Figure 17:** Screenshot of the HITL Operations Dashboard (`app/frontend/index.html`) displaying a pending deployment request (`dep-7f3a2c1b`) and consolidated operational metrics. *(Chapter 7)* 96

**Figure 18:** Sequence Diagram of the HITL workflow. (Top) Asynchronous polling and Approval Vector. (Bottom) Rejection Vector (terminal immutable state). *(Chapter 7)* 97

**Figure 19:** Screenshot of the researcher conversational interface (Streamlit) showing the asynchronous feedback panel with `PENDING_APPROVAL` status and manual refresh button. *(Chapter 7)* 100

**Figure 20:** Hybrid Testing Pyramid architecture implemented in Agentic Deployer, adapting Mike Cohn's classic model to Generative AI demands. *(Chapter 8)* 102

## Algorithms (Pseudocode)

**Algorithm 1:** Strict Hexagonal Domain Entity Validation (SecurityContextValidator). *(Chapter 5)* 71

**Algorithm 2:** Dynamic Tool Contract Introspection. *(Chapter 6)* ..... 81

**Algorithm 3:** Cognitive Orchestration Loop (ReAct Loop). *(Chapter 6)* ..... 89

**Algorithm 4:** Immutable Finite State Machine Transition (FSM_Transition). *(Chapter 7)* ..... 93

**Algorithm 5:** Property-Based Test for Hexagonal Invariants (Hypothesis Fuzzing). *(Chapter 8)* ..... 107

# Code Listings and Manifests

**Listing 1:** Complete Kubernetes manifest (Deployment + Service + Ingress) for a Python API. All three resources must coordinate correctly—especially the label highlighted with (A)—for the application to be accessible externally. *(Chapter 2)* ..... 28

**Listing 2:** Complete Kubernetes manifest generated by Agentic Deployer FakeK8sAdapter (Deployment + Service + Ingress) for Scenario 1 (Happy Path). *(Annex A)* ..... 151

# List of Acronyms and Terms

To facilitate technical reading and conceptual clarity, this section outlines the acronyms and terms frequently used throughout this thesis, particularly those pertaining to Platform Engineering, Generative Artificial Intelligence, and Cloud Infrastructure.

| Acronym | Original Meaning | Definition in the Context of this Thesis |
|---|---|---|
| **A2A** | Agent-to-Agent Protocol | Emerging protocol designed for standardized communication between multiple autonomous AI agents. |
| **API** | Application Programming Interface | Interface that enables two software applications to communicate. In this thesis, it primarily refers to the Kubernetes API and the LLM inference endpoints. |
| **C4** | Context, Containers, Components, Code | Standardized architectural diagramming model created by Simon Brown to visualize software architectures across four levels of abstraction. |
| **CAPEX** | Capital Expenditure | Upfront capital investment required to acquire, develop, or license permanent assets or custom software. |
| **CI/CD** | Continuous Integration / Continuous Deployment | DevOps practices that automate code integration, testing, packaging, and delivery to deployment targets. |
| **DAG** | Directed Acyclic Graph | Directed Acyclic Graph. A mathematical structure governing state machines, where transitions flow unidirectionally without closed cycles. |
| **DSL** | Domain-Specific Language | Specialized computer language tailored to a specific problem domain (e.g., PromQL, HCL, or Kubernetes declarative YAML). |
| **FSM** | Finite-State Machine | Finite-State Machine. A computational model ensuring that a deployment request resides in exactly one deterministic state at any point in time. |
| **HITL** | Human-In-The-Loop | Architectural pattern where human operators are intentionally placed in an automated control loop to audit, validate, and authorize critical actions. |
| **IaC** | Infrastructure as Code | Practice of managing and provisioning computational resources through machine-readable definition files (e.g., YAML manifests or OpenTofu scripts). |
| **IDP** | Internal Developer Portal/Platform | Internal Developer Platform. A self-service platform that abstracts infrastructure complexity for software engineers and researchers within an organization. |

| Acronym | Original Meaning | Definition in the Context of this Thesis |
|---|---|---|
| **ITIL** | Information Technology Infrastructure Library | Worldwide framework of standardized practices for Information Technology Service Management (ITSM). |
| **ITSM** | Information Technology Service Management | Activities, policies, and processes conducted by IT departments to design, deliver, operate, and control IT services. |
| **LLM** | Large Language Model | Deep neural network based on Transformer architectures capable of understanding and generating natural language and structured code. |
| **MCP** | Model Context Protocol | Open standard introduced by Anthropic to standardize how AI agents discover and interact with external data sources and tools. |
| **MR** | Metamorphic Relation | Metamorphic Relation. A formal invariance property describing expected system output behaviors under controlled input transformations, addressing the test oracle problem. |
| **MVP** | Minimum Viable Product | Minimum Viable Product. An initial release containing core functional capabilities sufficient to validate technical feasibility and satisfy core requirements. |
| **OPEX** | Operational Expenditure | Operational expenditure. Ongoing recurring operational costs (cloud subscriptions, electrical power, maintenance hours). |
| **PBT** | Property-Based Testing | Testing methodology where automated testing frameworks generate randomized inputs to verify universal logical invariants across domain models. |
| **QA** | Quality Assurance | System of procedures, testing layers (unit, property-based, mutation, and performance), and verification gates ensuring software satisfies formal specifications. |
| **RBAC** | Role-Based Access Control | Access control mechanism restricting computational resource access according to authorized roles assigned to users and service accounts. |
| **ReAct** | Reason and Act | Iterative prompting paradigm where the LLM alternates between internal deductive reasoning (*Thought*) and environment action invocations (*Action*). |
| **ROI** | Return on Investment | Financial metric comparing net economic benefits against total investment expenditures (personnel time saved versus development and deployment costs). |
| **SIC** | Servicio de Informática y Comunicaciones | University IT and Communications Department responsible for managing institutional infrastructure, networks, and computing facilities. |

| Acronym | Original Meaning | Definition in the Context of this Thesis |
|---|---|---|
| **TTM** | Time To Market / Time To Mitigation | Elapsed duration between the inception of an infrastructure requirement and the full operational availability of the corresponding service. |

# Technical Terminology

The following technical terms are retained in English according to conventional industry and academic practice:

| Term | Contextual Meaning | Usage in this Thesis |
|---|---|---|
| **Air-Gapped** | Disconnected or isolated network environment | Refers to network topologies physically or logically severed from public networks (the Internet), standard in high-security datacenters. In this thesis, it describes on-premise inference with Ollama. |
| **Boilerplate** | Standardized repetitive code templates | Reusable code structures required in multiple project components with minimal variation (e.g., FastAPI initialization or baseline schemas). |
| **Garbage Collector** | Runtime memory manager | Execution environment component that reclaims memory occupied by unreachable objects. Referenced in the context of the FSM terminal state transitions. |
| **Overhead** | Additional computational or latency cost | Supplementary resources (CPU cycles, RAM footprint, network latency) consumed by mediation middleware beyond raw business logic processing. |
| **Scaffolding** | Baseline project code generation | Automated generation of code skeletons and directory structures from predefined architectural archetypes. |

# Chapter 1. Introduction

## 1.1. Context and Motivation

Over the past decade, the software engineering and systems administration ecosystem has undergone a profound structural shift. The demand for accelerating time-to-market while ensuring higher scalability and resilience has driven a transition from monolithic architectures hosted on physical bare-metal servers toward distributed microservices architectures [1] and cloud computing environments [2]. At the core of this transition lies application containerization—popularized by technologies such as Docker [3] and open standards such as the Open Container Initiative (OCI) [4]—and, critically, the orchestration of containerized workloads through industry-standard platforms such as Kubernetes [5] and enterprise distributions such as Red Hat OpenShift and OKD.

While Kubernetes has addressed fundamental challenges regarding high availability, horizontal auto-scaling, and self-healing fault management, its adoption has introduced considerable operational complexity. The Infrastructure as Code (IaC) paradigm and declarative resource management require engineers to interact with the platform through extensive and complex YAML or JSON manifests. These documents describe not only the computational workloads themselves (*Deployments* or *Pods*), but also demand the meticulous specification of network topologies (*Services*, *Ingress* / *Routes*), access control policies (*RBAC*), hardware resource allocations and limits (CPU, memory), and persistent volume claims.

Historically, this landscape has created a substantial barrier to entry for application developers, researchers, and academic staff. These stakeholders frequently possess deep domain expertise regarding application business logic, yet lack specialized systems operations backgrounds. Although the *DevOps* movement aimed to bridge the operational gap between development (Dev) and operations (Ops) teams through shared responsibility [6], in practice it has frequently resulted in excessive cognitive overhead for personnel outside platform administration. Expecting university researchers to master the declarative nuances of Kubernetes merely to publish a departmental web service introduces disproportionate operational friction.

To address this friction, the industry has turned toward **Platform Engineering** [7]. This discipline advocates the construction of Internal Developer Platforms (IDPs), designed to provide self-service portals and "Golden Paths" [8]. A *Golden Path* is a standardized, institutionally supported route that encapsulates underlying infrastructure complexities: the user specifies high-level application requirements, and the platform automatically generates compliant technical configurations in alignment with corporate security policies. However, even modern IDPs typically rely on rigid graphical user interfaces, extensive forms, or Domain-Specific Languages (DSLs) that remain unintuitive for non-specialized users.

Concurrently, recent advancements in Generative Artificial Intelligence, driven by the emergence of Large Language Models (LLMs), have expanded human-computer interaction (HCI) capabilities. Transformer-based models such as GPT, Claude, or Llama, trained on billions of parameters, have demonstrated capabilities extending beyond stochastic text generation. They exhibit emergent deductive reasoning abilities, strong technical context comprehension, and the capability to translate imprecise natural language into structured code representations.

More recently, the evolution of these models has led to the development of **Autonomous AI Agents**. Leveraging function calling and iterative reasoning paradigms such as ReAct (*Reason + Act*) [9], LLMs are no longer passive query-answering engines, but cognitive controllers capable of making decisions, querying data sources, planning execution steps, and executing actions against external systems in real time.

The motivation for this Master's Thesis lies precisely at the intersection of these two paradigms: distributed infrastructure orchestration and agentic artificial intelligence. This convergence presents an opportunity to fundamentally transform how human operators interact with distributed operating environments.

The motivation of this research is to explore and demonstrate the technical feasibility of replacing complex declarative interfaces (YAML manifests) and static IDP forms with a conversational interface driven by an intelligent agent.

Under this paradigm, the system interprets user intent expressed in natural language ("I need to host a Python web API"), reasons about technical dependencies, and interactively negotiates missing requirements. Subsequently, it translates abstract intent into secure, standardized, and production-ready infrastructure definitions compatible with Kubernetes and enterprise distributions such as OpenShift. This approach lowers the technical barrier to entry for non-specialized users and streamlines software delivery lifecycles within institutional settings.

# 1.2. Problem Formulation

Despite the operational capabilities offered by conversational agents, integrating them directly into IT operations (IT Ops) workflows and critical infrastructure introduces several systemic challenges. This thesis focuses on the IT and Communications Department (*Servicio de Informática y Comunicaciones*, SIC) of a university environment as a primary case study; however, the underlying problem and proposed architecture generalize directly to other regulated sectors undergoing digital transformation.

This generalizability stems from the decoupled nature of the system design: in healthcare environments, for instance, *Golden Paths* would be configured to deploy clinical analytics workloads under strict data protection regulations (such as the Spanish *Esquema Nacional de Seguridad* (ENS) and GDPR/LOPD), where Human-in-the-Loop (HITL) oversight serves as an imperative compliance gate. Similarly, in regional public administration or financial banking environments, on-premise inference with self-hosted models ensures data sovereignty, while the immutable state machine guarantees auditability without requiring architectural modifications to the core engine.

Within the target academic institution, diverse user profiles—including research groups, teaching staff, and administrative personnel—frequently require standardized technological services. Recurring requests include deploying e-learning platforms (Moodle), content management systems (WordPress) for departmental sites, static web portals for academic conferences, and hosting lightweight microservices or APIs (typically built in Python or Node.js) resulting from applied research projects.

Manually managing these requests creates substantial operational overhead for IT engineers, who spend a significant portion of their time performing repetitive triage and baseline provisioning tasks. Conversely, granting direct cluster access to university personnel is unfeasible due to security governance constraints and specialized skills gaps. Automating service provisioning through Artificial Intelligence represents a logical progression; however, delivering an enterprise-grade solution requires addressing three fundamental technological challenges:

## 1.2.1. Lack of Standardization in Tool Orchestration

For an LLM to evolve from a passive conversational agent into an operational tool, it must interact with external systems. Historically, integrating external capabilities (API invocations, script executions, database queries) required tightly coupled, proprietary middleware specific to individual AI vendors. If an institution elected to migrate from the OpenAI API to a local open-weights model (such as Ollama) to satisfy institutional data privacy mandates, large portions of the orchestration

middleware had to be rewritten.

This challenge stems from the historical absence of a standardized communication protocol between language models and external tool ecosystems. A universal, bidirectional abstraction layer is required to allow IT departments to expose deployment routines in a vendor-agnostic manner, enabling any authorized agent to discover and invoke them regardless of model provider.

### 1.2.2. Infrastructure Abstraction and Hallucination Risks

The second critical challenge arises from the generative nature of LLMs. General-purpose models trained on vast internet corpora tend to produce syntactically valid Kubernetes manifests or Docker configurations that are semantically flawed or insecure within a specific corporate environment. This hallucination risk can cause an agent to propose untrusted container images [4] (e.g., using mutable `:latest` tags vulnerable to supply-chain risks), expose unapproved network ports, or ignore restrictive CPU and memory limits (*ResourceQuotas*) enforced by platform teams to prevent noisy-neighbor contention.

Consequently, naive automation approaches that delegate the unrestricted synthesis of Infrastructure as Code scripts (IaC via OpenTofu [10] or Terraform) directly to LLMs prove excessively fragile and unpredictable in production environments [11, 12]. Addressing this challenge with sound engineering requires that the agent never generate raw infrastructure definitions from scratch. Instead, the system must invoke predefined *Golden Paths*, where the model negotiates and infers high-level parameters (application name, expected traffic volume, runtime version), while the underlying architecture deterministically enforces institutional security, networking, and sizing policies.

### 1.2.3. Security and Auditability: The Human-in-the-Loop Imperative

The most critical challenge lies in delegating operational authority. Granting a stochastic system—inherently vulnerable to prompt injection attacks or reasoning lapses—unrestricted execution privileges over production clusters introduces unacceptable security risks in critical institutional infrastructure. An erroneous or adversarial instruction could trigger accidental data loss or deplete available cluster compute resources.

Therefore, fully autonomous closed-loop provisioning (where AI plans, decides, and executes without human mediation) is unacceptable. The architecture must integrate human oversight as a mandatory verification gate in the execution pipeline. The system requires an asynchronous *Human-in-the-Loop* (HITL) model where the agent shoulders the cognitive burden of eliciting and validating technical specifications, while the authority to execute deployments (*apply* / *commit*) remains strictly in the hands of authorized human engineers. This boundary ensures end-to-end traceability and operational safety, balancing AI-driven automation with classic engineering rigor.

## 1.3. Project Objectives

To address the identified challenges, this project establishes a set of structured technical objectives.

### 1.3.1. General Objective

The central objective of this Master's Thesis is to design and implement an enterprise-grade **Internal Developer Platform (IDP)**, materialized in the **«Agentic Deployer»** middleware: an advanced orchestrator that securely translates natural language requests into declarative infrastructure configurations interoperable with OpenShift and Kubernetes environments. This middleware serves as a strict security boundary between an LLM-driven conversational agent and underlying container

infrastructure, leveraging the open **Model Context Protocol (MCP)** [13] standard and structured under **Hexagonal Architecture** principles.

### 1.3.2. Specific Objectives

To achieve the general objective, five specific technical milestones have been defined and implemented:

1. **Design of a Core based on Hexagonal Architecture (*Ports and Adapters*):**
   Construct a strongly typed, immutable domain model that isolates business rules (provisioning workflows and institutional security policies) from volatile external dependencies, such as third-party LLM client SDKs or container orchestrator APIs.
2. **Implementation of an Open Standards Tool Server (MCP):**
   Demonstrate the architectural advantages of open protocols over proprietary vendor lock-in by implementing an Anthropic *Model Context Protocol* server. This server exposes IT infrastructure operations (service provisioning and teardown routines) such that any compliant agent can discover and invoke them using typed schemas.
3. **Integration of an Autonomous AI Agent (ReAct Paradigm):**
   Develop a cognitive orchestrator that moves beyond static *Zero-Shot* extraction to implement a *Reason + Act* loop. The agent must reason over error responses returned by underlying infrastructure components (e.g., policy violations) and engage in an interactive self-correcting feedback dialogue with the user.
4. **Rigid Specification of Golden Paths:**
   Prevent unconstrained manifest generation by delegating container template instantiation to the infrastructure adapter layer. Dedicated deployment patterns are designed for the most common university workloads: static web portals (Nginx), institutional content management systems (WordPress), and backend microservices (Python APIs).
5. **Security Governance via Human-in-the-Loop (HITL) Controls:**
   Implement an asynchronous operations dashboard that intercepts every deployment intent generated by the agent, holding it in a quarantined pending state until an authorized human engineer inspects generated configurations and issues explicit deployment approval.

## 1.4. Scope and Limitations

Delineating the precise boundary of a project combining Generative AI and Cloud-Native infrastructure is essential to ensure methodological rigor. This thesis focuses on developing the central mediation component (the intelligent middleware and its orchestration engine) as an advanced Minimum Viable Product (MVP).

Accordingly, the project scope incorporates the following deliberate methodological and architectural boundaries:

1. **Cluster Abstraction via Execution Emulation:**
   The orchestrator fully executes domain logic, security validations, human approvals, and structured declarative YAML generation for Kubernetes resources (`Deployment`, `Service`, `Ingress`). However, live network calls against physical production cluster APIs are excluded from the current baseline. Instead, an emulation adapter (`FakeK8sAdapter`) writes generated manifests to the local filesystem. This preserves Hexagonal Architecture principles, isolates the test harness from cluster network dependencies, and enables straightforward drop-in integration of a live cluster adapter in future work (Section 10.3.1).

2. **LLM Engine Agnosticism:**
   The platform is decoupled from specific AI vendors. By implementing an *Adapter* pattern for the LLM client with two concrete implementations (`OllamaLLMClient` for local air-gapped inference with Zero Data Retention, and `OpenAILLMClient` for commercial cloud APIs), the architecture supports both isolated private networks and public cloud endpoints.
3. **Embedded Persistence (SQLite vs. Client-Server RDBMS):**
   Transactional persistence is vital for operational infrastructure: service restarts must not compromise auditability or discard in-flight deployment requests. While early exploratory prototypes utilized an in-memory dictionary, the final platform adopts an on-disk SQLite repository [14]. This embedded database provides full ACID transactional guarantees without the operational overhead of running a dedicated client-server database cluster (such as PostgreSQL or MySQL). Foregoing a distributed persistence backend represents a deliberate scope boundary, and migrating to enterprise databases is straightforward due to the *Repository* port abstraction (Section 10.3.1).
4. **Absence of Authentication in the Prototype:**
   HTTP endpoints operate without authentication mechanisms (JWT, OAuth2) in the current release. This design choice is acceptable within a local research prototype and is documented as a future extension in Section 10.3.1.

A comprehensive technical analysis of prototype limitations—including advanced prompt injection considerations, backend horizontal scalability, and MCP tool catalog expansion—is provided in **Section 10.2**.

# 1.5. Formal System Requirements

To ensure traceability from project objectives (Section 1.3) through implementation, requirements are formalized into Functional Requirements (FR) and Non-Functional Requirements (NFR):

**Functional Requirements (FR):**

- **FR-01 (Conversational Interface):** The system must provide a web chat interface allowing users to request infrastructure provisioning using natural language.
- **FR-02 (Agentic Orchestration):** The system must infer missing parameters, query runtime state, and iteratively resolve errors through a reasoning loop (ReAct).
- **FR-03 (MCP Tool Catalog):** The backend must expose infrastructure capabilities (Golden Paths) in a decoupled, standardized manner via the MCP protocol.
- **FR-04 (HITL Approval Dashboard):** The system must provide an independent dashboard for IT operators to review, inspect, approve, or reject deployment intents generated by the AI agent.
- **FR-05 (Asynchronous Status Polling):** Users must be able to query the real-time status of their requests (Pending, Approved, Rejected) asynchronously.

**Non-Functional Requirements (NFR):**

- **NFR-01 (Data Sovereignty):** The architecture must support local inference models (e.g., Ollama) ensuring no institutional data leaks to third-party APIs.
- **NFR-02 (Domain Isolation):** Business logic must follow Hexagonal Architecture principles, ensuring core domain invariants remain independent of AI models and persistence technologies.

- **NFR-03 (Quality and Robustness):** Core validation logic must exceed 80% unit test coverage, complemented by advanced verification techniques including Property-Based Testing and Mutation Testing.
- **NFR-04 (Operational Resilience):** The system must not mutate state under ambiguous linguistic inputs, ensuring that erroneous or incomplete model inferences fail safely without applying unverified configurations (*Fail-Safe*).

## 1.6. Structure of the Thesis

To facilitate structural traceability from theoretical foundations to software verification, this document is organized as follows:

- **Chapter 2** presents the **State of the Art**, reviewing autonomous conversational agents, the genesis and architecture of the *Model Context Protocol* (MCP), declarative Kubernetes infrastructure, Hexagonal Architecture principles, and —in Section 2.5—a comparative analysis of related works highlighting the differential positioning of *Agentic Deployer*.
- **Chapter 3** outlines the **Methodology and Technology Stack**, detailing software lifecycle practices and justifying architectural choices. It describes post-MVP consolidation phases (Phases 6 and 7), focusing on HITL hardening, Ollama local inference integration, SQLite persistence, and multi-model benchmarking.
- **Chapter 4** covers **Project Management and Viability**, detailing development constraints, sprint schedules (Gantt charts in Figures 3 and 4), effort tracking and deviation analyses, economic assessments (personnel and infrastructure costs), and institutional ROI projections.
- **Chapter 5** details **System Design and Architecture**, beginning with requirements traceability (Tables 18 and 19), presenting C4 topology models (Level 1 Context and Level 2 Containers), and examining the hexagonal core and validation contracts. Figures 11, 12, and 13 document complete end-to-end flows, including self-correction (HTTP 422) and HITL governance.
- **Chapter 6** examines the **Cognitive Agent and MCP Server**, analyzing the ReAct reasoning loop, dynamic tool discovery, and multi-provider client abstractions (`OllamaLLMClient` / `OpenAILLMClient`).
- **Chapter 7** describes the **HITL Pattern and Finite State Machine (FSM)** implementation, detailing the asynchronous human approval workflow, the status tracking endpoint, and user notification mechanics.
- **Chapter 8** presents the **Quality Assurance (QA) and Advanced Testing Plan**, covering unit testing, Property-Based Testing with Hypothesis, formal Metamorphic Testing to address the test oracle problem, and Mutation Testing with Mutmut survivor analyses.
- **Chapter 9** documents **Experimental Results and Empirical Evaluation**, structured across six sections: experimental testbed setup (§9.1), provisioning walkthroughs (§9.2), comparative multi-model benchmarking in Zero-Shot and ReAct loops (§9.3), cognitive architecture evaluations for 7B models (§9.4), quantitative ITSM Time-to-Market impact models (§9.5), and threats to validity (§9.6).
- **Chapter 10** summarizes **Conclusions, Limitations, and Future Work**, synthesizing the objective compliance matrix and engineering contributions (§10.1), detailing prototype limitations (§10.2), and outlining short- and medium-term research directions (§10.3).
- **Chapter 11** compiles the **Bibliography and References** supporting the theoretical foundation of this thesis.
- **Annex A** provides the complete Kubernetes YAML manifest (`Deployment` + `Service` + `Ingress`) generated during

Scenario 1 (Happy Path), accompanied by technical annotations.

- **Annex B** documents **Detailed Sprint Retrospectives**, breaking down metrics, effort deviations, and technical milestones achieved across all development phases.
- **Annex C** provides the **Deployment, Installation, and User Manual**, detailing step-by-step setup requirements, subsystem execution commands, and operator instructions for the HITL dashboard.

# Chapter 2. State of the Art

This chapter establishes the theoretical and technological framework underpinning the architecture of *Agentic Deployer*. It provides a comprehensive review of contemporary paradigms in Artificial Intelligence, standardization protocols for foundational model communication, and modern principles of Platform Engineering.

## 2.1. Autonomous Agents Based on Large Language Models (LLMs)

For decades, Natural Language Processing (NLP) research focused on developing models capable of understanding and generating fluent human text. However, with the introduction of the Transformer architecture [15] and the subsequent maturation of Large Language Models (LLMs), empirical evidence revealed that beyond certain parameter and training corpus scale thresholds, models exhibit emergent capabilities that transcend simple probabilistic next-token prediction.

These capabilities include deductive logical reasoning, zero-shot generalization, and complex instruction-following [16]. Leveraging these capabilities has driven a paradigm shift in the field: the evolution from passive conversational chatbots to **Autonomous Agents**. An LLM-based agent is a computational system in which the language model functions not merely as an interface, but as the central reasoning engine that perceives runtime state, plans sequential steps, and executes actions upon its operating environment to modify that state.

To transcend the model's static encapsulation—typically isolated from real-time networks and bounded by training knowledge cutoffs—and attain actionable agency, the software industry has consolidated two fundamental technical advancements: the *ReAct* reasoning paradigm and *Function Calling* (or *Tool Calling*).

### 2.1.1. The ReAct (Reason + Act) Paradigm

Prior to the formalization of agentic architectures, conventional prompting paradigms required models to emit immediate final responses or reason statically through techniques such as *Chain-of-Thought* (CoT) [17]. Although CoT significantly improves deductive reasoning by prompting models to think step by step, it suffers from an intrinsic limitation: the model reasons exclusively over information contained within its initial prompt or static parameter weights, lacking the ability to query external runtime state or rectify invalid premises dynamically. In mission-critical infrastructure operations, this frequently leads to hallucination and error propagation.

To address this challenge, Yao et al. [9] introduced the **ReAct** paradigm. This conceptual framework alternates human-interpretable reasoning trace generation with explicit environment action executions. In a ReAct control loop, the agent operates through an iterative cycle governed by three primitives:

1. **Thought:** The agent evaluates the current conversational context and user request to deduce the next logical action. For instance: *"The user requested a CMS deployment. I must verify departmental quotas before allocating memory."*
2. **Action:** The agent selects an external tool from an available catalog to execute. For example: `consultar_cuotas_departamento(departamento="informatica")`.
3. **Observation:** The execution environment (the underlying application layer) invokes the tool against real infrastructure and returns the structured output (text or JSON) to the LLM. The model integrates this observation into its context window and returns to Step 1.

This iterative loop terminates when the model determines, during its *Thought* phase, that sufficient context has been assembled and all required environment mutations have completed, at which point it emits its **Final Answer**. In the context of this thesis, ReAct serves as the orchestrator's primary control mechanism: it enables the agent to interactively negotiate deployment specifications (e.g., requesting an alternative port from the user if initial validation against Kubernetes policies fails), substantially mitigating hallucination risks by anchoring reasoning in real environment observations.

### 2.1.2. Tool Calling and Function Calling

The technical mechanism enabling an LLM to execute the *Action* phase of a ReAct loop is *Function Calling* (or *Tool Calling*). Prior to its standardization, extracting structured entities from unstructured text relied on brittle Regular Expressions (RegEx) or secondary classification layers. A user might prompt: "Deploy a redis database", but enforcing strictly parseable JSON outputs (e.g., `{"action": "deploy", "service": "redis"}`) was prone to schema violations, unwanted whitespace, or invalid escape sequences.

A qualitative leap occurred when major model providers and open-source communities (e.g., via Ollama and vLLM) introduced specialized instruction fine-tuning targeted explicitly at structured data generation and schema compliance.

Under contemporary *Tool Calling*, the application injects a structured schema (typically *JSON Schema*) into the model's system prompt alongside user input, defining available functions, descriptions, and strongly typed parameter definitions (integers, booleans, enums). When the model infers that a user request necessitates external action, it suspends conversational generation and returns a structured machine-readable payload. This payload instructs the hosting runtime to execute the designated routine on the server.

This capability has transformed the role of general-purpose programming languages in AI systems. Languages such as Python or Go are no longer merely HTTP client wrappers; they serve as operational execution interfaces for agents, granting them direct access to SQL databases, messaging channels, and, in the case of *Agentic Deployer*, container orchestration APIs. The determinism achieved through modern *Tool Calling* is the critical enabler that allows engineering teams to entrust DevOps workflows to stochastic models.

## 2.2. Integration Standardization: Model Context Protocol (MCP)

Although *Tool Calling* endowed language models with execution capabilities, its initial adoption spawned a fragmented software ecosystem. Major AI providers (OpenAI, Google, Anthropic) each introduced proprietary specifications for tool registration. Concurrently, open-source orchestration frameworks (such as LangChain, LlamaIndex, and AutoGen) established incompatible abstraction layers.

This fragmentation led to classic vendor lock-in. If an infrastructure operations team developed a suite of deployment tools tailored to LangChain and OpenAI, migrating to local open-weights models (e.g., Ollama or vLLM) to comply with data privacy policies required rewriting the tool integration layer. Maintaining redundant adapters across evolving model families proved unsustainable. **Figure 1** illustrates this $N \times M$ integration bottleneck contrasted against the universal bus architecture established by standardized context protocols.

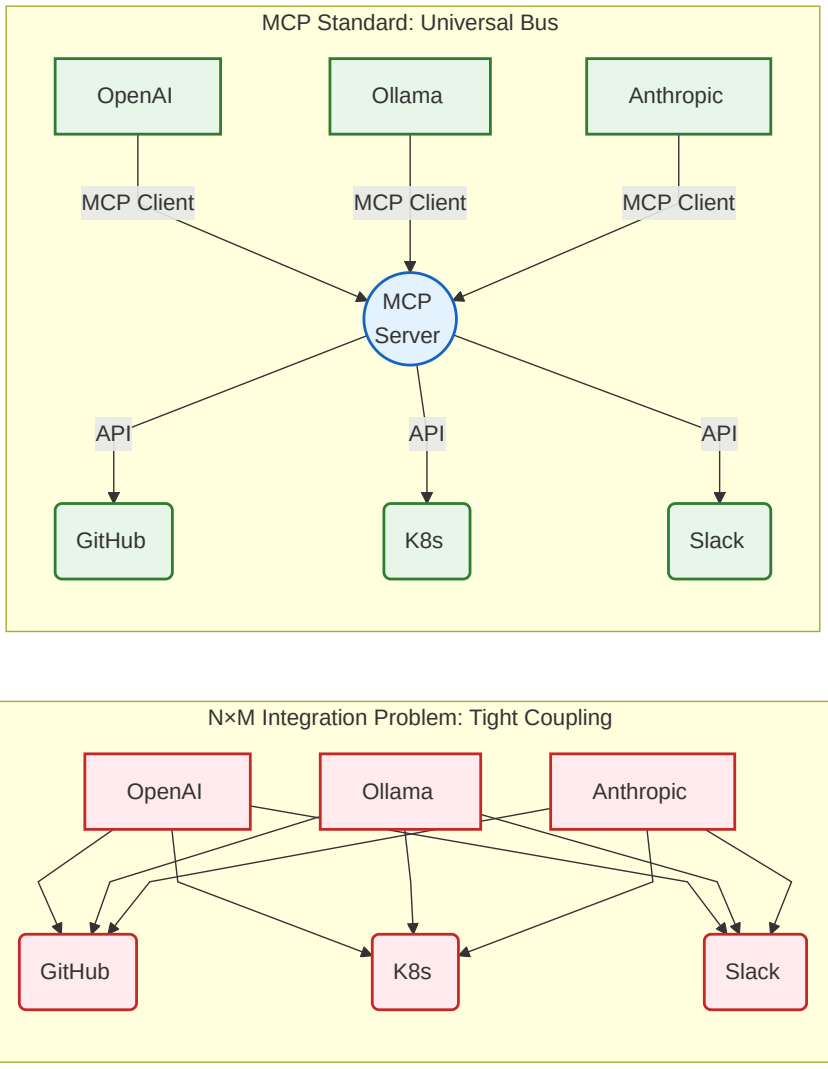



***Figure 1:*** *Architectural contrast between the N×M integration problem (proprietary coupling) and the Universal Bus topology proposed by the Model Context Protocol (MCP).*

As illustrated in Figure 1, the traditional topology (left) suffers from combinatorial growth: connecting $N$ cognitive models (OpenAI, Ollama, Anthropic) directly to $M$ infrastructure tools (GitHub, Kubernetes, Slack) requires implementing and maintaining $N \times M$ tightly coupled, proprietary adapters. Conversely, adopting the MCP standard (right) introduces a universal bus architecture: cognitive clients implement a single standardized interface (*MCP Client*), while infrastructure services expose capabilities through a shared server (*MCP Server*). This mediation reduces integration complexity to linear order $O(N + M)$, decoupling agent lifecycles from infrastructure tool evolutions.

## 2.2.1. Genesis and Principles of MCP

To resolve this architectural fragmentation, in late 2024 Anthropic led the release of an open-source standard: the **Model Context Protocol (MCP)** [13]. The core motivation of MCP is to strictly decouple AI consumers (agents) from data sources and local infrastructure tools, establishing a universal interaction language.

MCP draws direct architectural inspiration from Microsoft's *Language Server Protocol (LSP)*, which standardized communication between code editors (IDEs) and language compilers. Similarly, MCP establishes a standard layer connecting any generative model with enterprise computational environments, promoting a pluggable architecture built upon lightweight `JSON-RPC 2.0` messaging.

## 2.2.2. Client-Server Architecture of MCP

The protocol replaces monolithic integrations with a formal client-server architecture, establishing strict security boundaries:

- **MCP Host / Client (The Agent):** The application interacting with the user and coordinating LLM reasoning. The client remains agnostic to tool implementations; its role is to communicate with the server via standard messages, inject retrieved context into the model prompt, and dispatch tool execution requests selected by the AI. In this project, the `AgentOrchestrator` component implements the MCP client logic.
- **MCP Server (The Environment):** An independent, lightweight process hosted within the organization's trusted network zone (operated by university IT administrators). The server operates independently of LLM reasoning; its sole purpose is to encapsulate imperative business logic (e.g., generating Kubernetes YAML manifests) and expose these operations through standardized JSON schemas.

Communication between client and server occurs via standardized transports. MCP supports direct local communication via standard operating system input/output streams (**stdio**), ideal for tools co-located on the same host as the agent, as well as network transport via **Server-Sent Events (SSE)** over HTTP, essential for distributed and cloud-native deployments.

## 2.2.3. Protocol Primitives: Resources, Prompts, and Tools

The MCP specification structures model-environment interactions around three core primitives:

1. **Resources:** Read-only data sources controlled by the server. They supply contextual data to the LLM's context window without granting mutation permissions. Examples include retrieving cluster state, database schemas, or internal documentation.
2. **Prompts:** Parameterized prompt templates stored on the server to enforce organizational consistency. An administrator can define standardized prompts (e.g., "Summarize cluster node health"), which the client retrieves and renders for users.
3. **Tools:** Executable routines with side effects (environment mutations) governed by the server. Tools represent the standardized equivalent of Function Calling. Upon initialization, the server advertises its tool catalog (e.g., `deploy_python_app`) along with required argument schemas (such as project name and language runtime version).

Adopting the Model Context Protocol via its official Python SDK forms the architectural backbone of *Agentic Deployer*. This decision ensures that institutional infrastructure automations remain decoupled from rapid evolutions in AI provider SDKs. The tools exposed by the project's MCP server can be consumed interchangeably by the local agent interface (Streamlit + Ollama) and external third-party tools (such as the official MCP Inspector or commercial IDEs), ensuring long-term maintainability and robust interoperability.

Simultaneously, the industry is exploring parallel standardization initiatives such as the **Agent Protocol** championed by the AI Engineer Foundation [18], which aims to establish a uniform REST API for communicating with autonomous agents. While the Agent Protocol addresses agent-to-agent and client-to-agent interactions, MCP addresses agent-to-environment tool integration. Both frameworks highlight the maturation of standardized protocols in autonomous computing.

# 2.3. Declarative Infrastructure and Orchestration (Kubernetes and OpenShift)

Understanding automated infrastructure provisioning requires analyzing the operational model established by Kubernetes (K8s) [5] and enterprise distributions such as Red Hat OpenShift and OKD. Originally developed at Google as the internal *Borg* system and later donated to the Cloud Native Computing Foundation (CNCF), Kubernetes has become the de facto standard for container orchestration. Its widespread adoption is rooted in its declarative management paradigm. In university datacenters and enterprise environments, this model is frequently deployed on **OpenShift / OKD** clusters, which package Kubernetes core components with hardened multi-tenant security controls (*SecurityContextConstraints*) and lifecycle operators. Standardizing the IDP on universal Kubernetes primitives guarantees native compatibility across both upstream Kubernetes and downstream OpenShift platforms.

## 2.3.1. Imperative vs. Declarative Management

Traditional systems administration relied predominantly on imperative workflows. Operators managed infrastructure through explicit command sequences (e.g., shell scripts executed against remote servers). Under an imperative model, the operator defines *how* tasks must be carried out step by step: "download this image, start the container, bind port 80, verify process health". While straightforward, this approach is fundamentally fragile. If a host restarts or a process crashes, imperative scripts lack runtime context to restore normal operations without manual intervention.

In contrast, Kubernetes adopts a declarative Infrastructure as Code (IaC) model. In a declarative paradigm, engineers do not prescribe operational procedures; instead, they specify the **Desired State** of the system. Using structured YAML or JSON manifests, the operator declares: "Maintain exactly 3 replicas of the WordPress container exposed on port 8080".

## 2.3.2. Reconciliation Loops and the Control Plane

Translating desired state into actual infrastructure is the responsibility of the Kubernetes Control Plane, operating via asynchronous **Control Loops** (or Reconciliation Loops).

A Kubernetes cluster continuously monitors network topologies and active processes. Asynchronously, it compares the **Actual State** (e.g., "only 2 WordPress pods are running because a worker node crashed") against the **Desired State** stored in its distributed key-value store (`etcd`). Upon detecting a divergence (2 actual pods vs. 3 desired pods), the reconciliation loop automatically calculates necessary remediation actions (scheduling a new pod on an available healthy node) and executes them without requiring human intervention.

## 2.3.3. Syntactic Complexity and Cognitive Load

Despite the clear resilience and self-healing benefits of declarative management, authoring desired state configurations imposes significant cognitive overhead. The Kubernetes API encompasses dozens of native resource kinds (Pods, Deployments, Services, Ingresses) and supports domain extensions via Custom Resource Definitions (CRDs), each adhering to strict schema validation rules.

Deploying a production web service requires authoring and interconnecting multiple distinct primitives:

1. **Deployment:** Manages the container lifecycle, base image tags, restart policies, health probes (*liveness* and *readiness*), and hardware resource requests and limits.

2. **Service:** Abstrates ephemeral container IP addresses by establishing a stable virtual endpoint (*ClusterIP*) to load-balance traffic across pods.
3. **Ingress:** Manages external HTTP/HTTPS routing into internal Services, handling host-based routing rules and SSL/TLS termination.

The interdependencies among these manifests require exact syntactic coordination (such as matching label selectors between Deployments and Services). A minor YAML indentation error or mismatched label key causes the workload to deploy into an unreachable network silo without triggering overt API error messages.

To illustrate this complexity, Listing 1 provides the minimum compliant manifest structure required to expose a Python API in Kubernetes. Each component is independent, yet all must remain strictly aligned:

```
# 1. Deployment: defines container specifications, replicas, and resource boundaries
apiVersion: apps/v1
kind: Deployment
metadata:
  name: python-api           # Unique resource identifier within the cluster
  namespace: sic-research    # Departmental namespace
spec:
  replicas: 2                # Number of concurrent active instances
  selector:
    matchLabels:
      app: python-api        # (A) Deployment matches Pods carrying this label
  template:
    metadata:
      labels:
        app: python-api      # (A) Pods inherit this label for selector matching
    spec:
      containers:
        - name: api
          image: harbor.universidad.edu/sic-research/python-api:1.2.0  # Audited enterprise image;
:latest forbidden
          ports:
            - containerPort: 8080
          resources:
            requests: { cpu: "100m", memory: "128Mi" }   # Guaranteed allocation
            limits:   { cpu: "500m", memory: "512Mi" }   # Maximum ceiling (IT ResourceQuota)
---
# 2. Service: creates a stable network abstraction over ephemeral Pod IPs
apiVersion: v1
kind: Service
metadata:
  name: python-api-svc
  namespace: sic-research
spec:
  selector:
    app: python-api          # (A) EXACT match to Deployment label; any mismatch
                             # breaks routing silently without explicit error output
  ports:
    - port: 80
      targetPort: 8080       # Forwards traffic from port 80 to container port 8080
---
# 3. Ingress: exposes the Service externally with host-based HTTP routing
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: python-api-ingress
  namespace: sic-research
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "true"  # Enforces HTTPS
spec:
  rules:
    - host: api.investigacion.univ.es
      http:
        paths:
          - path: /
```

```
            pathType: Prefix
            backend:
              service:
                name: python-api-svc   # Reference to Service defined above
                port:
                  number: 80
```

***Listing 1:*** *Complete Kubernetes manifest (Deployment + Service + Ingress) for a Python API. The three resources must coordinate precisely—especially the label designated by (A)—for the application to be accessible externally.*

Analyzing this manifest demonstrates that deploying containerized workloads requires coordinating four critical architectural dimensions:

1. **Declarative Coupling via Label Selectors:** Kubernetes traffic routing operates via decoupled labels. The *Deployment* manages the lifecycle of pods labeled with `app: python-api` (annotated as `(A)`), while the *Service* uses `spec.selector` to dynamically discover matching pods and maintain its internal routing table (*Endpoints*). Any lexical or typographical mismatch causes the Kubernetes API to accept the manifest as syntactically valid; however, the *Service* resolves zero endpoints, causing a silent failure where no deployment exception is thrown yet the application remains completely unreachable.
2. **Network Topology and Transport-Layer Port Mapping:** A multi-tier port mapping hierarchy is established. The ingress controller receives external HTTP traffic on standard port `80` and routes it to the *Service* port. In turn, the *Service* translates traffic to `targetPort: 8080`, which must match the port on which the containerized API listens (`containerPort: 8080`). Restricting ports to values $\ge 1024$ complies with least-privilege principles, avoiding root user privileges inside the container's network namespace.
3. **Resource Scheduling and Governance (*Requests and Limits*):** Defining CPU and memory `requests` and `limits` is mandatory in multi-tenant clusters. Minimum guaranteed values (`requests`) inform scheduling decisions made by `kube-scheduler`, while hard ceilings (`limits`) mitigate noisy-neighbor contention and prevent runaway processes from triggering out-of-memory kernel terminations (*OOMKilled*) on adjacent workloads. These parameters must strictly adhere to the departmental `ResourceQuota` enforced in `sic-research`.
4. **Host-Based Ingress Resolution:** External exposure depends on strict naming alignment: the HTTP routing rule references `backend.service.name`, which must match the identifier of the target *Service* (`python-api-svc`). Any mismatch prevents ingress controllers from synthesizing valid forwarding rules.

This intricate dependency graph illustrates the cognitive burden that direct infrastructure management places upon non-specialized researchers and developers. Rather than requiring users to master low-level declarative syntax, *Agentic Deployer* serves as an intelligent intermediary that infers and validates these relationships from high-level natural language prompts.

For empirical comparison, **Annex A (Section A.2, Listing 2)** provides the manifest autonomously generated by *Agentic Deployer* during a live deployment test (Scenario 1). That artifact demonstrates the agent's ability to synthesize label mappings, configure liveness and readiness health probes, inject non-privileged runtime security contexts (`securityContext` with dropped capabilities), and attach HITL governance metadata.

# 2.4. Hexagonal Architecture and Security in Stochastic Systems

Integrating Large Language Models into critical infrastructure environments presents a fundamental architectural dilemma. By their statistical nature, LLMs are stochastic and non-deterministic. In contrast, enterprise IT provisioning requires absolute determinism (e.g., rejecting any deployment attempting to expose privileged ports or untrusted registries). To reconcile generative agility with strict infrastructure compliance, *Agentic Deployer* adopts the principles of **Hexagonal Architecture**.

## 2.4.1. Foundations of Ports and Adapters

Formalized by Alistair Cockburn [19] under the name *Ports and Adapters*, Hexagonal Architecture emerged to resolve the limitations of traditional layered architectures (Presentation $\rightarrow$ Business Logic $\rightarrow$ Database). In layered designs, business logic often becomes contaminated with transitive dependencies on database drivers or UI presentation frameworks.

The central goal of Hexagonal Architecture is to enable an application to be driven equally by users, external programs, automated test suites, or batch scripts, and to be developed and tested in total isolation from concrete runtime infrastructure and databases.

Represented conceptually as a hexagon, the architecture establishes a clean boundary between two zones:

- **The Core (Domain):** Encapsulates pure business rules, domain invariants, and data models. It possesses zero external dependencies; it imports no HTTP libraries, database drivers, or AI vendor SDKs.
- **The Outer Layer (Adapters):** Contains concrete technological implementations (REST APIs, database repositories, OpenAI client libraries, Kubernetes cluster adapters).

## 2.4.2. Dependency Inversion (The SOLID 'D' Principle)

Communication between the immutable Core and volatile Adapters is governed by **Ports**. A Port is an abstract interface or protocol contract.

The structural integrity of this pattern relies on the strict application of the Dependency Inversion Principle (DIP). Rather than allowing the business core to depend on a concrete Kubernetes client to execute deployments, the core defines an abstract `DeployPort` contract (e.g., `def deploy(intent: DeploymentIntent) -> str`). The outer adapter layer implements this contract. The core remains aware only of the abstract interface, never of concrete implementations.

## 2.4.3. Sandboxing the Stochastic AI Engine

Adopting Hexagonal Architecture provides the foundation for system security in this thesis. By defining domain entities (such as `DeploymentIntent`) within the pure application core using strongly typed Pydantic models, the LLM is structurally isolated from physical infrastructure adapters.

In this topology, the AI agent acts as a **Primary (Driving) Adapter**, formulating intents that invoke application use cases. Because of hexagonal boundaries, the language model has no direct visibility into or access to physical infrastructure adapters (**Secondary or Driven Adapters**), preventing prompt injection attacks or hallucinations from reaching the cluster unmediated. Every intent emitted by the AI must cross an inbound Port, where it undergoes deterministic policy inspection

by the domain rule engine (`SecurityContextValidator`).

If the model proposes a configuration violating institutional security policies, the core rejects it immediately by raising a strongly typed domain exception. This exception does not degrade system availability; rather, it flows back to the driving adapter, where the orchestrator captures it and re-injects the error description into the LLM's context window as structured feedback (the ReAct feedback loop).

Thus, Hexagonal Architecture functions as a deterministic validation sandbox, harnessing generative reasoning while withholding unmediated execution privileges over production infrastructure.

# 2.5. Related Works and Differential Positioning

The convergence of conversational agents and infrastructure automation has produced several academic and commercial initiatives in recent years. This section analyzes key approaches and establishes the differential positioning of *Agentic Deployer*.

## 2.5.1. Commercial Cloud Assistants and Copilots

Major cloud hyperscalers have launched conversational assistants embedded within their platforms. Microsoft provides **GitHub Copilot for Azure** and the **Azure AI Assistant** within its management portal. Google offers **Gemini for Google Cloud**, and Amazon Web Services provides **Amazon Q Developer**.

While powerful, these commercial solutions exhibit significant structural limitations for institutional and academic environments:

- **Vendor Lock-In:** They operate exclusively within their respective proprietary clouds, preventing unified governance across hybrid or on-premise Kubernetes clusters.
- **Data Sovereignty Violations:** Operational metadata and prompts are transmitted to commercial cloud datacenters, conflicting with European and institutional data protection regulations (GDPR, ENS).
- **Unsupervised Execution Risks:** Many cloud copilots allow direct execution without a mandatory, immutable Human-in-the-Loop approval gate.
- **Unpredictable Operational Costs (OPEX):** Their commercial models rely on per-token consumption or recurring per-seat subscriptions, introducing budgetary volatility unacceptable in public sector institutions.
- **Rigid Policy Customization:** Their generation templates are generic and cannot natively enforce localized departmental quotas, naming conventions, or customized security baselines.

## 2.5.2. Research Approaches: LLMs for IaC, ChatOps, and General-Purpose Frameworks

In academic research, multiple studies have explored applying language models to infrastructure engineering:

- **Automated Infrastructure as Code (IaC) Synthesis:** Recent empirical studies investigating LLM-generated IaC security [11] evaluate models generating Terraform and Kubernetes manifests from natural language. Large-scale quantitative assessments of LLM-assisted IaC development [12] find that while models achieve high syntactic completeness, they frequently suffer from subtle networking and storage hallucinations, as well as cumulative

configuration drift that necessitates strict external validation. Similarly, cluster security frameworks like *GenKubeSec* [20] demonstrate the utility of LLMs for misconfiguration detection and automated remediation; however, the authors emphasize that unmediated automated remediation without human oversight introduces prohibitive operational risks in production. These studies confirm that delegating security and semantic correctness directly to LLMs without a deterministic verification layer is insufficient: models produce syntactically valid manifests that remain semantically vulnerable (e.g., unapproved ports or unvetted container registries).

- **ChatOps Integration for DevOps Pipelines:** Leppänen et al. [21] analyze real-world conversational tool integration within CI/CD pipelines in *"The Highways and Country Roads to Continuous Deployment"*. While conversational assistants in platforms like Slack or Teams improve collaboration, existing implementations typically execute commands directly without an intermediate strongly typed domain model or a formally verified state machine.
- **Demarcation from General-Purpose Agent Frameworks:** It is essential to distinguish this work from generic agent libraries such as **LangChain** [22] or **AutoGen** [23]. These frameworks provide baseline utilities for chaining model invocations at a lower software layer. Conversely, *Agentic Deployer* is an end-to-end infrastructure platform: it avoids volatile external framework dependencies by implementing a pure Hexagonal Architecture, strict domain typing, and standardized interoperability based on the open Model Context Protocol (MCP).

## 2.5.3. Differential Positioning of Agentic Deployer

*Agentic Deployer* occupies a distinct position at the intersection of agentic automation and Platform Engineering. Unlike proprietary cloud copilots operating as black boxes or experimental systems relying on unconstrained IaC generation, the architecture presented in this thesis establishes a **formal separation between the cognitive reasoning plane (LLM) and the physical execution plane (Hexagonal Architecture)**.

Table 1 summarizes this multidimensional comparison between proprietary cloud assistants and *Agentic Deployer*:

| Dimension | Proprietary Cloud Assistants | Agentic Deployer |
|---|---|---|
| **Deployment Target** | Bound to vendor's proprietary cloud | Any Kubernetes cluster (on-premise, hybrid, or multi-cloud) |
| **Data Sovereignty** | Inferences and telemetry processed in cloud provider datacenters | *Zero Data Retention* (local inference via Ollama or private cloud APIs) |
| **Human Supervision (HITL)** | Optional or reactive; direct execution permitted | Mandatory and preventive; immutable FSM gate cannot be bypassed |
| **Golden Paths** | Generic templates managed by cloud provider | Organizationally audited Golden Paths and custom security policies |
| **Interoperability** | Closed proprietary APIs | Open universal standard (MCP, interoperable with any client) |
| **Operational Cost** | Variable per-token or per-user OPEX | Self-hosted infrastructure with zero marginal inference cost |

| Dimension | Proprietary Cloud Assistants | Agentic Deployer |
|---|---|---|
| **Auditability and Governance** | Proprietary logs subject to external retention policies | Full local ACID transactional audit trail persisted on disk |

***Table 1:*** *Multidimensional comparison between proprietary cloud assistants and Agentic Deployer.*

In conclusion, *Agentic Deployer* prioritizes technological sovereignty, deterministic security, and operational auditability over SaaS convenience, providing public institutions and enterprise organizations with a robust, economically sustainable foundation for integrating intelligent agents into critical infrastructure operations.

# Chapter 3. Methodology and Technology Stack

Constructing an orchestration platform that bridges classical Software Engineering practices (Hexagonal Architecture, rigorous automated testing, REST APIs) with emerging, inherently stochastic domains such as Generative Artificial Intelligence requires a structured methodological framework. This chapter describes the development methodology adopted throughout the lifecycle of *Agentic Deployer*, the scheduled development phases, and the architectural rationale behind the chosen technology stack.

## 3.1. Methodological Approach

Software engineering tailored to Artificial Intelligence differs significantly from conventional web or enterprise application development. In deterministic systems, input $A$ (e.g., clicking a button) invariably yields output $B$. In a system orchestrated by a Large Language Model (LLM), input $A$ (a natural language prompt) can yield multiple syntactically divergent yet semantically equivalent outputs. This variance necessitates establishing foundational practices that strictly decouple deterministic business constraints from stochastic reasoning.

To govern this risk, this Master's Thesis adopted an **iterative and incremental development methodology**, guided by the principles of *Domain-Driven Design* (DDD) and *Test-Driven Development* (TDD). Rather than constructing the system horizontally (designing the database schema, followed by backend endpoints, and concluding with frontend views), the methodology enforced a concentric, inside-out progression aligned with Hexagonal Architecture principles.

The central methodological rule was **core stability prior to stochastic integration**: not a single line of integration code for the OpenAI API, Ollama, or the MCP SDK was authored until the internal domain model (`DeploymentIntent`) and rule engine (`SecurityContextValidator`) demonstrated 100% deterministic reliability under extensive automated test suites. Introducing AI integrations before stabilizing validation invariants would have exposed the system to severe security vulnerabilities during early development stages.

## 3.2. Development Phases

To realize the final product, the execution schedule was structured into **five core development sprints** complemented by two transversal consolidation phases (Phases 6 and 7, detailed in Section 4.2 and Annex B). Each sprint produced a functional, test-verified software increment serving as an immutable foundation for subsequent phases.

> *Methodological Note: The following sections reference architectural design artifacts (such as `DeploymentIntent`, `SecurityContextValidator`, or `DeployPort`) that are formally detailed in **Chapter 5**. They are introduced here to establish chronological context across development sprints.*

> *Planning Note: While the initial roadmap defined a sequential sprint progression, the reality of academic research and concurrent graduate commitments necessitated flexible phase overlap. Project tracking and sprint retrospective analyses are documented in **Chapter 4** and **Annex B**.*

### 3.2.1. Sprint 1: Hexagonal Core and Domain Baseline

This phase established the project's foundation. The objective was to formally model university IT operational rules in a strictly deterministic manner, independent of AI integrations:

- **Entity Modeling:** Defined the deployment domain ontology. The `DeploymentIntent` value object was implemented with strict static typing, enforcing essential provisioning constraints (application name, base container image, network port, hardware resource quotas).
- **Validation and Use Cases:** Developed `SecurityContextValidator`, a deterministic rule engine designed to intercept invalid or malicious requests (e.g., privileged system ports $< 1024$ or unapproved container registries).
- **Contracts (Ports):** Established the abstract deployment port (`DeployPort`), defining the interface contract that downstream infrastructure adapters (Kubernetes, Docker Swarm, Nomad) must satisfy.

## 3.2.2. Sprint 2: HITL Operations Backend and FSM

Before enabling autonomous action capabilities in the AI agent, operational oversight controls were implemented:

- **REST API Layer:** Built the primary driving adapter via a FastAPI HTTP server to ingest deployment intents in a structured format.
- **Quarantine Workflow:** Implemented transitional state logic. Intents validated by the domain engine were not executed immediately; instead, they were held in a quarantined `PENDING_APPROVAL` state.
- **Operator Dashboard:** Designed and implemented an asynchronous web dashboard enabling IT administrators to inspect quarantined intents and execute final authorization decisions (`POST /hitl/approve`), closing the *Human-In-The-Loop* (HITL) architectural loop.

## 3.2.3. Sprint 3: Golden Paths and FakeK8s Adapter

The third increment introduced declarative artifact generation. While direct execution against live production clusters was scoped out of the MVP baseline, the orchestrator needed to demonstrate end-to-end synthesis of valid infrastructure configurations:

- **Emulation Adapter:** Implemented `FakeK8sAdapter`, fulfilling the `DeployPort` interface contract.
- **Declarative Materialization:** Created templated *Golden Paths*. Upon receiving an approved intent, the adapter extracts runtime variables (application name, sizing specifications) and writes complete, cross-linked Kubernetes YAML manifests (`Deployment`, `Service`, `Ingress`) to disk, verifying that the system generates production-grade manifests while eliminating manual authoring complexity.

## 3.2.4. Sprint 4: ReAct Agent and MCP SDK Integration

With a validated, deterministic infrastructure core capable of generating Kubernetes manifests, the fourth sprint integrated the cognitive reasoning layer, exposing a natural language interface to end users:

- **MCP Protocol Implementation:** Integrated the official *Model Context Protocol* SDK, refactoring IT capabilities (e.g., `deploy_python_app`) into standardized tools exposed over the `stdio` transport.
- **ReAct Engine:** Developed `AgentOrchestrator`, an iterative reasoning loop communicating with models via a provider-agnostic adapter (`LLMClient`). The engine parses model responses, executes backend MCP tools, and re-injects observations into the context window (establishing the security feedback loop).

### 3.2.5. Sprint 5: Advanced QA (Metamorphic and Property-Based Validation)

Added as an advanced software quality layer to verify system resilience against probabilistic model behavior:

- Integrated *Property-Based Testing* via Hypothesis, generating thousands of randomized deployment intents to evaluate validation boundary conditions.
- Formalized Metamorphic Testing to assess LLM semantic stability under linguistic perturbations (lexical noise, word-order transformations, prompt mutations).
- Executed *Mutation Testing* with Mutmut to audit test suite fault-detection effectiveness against logical mutant injections.

### 3.2.6. Phase 6: Post-MVP Consolidation (Hardening and HITL Lifecycle Closure)

Following MVP stabilization, several usability and architectural refinements were implemented:

- **Native Ollama Client (`OllamaLLMClient`):** Replaced the generic OpenAI SDK wrapper for local Ollama communication with direct asynchronous `httpx` calls against Ollama's native REST endpoints. This refactoring guarantees true *Zero Data Retention* without third-party cloud dependencies.
- **Bidirectional User Notification Channel:** Implemented the `GET /hitl/status/{id}` polling endpoint and corresponding Streamlit UI notification widget (`My Pending Requests`), closing the asynchronous feedback loop back to the researcher.
- **Empirical Evidence Artifacts:** Populated the `demos/` directory with complete JSON traces from live sessions and generated Kubernetes YAML manifests (Annex A).

### 3.2.7. Phase 7: Experimental Extension and Multi-Model Benchmarking

As a final research expansion (Priority P2), the prototype was extended across three technical dimensions:

- **ACID Transactional Persistence:** Migrated runtime state from in-memory structures to an on-disk **SQLite** database, providing durable transaction logging for the finite state machine.
- **MCP Tool Catalog Scaling:** Expanded the server catalog from 4 to 7 operational tools, incorporating database provisioning and workload teardown capabilities.
- **Empirical Multi-Model Evaluation:** Executed comparative benchmarks across cloud models (Gemini, Cohere, Groq) and local specialized models (Qwen Coder), consolidating feasibility and performance conclusions.

## 3.3. Technology Stack and Architectural Rationale

Technology selection in an architecture combining Artificial Intelligence with enterprise infrastructure must balance generative flexibility with deterministic reliability. The following sections outline the rationale behind the selected stack.

## 3.3.1. Core Programming Language: Python 3.12

The project is built entirely on **Python (version 3.12)**. While languages like Go dominate the cloud-native ecosystem (Kubernetes and Terraform are written in Go), Python remains the lingua franca of AI research and systems development. Python 3.12 delivers significant runtime performance optimizations and advanced static typing support (`TypeVar`, `Protocol`, generics). These capabilities are essential for establishing rigorous abstract interfaces in Hexagonal Architecture, mitigating dynamic typing risks in critical systems.

## 3.3.2. Backend and Domain Modeling

- **FastAPI:** Selected over legacy web frameworks (Django, Flask) due to its high-performance asynchronous runtime (built on ASGI and Starlette) and native OpenAPI/Swagger schema generation. For an LLM agent, interacting with an API that produces rigorous, self-describing schemas significantly improves tool comprehension.
- **Pydantic (v2):** Serves as the domain validation core [24]. Rewritten in Rust for optimal execution speed, Pydantic is utilized to enforce domain model invariants (`DeploymentIntent`). It guarantees business constraints (e.g., port boundary conditions and regex matching) prior to application logic execution.

## 3.3.3. Conversational Interface and Agent Orchestration

- **Streamlit [25]:** Developing dedicated single-page applications in React or Vue introduces substantial frontend overhead. Streamlit enables authoring reactive conversational interfaces—including chat message histories, status avatars, and HITL notification panels—entirely in Python. This streamlined visual iterations, allowing development effort to remain focused on infrastructure middleware engineering.
- **Model Context Protocol (MCP) SDK:** Rather than maintaining proprietary HTTP tool interfaces, the official Python MCP SDK was adopted. This library provides decorators (e.g., `@server.tool()`) that introspect function signatures and type annotations at runtime, transforming them into standardized JSON schemas consumable by compliant LLM clients.
- **Provider-Agnostic LLM Abstraction:** The orchestrator defines an abstract contract (`LLMClient`) supporting two concrete implementations:
    - **`OllamaLLMClient`:** A native client built using Ollama's official Python SDK [26] communicating with the local daemon on port 11434 (`POST /api/chat`). It enables self-hosted execution of models like **`qwen2.5-coder:7b`** on institutional hardware, guaranteeing strict **Data Sovereignty** (*Zero Data Retention*) with zero prompt data leaving institutional networks.
    - **`OpenAILLMClient`:** An adapter implementing the standard OpenAI interface, used as a gateway to empirically evaluate external cloud models (Google Gemini, Cohere Command-R, Mistral, Llama 3) without coupling domain logic to a single provider. Model provider selection is controlled via the `LLM_PROVIDER` environment variable.

## 3.3.4. Quality Assurance (QA) Ecosystem

Operational trust in *Agentic Deployer* is anchored in a multi-layered automated verification pipeline:

- **Pytest:** Baseline testing framework for unit and integration suites, offering robust fixture dependency injection.

- **Hypothesis:** Property-Based Testing framework that injects pseudo-randomized inputs to test boundary conditions across domain validators.
- **Mutmut:** Mutation testing tool that introduces logical mutations into source AST nodes to empirically evaluate test suite fault-detection coverage.
- **Locust and Playwright:** Utilized for non-functional verification. *Locust* executes concurrency stress tests against backend endpoints. *Playwright* orchestrates end-to-end (E2E) browser interactions to verify operator workflows in the HITL dashboard.
- **Ruff and Mypy:** Static analysis tools (*Linting* and *Type-Checking*) enforced in local verification to maintain code style consistency and enforce static type safety.

**Figure 2** summarizes the component topology of the technology stack, illustrating the clear separation between presentation, cognitive reasoning, the restrictive backend, and the quality assurance harness.

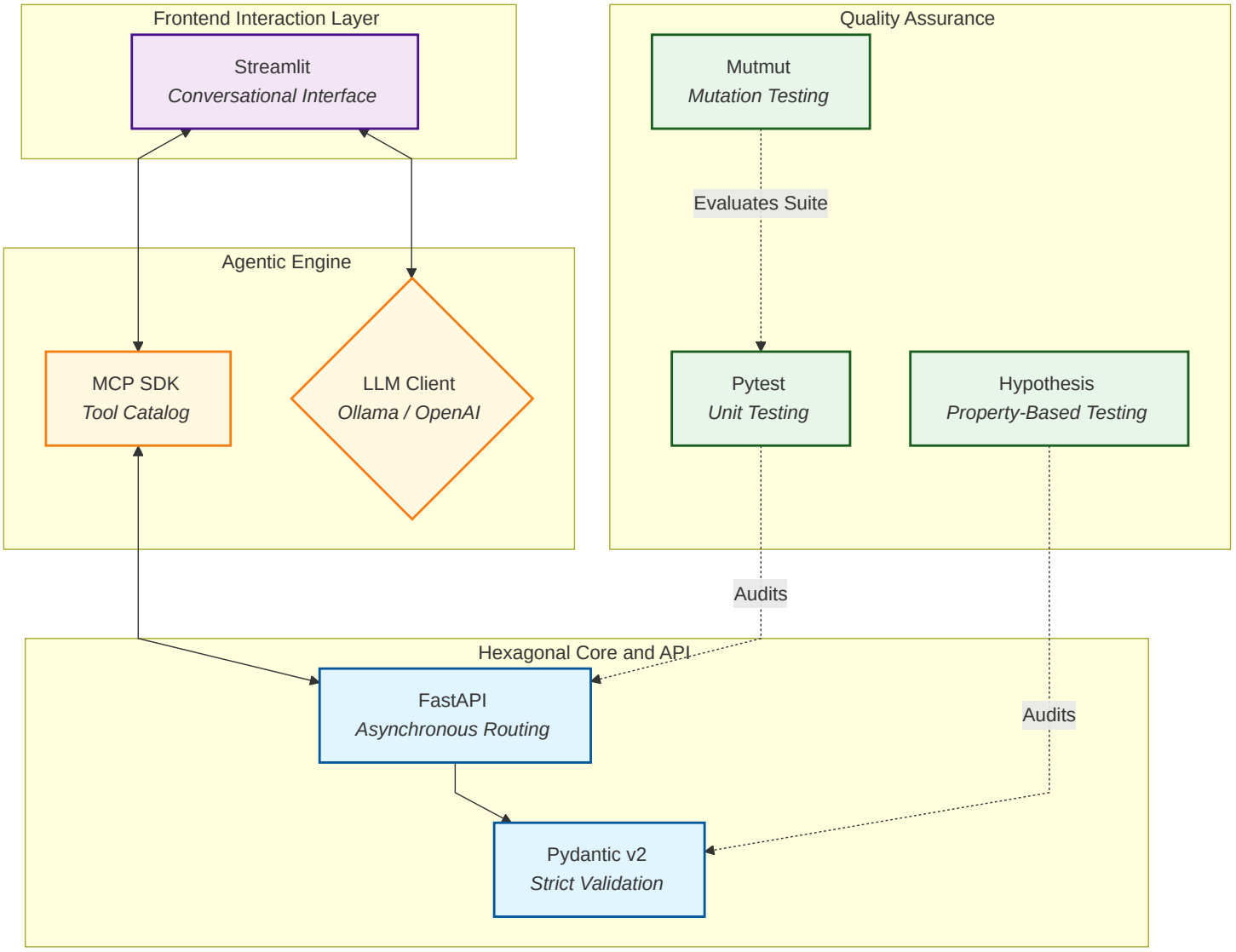



***Figure 2:*** *Component diagram of the technology stack, showing separation across the presentation layer, cognitive reasoning, restrictive backend, and continuous quality assurance tools.*

The component topology shown in Figure 2 establishes four decoupled tiers:

1. **Frontend Interaction Layer (`UI`):** The Streamlit conversational interface (`ST`) handles user interactions. It

maintains bidirectional communication with the cognitive engine (`LLM`) for dialogue generation and with the MCP server (`MCP`) for tool discovery and execution.

2. **Agentic Engine (`Cognition`):** Encapsulates the *ReAct* loop. The LLM client (`LLM`, configurable between local Ollama and cloud APIs) resolves user intentions using tools exposed by the MCP SDK (`MCP`), which serves as a standardized bridge to backend services.
3. **Hexagonal Core and API (`Backend`):** The FastAPI server (`FA`) exposes asynchronous endpoints for intent dispatching (`/mcp/intent`) and operator review (`/hitl`). Requests are mapped to Pydantic v2 models (`PY`), deterministically enforcing domain invariants, typing rules, and security policies prior to state mutation.
4. **Continuous Quality Assurance (`QA`):** Advanced testing tools operate across critical stack components: Pytest (`PYT`) verifies FastAPI endpoints and use cases; Hypothesis (`HYP`) evaluates Pydantic validation boundaries via Property-Based Testing; and Mutmut (`MUT`) measures test suite efficacy via AST mutation analysis.

# Chapter 4. Project Management and Viability

Constructing a platform that bridges classical Software Engineering practices with emerging disciplines like Generative Artificial Intelligence requires a structured yet adaptable management framework. This chapter documents project constraints, the sprint structure, effort tracking and deviation analyses, and an economic model assessing both development expenditures and institutional adoption viability. Structuring this chapter immediately after the methodology aligns with formal software engineering principles: constraints, sprint schedules, and development budgets are established prior to detailed architectural design. Furthermore, the institutional adoption viability study and projected Return on Investment (ROI) are formulated here as an ex-ante analytical model based on operational hypotheses, whose empirical validation is evaluated in Chapter 9 (Experimental Results and Evaluation). Detailed retrospective analyses per sprint are documented in **Annex B**.

---

## 4.1. Project Plan — Context and Constraints

### 4.1.1. Structural Constraints

*Agentic Deployer* was developed in the context of an academic Master's Thesis under the following constraints:

- **Workload Dedication:** Part-time research allocation (~15–20 hours per week, maintained concurrently with external professional employment).
- **Project Team:** 1 graduate researcher + 1 academic advisor (periodic supervisory sessions, scheduled according to mutual availability).
- **Infrastructure:** Local development environment (personal workstation), incurring zero cloud hosting expenditures during development.
- **Methodology:** Iterative and incremental development centered on domain invariants (*Domain-First*).

### 4.1.2. Flexibility Principle and Adaptive Prioritization

> **Note on Planning Character:** The planning presented herein is **indicative and orientative**, rather than rigidly prescriptive. Because this thesis was conducted on a part-time basis alongside professional commitments, sprint temporal boundaries were treated as permeable: execution could be accelerated, deferred, or overlapped based on researcher availability and advisor review cadences.

This approach complies with agile management recommendations for applied R&D projects within academic frameworks (IEEE Std 12207, ISO/IEC 29110), where rigid schedules are replaced by **explicit goal prioritization**: the Minimum Viable Product (MVP) is secured first, and optional higher-complexity modules are undertaken based on remaining available time.

**Project Priority Hierarchy:**

| Priority | Module | Status |
|---|---|---|
| **P0 — Mandatory MVP** | Hexagonal core, security validator, REST API, FSM, HITL Dashboard | Completed |

| Priority | Module | Status |
|---|---|---|
| **P1 — Academic Differentiator** | MCP server (official SDK), ReAct agent, multi-provider LLM abstraction, advanced QA | Completed |
| **P2 — Engineering Excellence** | `OllamaLLMClient`, SQLite persistence, extended 7-tool catalog, empirical traces (MCP Inspector), multi-model cognitive experiments | Completed |
| **P3 — Future Work** | `RealK8sAdapter` (physical cluster integration), RBAC, cloud CI/CD, multi-cluster management | Roadmap |

***Table 2:*** *Development modules and priority hierarchy.*

Only modules **P0 and P1 were strictly mandatory for academic degree evaluation**. Modules P2 were addressed during the final consolidation period, elevating the technical robustness of the system. Modules P3 are documented as future research directions (Section 10.3).

## 4.1.3. Stakeholder Identification

In alignment with software engineering methodologies (ISO/IEC 29110), identifying key stakeholders bounds operational expectations, technical responsibilities, and system acceptance criteria. Five key roles participate in the project lifecycle:

- **Graduate Researcher / Lead Developer (Jesús Solís Ortega):** Responsible for end-to-end technical execution, encompassing requirements elicitation, C4 architectural modeling, hexagonal core and MCP server development, HITL interface design, automated testing pipelines, and technical thesis drafting.
- **Academic Advisor (Manuel Resinas Arias de Reyna):** Director of the Master's Thesis. Responsible for continuous methodological supervision, research guidance, conceptual design validation, and technical review gates.
- **Academic Examination Committee (EIP Evaluation Board):** Academic tribunal responsible for formal evaluation and defense grading, adhering to University of Seville assessment rubrics.
- **IT and Communications Department (SIC):** Institutional benchmark stakeholder for the primary case study. Represents corporate system operators and cluster administrators, establishing security constraints (*SecurityContext*), resource quotas, and exercising final approval authority over deployment requests.
- **University Faculty and Researchers (End Users):** Academic workloads consumers (Python APIs, CMS platforms, conference web applications) who interact with the system via natural language without requiring Kubernetes administration expertise.

## 4.1.4. Communication and Project Tracking Plan

To maintain milestone compliance within flexible timelines, a continuous tracking plan was established via **Microsoft Teams** and institutional email:

1. **Supervisory Review Sessions:** Periodic direction and milestone meetings held via Microsoft Teams videoconferencing, focusing on key deliverables: problem formalization and DSR framing (Sprint 1), MCP server architecture and LLM decoupling (Sprints 3 and 4), and QA audit strategies alongside empirical results (Sprint 5 and Phase 7).

2. **Continuous Communication Channel:** Asynchronous communication via institutional email and Microsoft Teams for progress updates, methodological inquiries, and technical blocker resolution.
3. **Iterative Document Review:** Modular document delivery through institutional channels across major thematic blocks (theoretical background, architecture and implementation, experimental evaluation, and conclusions), ensuring structured feedback prior to formal submission.

---

# 4.2. Sprint Structure and Effort Estimation

## 4.2.1. Project Scheduling Diagrams

The project roadmap was structured into **five thematic development sprints**, accompanied by a transversal documentation phase and a technical consolidation phase (illustrated in the Gantt charts of **Figure 3** and **Figure 4**), representing an initial estimated baseline of **~300 hours** ($\pm 15\%$ standard contingency buffer). Recorded actual effort at project completion was **~340 hours** (+13%); deviation analyses are detailed in §4.3.

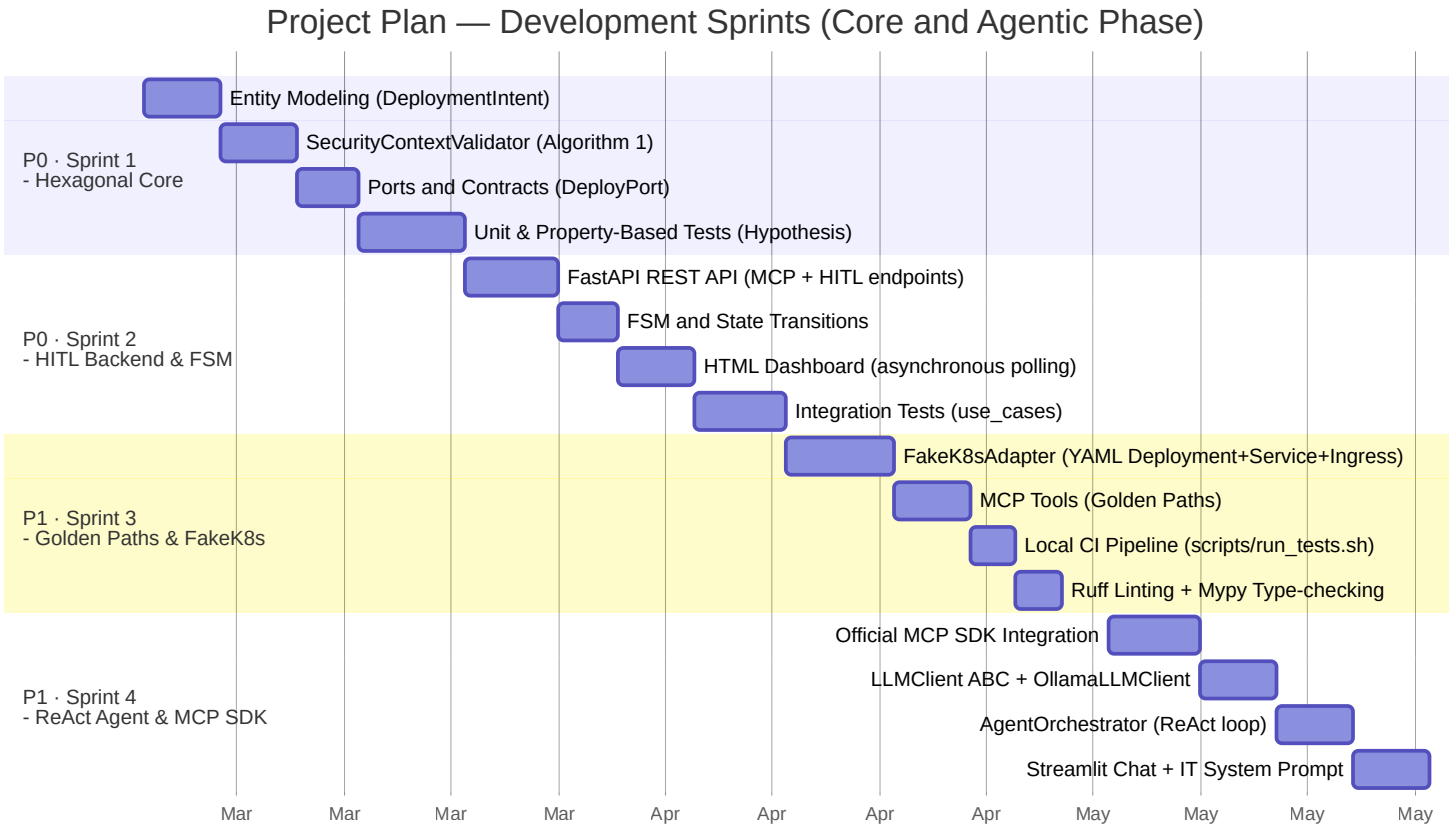



***Figure 3:*** *Gantt chart (Part 1). Indicative schedule for Sprints 1 through 4 (core and agent).*

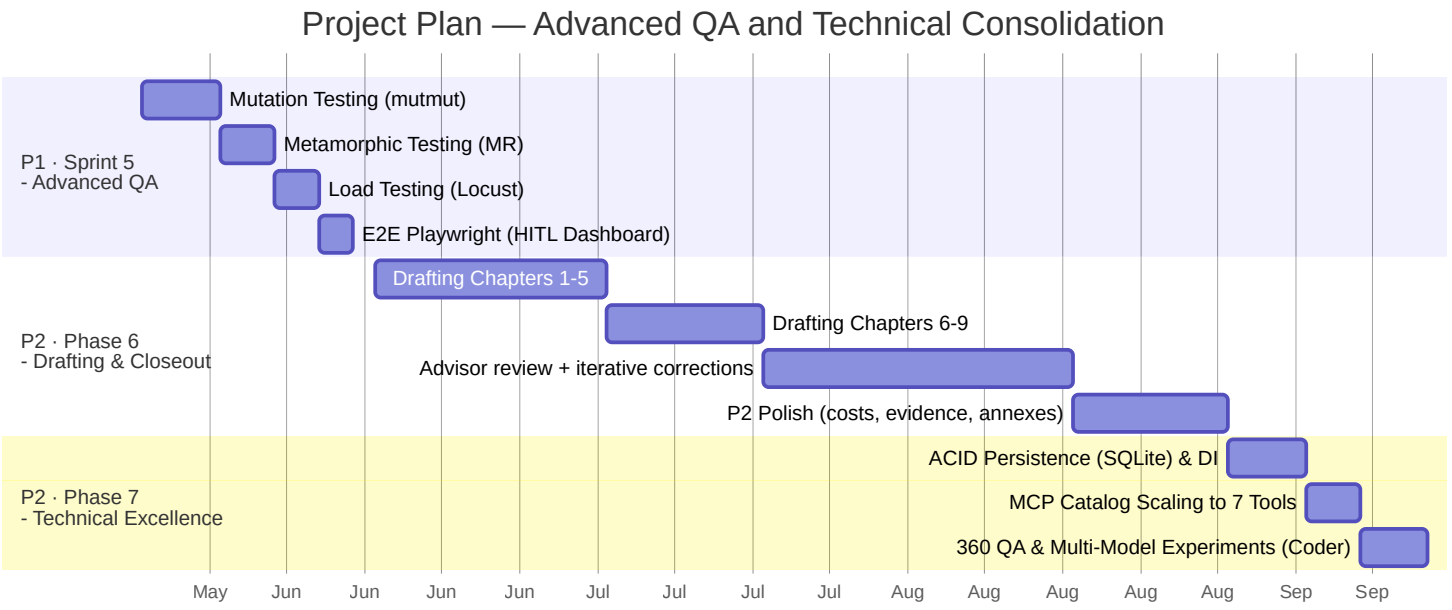



***Figure 4:*** *Gantt chart (Part 2). Indicative schedule for advanced QA and closeout/consolidation phases.*

#### 4.2.1.1. Timeline Analysis and Critical Path

The Gantt charts in Figures 3 and 4 illustrate project distribution over a six-month window (March to August 2026), structuring critical path execution and priority assignments:

1. **MVP Construction Horizon (Figure 3 · Sprints 1 to 4):** Spans March to May 2026. This initial phase enforces a strictly linear critical path: Sprints 1 and 2 (Priority P0) focus on the hexagonal core and deterministic backend. Completing these foundations was a mandatory prerequisite for Sprint 3 (adapters and YAML templates) and Sprint 4 (agentic orchestration and MCP), ensuring AI capabilities were built upon an already functional deterministic boundary.
2. **Verification, Documentation, and Consolidation Horizon (Figure 4 · Sprint 5, Phases 6 and 7):** Spans late May to August 2026. This block transitioned to concurrent dynamics: Sprint 5 (advanced QA) subjected the MVP to mutation and metamorphic testing before freezing the codebase. Concurrently, Phase 6 (drafting and closeout) delivered thesis chapters in modular batches alternating with advisory review cycles (§4.1.2), concluding with Phase 7 technical enhancements (SQLite ACID persistence and comparative benchmarks).

## 4.2.2. Effort Estimation per Sprint

Table 3 summarizes baseline net effort estimations, assuming part-time dedication and the incorporation of generative AI programming assistance:

| Sprint | Phase | Estimation (h) | Range ($\pm 15\%$) | Priority |
|---|---|---|---|---|
| Sprint 1 | Hexagonal Core | 50h | 43–58h | P0 |
| Sprint 2 | HITL Backend & FSM | 45h | 38–52h | P0 |
| Sprint 3 | Golden Paths & FakeK8s | 40h | 34–46h | P1 |

| Sprint | Phase | Estimation (h) | Range ($\pm 15\%$) | Priority |
|---|---|---|---|---|
| Sprint 4 | ReAct Agent & MCP SDK | 55h | 47–63h | P1 |
| Sprint 5 | Advanced QA | 45h | 38–52h | P1 |
| Phase 6 | Drafting & Closeout | 55h | 47–63h | P2 |
| Phase 7 | Technical Excellence | 10h | 8–12h | P2 |
| **Total Estimated** | | **~300h** | **~255–345h** | |

***Table 3:*** *Net effort estimation per Sprint.*

> **Note on AI Assistant Usage:** Employing a coding assistant (Google Gemini Advanced, utilized for rapid scaffolding, boilerplate generation, logic cross-checking, and prose polishing) accelerated development phases that would have otherwise demanded substantially greater effort. This aligns directly with the core thesis premise regarding LLM utility in complex technical workflows.

## 4.3. Development Effort and Deviation Analysis

Compared to the initial baseline of ~300 hours, empirical project closeout recorded **~340 hours**, representing a deviation of **+13%**. This additional effort remained comfortably within the anticipated contingency boundary ($\pm 15\%$) and was deliberately invested during the consolidation phase to elevate prototype quality (implementing SQLite ACID persistence, expanding MCP tools from 4 to 7, analyzing surviving mutants, and conducting multi-model cognitive experiments).

Generative AI assistance functioned as an effective productivity multiplier, allowing a single researcher to navigate dense architectural requirements within an academic schedule.

Table 4 summarizes effort deviations across all phases:

| Phase | Estimated (h) | Actual (h) | Deviation | Primary Driver |
|---|---|---|---|---|
| Sprint 1 · Hexagonal Core | 50 | ~55 | +10% | Security validator expansion |
| Sprint 2 · HITL Backend & FSM | 45 | ~45 | 0% | Development matched baseline |
| Sprint 3 · Golden Paths & FakeK8s | 40 | ~46 | +15% | Dual MCP SDK integration requirements |
| Sprint 4 · ReAct Agent & MCP SDK | 55 | ~62 | +13% | ReAct loop tuning and native Ollama client |
| Sprint 5 · Advanced QA | 45 | ~50 | +11% | Mutation testing survivor analysis |

| Phase | Estimated (h) | Actual (h) | Deviation | Primary Driver |
|---|---|---|---|---|
| Phase 6 · Drafting & Closeout | 55 | ~64 | +16% | Technical density and document length (~160 pages vs. ~100 anticipated) |
| Phase 7 · Technical Excellence | 10 | ~18 | +80% | Database persistence, full QA 360, multi-model evaluation |
| **Total** | **~300** | **~340** | **~+13%** | |

***Table 4:*** *Summary of time deviations per phase.*

> **Clarifying Note on Schedule Deviations:** A deviation of **+13%** against initial estimates falls well within the $\pm 15\%$ contingency threshold. This deliberate investment of ~40 additional hours occurred in Phase 7 to deliver enterprise-grade finishes. Detailed retrospective analyses per sprint, including completed milestones and lessons learned, are provided in **Annex B**.

---

# 4.4. Change Control: Traceability and Scope Evolution

In compliance with software engineering best practices and academic assessment rubrics (*Change Control* section), this section formally documents architectural adaptations made to initial project proposals.

**Initial Registered Baseline:**

- *Title:* «Design and Implementation of an Internal Developer Platform (IDP) on OpenShift Governed by AI Agents via the MCP Protocol».
- *Summary:* Development of a Platform Engineering middleware automating application lifecycle management on OpenShift/OKD environments. Built under Hexagonal Architecture to expose Infrastructure as Code capabilities (OpenTofu [10]) to AI agents governed by the Model Context Protocol (MCP), reducing developer cognitive load through standardized conversational interactions.

During development iterations (Sprints 1 to 5), ongoing research and empirical findings motivated three decisive engineering refinements:

1. **Infrastructure Target (OpenShift/OKD $\leftrightarrow$ Native Kubernetes):**
    - *Decision:* Red Hat OpenShift and OKD build upon the standard Kubernetes orchestration core. Rather than coupling middleware to proprietary OpenShift APIs (`DeploymentConfig`, `Route`), the platform standardizes artifact generation on universal declarative Kubernetes primitives (`Deployment`, `Service`, `Ingress`, `SecurityContext`).
    - *Justification:* This choice ensures native compatibility on OpenShift/OKD clusters (which execute standard manifests seamlessly) while eliminating vendor lock-in, enabling deployment across any Cloud Native distribution (K3s, EKS, GKE, or bare-metal).
2. **Provisioning Abstraction (From OpenTofu to Native Declarative Golden Paths):**

- *Decision:* Early proposals envisioned the agent synthesizing raw OpenTofu scripts [10] (IaC in HCL syntax). Contemporary literature on LLM code generation (Siddiq & Santos [11]; Tarasova et al. [12]) demonstrated that unconstrained HCL synthesis introduces severe hallucination risks, networking misconfigurations, and state drift.
- *Justification:* In alignment with Platform Engineering best practices (Spotify, Gartner [7, 8]), the system adopted **Golden Paths**. The AI does not author raw infrastructure; it infers high-level parameters from natural language prompts, delegating manifest instantiation to pre-validated, strongly typed templates in the hexagonal core, guaranteeing compliance.

3. **Governance Model: Integrating Human-in-the-Loop (HITL) Controls and FSM:**
    - *Decision:* Implemented an immutable Finite State Machine (FSM) and an asynchronous operations dashboard for IT personnel.
    - *Justification:* To satisfy enterprise security mandates and neutralize prompt injection vulnerabilities [27], no action proposed by the AI can mutate cluster state without prior explicit authorization from an authorized human operator.

Table 5 provides the formal traceability matrix comparing the initial proposal against the final implemented platform:

| Dimension | Initial Proposal | Final Implementation | Justification and Technical Benefit |
|---|---|---|---|
| **Core Paradigm** | Internal Developer Platform (IDP) | Internal Developer Platform (IDP) | **100% Maintained.** Self-service abstraction reducing cognitive load. |
| **Architecture** | Hexagonal Architecture (*Ports & Adapters*) | Hexagonal Architecture (*Ports & Adapters*) | **100% Maintained.** Business logic strictly isolated from external dependencies. |
| **AI Protocol** | Model Context Protocol (MCP) | Model Context Protocol (MCP) | **100% Maintained.** Official FastMCP SDK with stdio and SSE transports. |
| **Compute Target** | OpenShift / OKD | Kubernetes Core / OpenShift / OKD | Standardized on universal K8s APIs, ensuring compatibility without vendor lock-in. |
| **Provisioning** | IaC via OpenTofu [10] / Terraform | Declarative Native Golden Paths (YAML) | Eliminates HCL code hallucinations [11, 12]; enforces deterministic templates. |
| **Governance** | Unmediated Agent Execution | Human-in-the-Loop Supervision (FSM) | Enforces enterprise security against hallucinations and Prompt Injection attacks. |

***Table 5:*** *Change control and traceability matrix compared to initial proposal.*

# 4.5. Economic Development Analysis

The project was implemented using open-source tools and personal hardware, resulting in negligible direct out-of-pocket costs. However, evaluating economic viability requires calculating the **equivalent opportunity cost**: the capital expenditure required if the project were commissioned in the Spanish technology market at 2026 rates.

## 4.5.1. Human Resources Costs

| Role | Rate / Hour | Hours | Equivalent Opportunity Cost |
|---|---|---|---|
| **Graduate Researcher** (Junior Engineer — market equivalent) | 22 €/h | ~340h | ~7,480 € |
| **Academic Advisor** (Senior Lead / R&D Supervisor) | 75 €/h | ~45h *(periodic sessions + documentation review)* | ~3,375 € |
| **Human Resources Subtotal** | | **~385h** | **~10,855 €** |

***Table 6:*** *Human Resources costs (equivalent CAPEX).*

> **Methodological Note:** Figures represent equivalent market opportunity costs: the expenditure required if executed under professional contract. The student receives no direct remuneration; the advisor is compensated institutionally outside this calculation.

## 4.5.2. Infrastructure and Tooling Costs

| Resource | Provider | Attributable Cost | Calculation / Rationale |
|---|---|---|---|
| **Development Laptop** | Personal Hardware | **~126 €** | Laptop ~1,200 € · 4-year lifecycle · project fraction (6/48 months) $\approx$ 150 € · part-time factor ~84% $\approx$ **126 €** |
| Operating System & Utilities | Linux (Ubuntu) | 0 € | Open source |
| Python 3.12 & Ecosystem | Open source | 0 € | — |
| FastAPI, Pydantic, Hypothesis, Pytest | Open source | 0 € | — |
| Official MCP SDK (`mcp==2.2.0`) | Anthropic (MIT License) | 0 € | Open source |

| Resource | Provider | Attributable Cost | Calculation / Rationale |
|---|---|---|---|
| Ollama (Local LLM Server) | Open source | 0 € | Free open-weights models |
| **AI Assistant (Google Gemini Advanced)** | Google | **~120 €** | 20 €/month $\times$ 6 months. Utilized for scaffolding, boilerplate generation, and prose review |
| OpenAI API (Testing & Validation) | OpenAI | ~15 € | Development test credits |
| **Infrastructure & Tooling Subtotal** | | **~261 €** | |

***Table 7:*** *Infrastructure and Tooling Costs (Development Phase).*

### 4.5.3. Total Development Expenditures

| Category | Cost |
|---|---|
| Human Resources (Opportunity Cost) | ~10,855 € |
| Infrastructure & Tooling | ~261 € |
| **Total Project Development Cost** | **~11,116 €** |
| **Effective Cost per Hour** | **~28.9 €/h** |

***Table 8:*** *Subtotal and total costs of development phase.*

## 4.6. Corporate Adoption Costs and ROI Analysis

This section addresses the strategic question: **What is the capital expenditure and recurring operational cost to adopt and operate *Agentic Deployer* within a production institutional environment**, such as a university IT department or corporate infrastructure team?

> **Methodological Note on Cost Sources:** All economic values in this section reflect market benchmarks as of **2026** and were derived from the following sources:
>
> - **API Pricing:** Official developer pricing pages (OpenAI API Pricing, Anthropic Console Pricing, Google AI Studio).
> - **Hardware (Local Ollama):** Based on official NVIDIA MSRP for the RTX 4000 series and B2B distributor averages.
> - **Cloud / VPS Infrastructure:** Public catalog pricing from IaaS providers (AWS EC2, Google Cloud Compute,

> Hetzner Cloud).
>
> - **Electricity Costs:** Calculated assuming an average institutional industrial rate of ~0.15 €/kWh in Spain, referencing Eurostat and Red Eléctrica de España market reports.
> - **Salaries (Hourly Technical Cost):** Estimated between 40 €/h and 60 €/h (total employer cost), aligned with Hays Spain and Michael Page compensation surveys for *Platform Engineer* and *DevOps* roles.

## 4.6.1. Adopting Organization Profiles

| Profile | Description | Operational Complexity |
|---|---|---|
| **A — Small University / Small Enterprise** | <5,000 users, IT team of 1–3 engineers, basic local infrastructure | Low |
| **B — Mid-Sized University / Medium Enterprise** | 5,000–30,000 users, structured IT team, on-premise/hybrid K8s | Medium |
| **C — Large University / Enterprise** | >30,000 users, multi-cluster topology, dedicated platform team, strict GDPR audit | High |

***Table 9:*** *Adopting Organization Profiles (Cases A, B, and C).*

## 4.6.2. Implementation Costs (CAPEX — Initial Investment)

| Component | Profile A | Profile B | Profile C |
|---|---|---|---|
| **Code Customization** *(customizing `SYSTEM_PROMPT`, MCP tools, and security policies)* | 20h $\times$ 40 €/h = **800 €** | 60h $\times$ 50 €/h = **3,000 €** | 160h $\times$ 60 €/h = **9,600 €** |
| **`RealK8sAdapter` Implementation** *(cluster integration via `kubernetes-client`)* | 20h $\times$ 40 €/h = **800 €** | 40h $\times$ 50 €/h = **2,000 €** | 80h $\times$ 60 €/h = **4,800 €** |
| **Configuration & Deployment** *(CI/CD, environment variables, SSL, LDAP/SAML)* | 10h $\times$ 40 €/h = **400 €** | 30h $\times$ 50 €/h = **1,500 €** | 60h $\times$ 60 €/h = **3,600 €** |
| **Operator Training (HITL)** | 4h $\times$ 1 engineer = **160 €** | 8h $\times$ 2 engineers = **800 €** | 12h $\times$ 4 engineers = **2,880 €** |
| **Total CAPEX** | **2,160 €** | **7,300 €** | **20,880 €** |

***Table 10:*** *Implementation Costs (CAPEX - Initial Investment).*

# 4.6.3. Annual Operating Expenditures (OPEX)

## Server Infrastructure

| Component | Profile A | Profile B | Profile C |
|---|---|---|---|
| Application Server (FastAPI + MCP) | VPS 4 vCPU / 8 GB $\approx$ **600 €/year** | Amortized on-prem server $\approx$ **300 €/year** | 3 cloud replicas $\approx$ **3,600 €/year** |
| Storage (Database + YAML manifests) | 50 GB SSD $\approx$ **60 €/year** | 200 GB $\approx$ **200 €/year** | 1 TB + backups $\approx$ **800 €/year** |

***Table 11:*** *Annual Infrastructure Operating Costs (OPEX).*

## LLM Engine — The Critical OPEX Driver

### Option 1 — Self-Hosted Ollama (Open-Source Models, Recommended for Public Institutions)

Local models eliminate recurring per-token fees in exchange for a one-time hardware investment. Based on empirical findings (Chapter 10), three corporate adoption tiers are established:

| Tier (Use Case) | Recommended Model | VRAM | Hardware Investment (One-off GPU) | API Cost | Privacy |
|---|---|---|---|---|---|
| **Basic** (MVP / Testing) | `qwen2.5-coder:7b` | 8 GB | ~400 € (e.g., RTX 4060) | **0 €/year** | Total (Zero Data Retention) |
| **Intermediate** (Balanced) | `qwen2.5-coder:14b` / `mistral-nemo:12b` | 16 GB | ~500 € (e.g., RTX 4060 Ti 16GB) | **0 €/year** | Total |
| **Enterprise** (Optimal Corporate) | `Qwen3-Coder-30B-A3B-Instruct` (MoE) | 24–48 GB | ~1,800 € – 5,000 € (RTX 3090/4090 or dual) | **0 €/year** | Total |

***Table 12:*** *Annual LLM Engine Operating Costs (Local vs Cloud) across adoption tiers.*

> **Technical Note on Scaling:** While early design assumptions posited that enterprise adoption would require 14B or 30B models to overcome reasoning limitations, empirical benchmarks with `qwen2.5-coder:7b` demonstrate high reliability even on basic 8 GB VRAM hardware (~400 €). Consequently, scaling to Intermediate or Enterprise tiers is primarily reserved for large institutions (Profile C) supporting high concurrency workloads.

### Option 2 — Commercial Cloud APIs

If an organization elects cloud inference, the `OpenAILLMClient` adapter seamlessly interfaces with cloud endpoints. Table 13 models estimated annual operational expenses based on token consumption:

| Provider | Model | Input Price | Output Price | Estimated Annual Cost* |
|---|---|---|---|---|
| OpenAI | GPT-4o-mini | 0.15 $/MTok | 0.60 $/MTok | **~160–650 €/year** |
| OpenAI | GPT-4o | 2.50 $/MTok | 10.00 $/MTok | **~2,700–11,000 €/year** |
| Anthropic | Claude Haiku | 0.25 $/MTok | 1.25 $/MTok | **~290–1,150 €/year** |
| Google | Gemini Flash | 0.075 $/MTok | 0.30 $/MTok | **~80–330 €/year** |

***Table 13:*** *Estimated LLM Engine Costs (Cloud API).*

> *Estimated for 50–200 daily requests, assuming **~50,000 average tokens per request** (90% input context, 10% output). In ReAct loops, context accumulates cumulatively across iterations, and Kubernetes manifest dumps often exceed 3,000 tokens per tool call.

## Consolidated Total Annual OPEX

| Component | Profile A | Profile B | Profile C |
|---|---|---|---|
| Application Server | 600 €/year | 300 €/year | 3,600 €/year |
| Storage | 60 €/year | 200 €/year | 800 €/year |
| GPU Electrical Consumption (Local)* | ~20 €/year | ~40 €/year | ~100 €/year |
| Local Ollama LLM (One-off Hardware, Year 0) | +400 € (Basic Tier) | +500 € (Intermediate Tier) | +2,000 € (Enterprise Tier) |
| Cloud LLM API (Alternative: GPT-4o)** | ~2,700 €/year | ~5,500 €/year | ~11,000 €/year |
| Maintenance & Upgrades | 10h $\times$ 40 €/h = **400 €/year** | 20h $\times$ 50 €/h = **1,000 €/year** | 40h $\times$ 60 €/h = **2,400 €/year** |
| **Total OPEX (with Ollama)** | **~1,080 €/year** | **~1,540 €/year** | **~6,900 €/year** |
| **Total OPEX (with Cloud API)** | **~3,760 €/year** | **~7,000 €/year** | **~17,800 €/year** |

***Table 14:*** *Total Annual OPEX consolidated by Adoption Profile.*

> *Electrical power costs are low because the GPU remains idle (15–30W) ~99% of the day, peaking only during inference execution bursts.
> **Production cloud options require resilient models (GPT-4o class), scaling with profile volume (50, 100, and 200 daily requests respectively).

## 4.6.4. Return on Investment (ROI) Analysis

Operational value is quantified based on workload execution time reductions documented in Chapter 9 (Section 9.5, Table 35).

**Annual Operational Savings Calculation — Profile B (Mid-sized University, 500 requests/year):**

| Automated Workflow | Net Time Saved / Request | Annual Requests | Technician Rate | Annual Savings |
|---|---|---|---|---|
| Requirements Negotiation (Triage) | ~25 min (active effort) | 500 | 50 €/h | **~10,417 €** |
| Manual YAML Translation | ~15 min | 500 | 50 €/h | **~6,250 €** |
| Policy Compliance Validation | ~5 min | 500 | 50 €/h | **~2,083 €** |
| **Total Annual Operational Savings** | | | | **~18,750 €** |

***Table 15:*** *Annual operational savings calculation (Profile B scenario).*

> **Note on Active Effort vs. Elapsed SLA:** The ~25 minutes saved refers strictly to **technician active engineering time** (analyzing, clarifying, and re-validating specifications), not total elapsed ticket resolution duration (which spans ~24 business hours due to asynchronous email round-trips). Savings derive from subtracting interactive agent dialogue (~5 min, Table 35) from standard manual triage effort (~30 min).

**Three-Year ROI — Profile B with Self-Hosted Ollama (Basic Coder 7B Tier):**

| Concept | Year 0 | Year 1 | Year 2 | Year 3 |
|---|---|---|---|---|
| CAPEX Investment | -7,300 € | — | — | — |
| LLM Hardware (One-off GPU) | -400 € | — | — | — |
| Annual OPEX | — | -1,540 € | -1,540 € | -1,540 € |
| Operational Savings | — | +18,750 € | +18,750 € | +18,750 € |
| **Net Cash Flow** | **-7,700 €** | **+17,210 €** | **+17,210 €** | **+17,210 €** |
| **Cumulative Cash Flow** | -7,700 € | +9,510 € | +26,720 € | +43,930 € |

***Table 16:*** *3-Year Return on Investment (ROI) (Profile B with Ollama).*

> **Payback Period: ~5.4 months** following initial deployment.
> **3-Year ROI: ~475%** (calculated over Year 0 + Year 1 consolidated outlay of 9,240 €; or **~357%** over total three-year cumulative expenditure of 12,320 €).

### ROI Sensitivity Analysis

The most sensitive variable in the ROI model is technician active effort baseline during triage. Table 17 models sensitivity variations against the 30-minute central baseline:

| Scenario | Baseline Active Time (Triage) | Time Saved / Request | Total Annual Savings | 3-Year ROI |
|---|---|---|---|---|
| **Optimistic** (Pre-optimized SLA, well-documented intake) | ~10 min | ~5 min | **~10,417 €** | **~205%** |
| **Intermediate** (Industry standard process) | ~20 min | ~15 min | **~14,583 €** | **~340%** |
| **Central** (Thesis Reference Baseline) | ~30 min | ~25 min | **~18,750 €** | **~475%** |

***Table 17:*** *Sensitivity analysis of Return on Investment (ROI) across technician baseline active time variations.*

Even in the most conservative scenario (a baseline process requiring only 10 minutes of active triage), the system maintains a **205% 3-year ROI** with an estimated payback period of ~10 months, demonstrating financial viability across varying organizational efficiencies.

---

# 4.7. Conclusions of the Viability Analysis

1. **Schedule flexibility is an intentional design attribute, not an oversight.** Part-time academic execution precludes rigid Gantt constraints; explicit prioritization (P0 $\rightarrow$ P3) ensured that academic core requirements were fully met before exploring optional modules.
2. **Generative AI coding assistance is a proven productivity multiplier.** Leveraging a coding agent (Google Gemini Advanced, ~120 € attributable expenditure) enabled a single engineer to implement an ambitious software architecture (Hexagonal design, official MCP SDK, multi-layer QA, ReAct engine) within ~340 total hours.
3. **Local inference is economically superior for public sector institutions.** Pairing Ollama with on-premise hardware limits OPEX to ~1,540 €/year, eliminating recurring cloud token costs and guaranteeing Zero Data Retention compliance under GDPR and ENS mandates.
4. **An ROI exceeding 470% (~355% cumulative over 3 years) validates institutional adoption.** Capital recovery is achieved in ~5.4 months for environments processing at least 500 annual provisioning requests.
5. **Hexagonal Architecture safeguards long-term technology investments.** Swapping any adapter (`FakeK8sAdapter` $\rightarrow$ `RealK8sAdapter`, Ollama $\rightarrow$ GPT-4o) requires zero modifications to business logic, minimizing technology migration costs via the Dependency Inversion Principle.

# Chapter 5. System Design and Architecture

Designing a software architecture that bridges Generative Artificial Intelligence decision-making with enterprise infrastructure operations presents a fundamental engineering challenge: reconciling the probabilistic natural language flexibility of a Large Language Model (LLM) with the deterministic guarantees and strict predictability demanded by production environments.

This chapter details the technical resolution to this dichotomy, presenting an architecture that functions as a structured, deterministic translator. The following sections outline the global solution topology using the C4 modeling framework, justifying the segregation of responsibilities across execution containers. Subsequently, the chapter analyzes the logical foundation of the application core through Hexagonal Architecture (*Ports and Adapters*), formalizes *Golden Path* provisioning contracts, and details the end-to-end operational lifecycle of a request from raw natural language input to final declarative manifest generation.

## 5.1. Requirements Analysis and Architectural Design Criteria

In conventional software engineering, architectural decisions derive directly from requirements analysis. In an agentic system where stochastic language models interface with mission-critical operational environments, this analysis must strictly delineate what Artificial Intelligence may suggest versus what deterministic software may execute.

### 5.1.1. Traceability between Requirements and Architectural Decisions

Strategic objectives formulated in Section 1.3 map to the formal specification of Functional Requirements (FR) and Non-Functional Requirements (NFR) presented in Section 1.5. **Table 18** details the direct traceability between each requirement and corresponding architectural decisions:

| Identifier | Formal Requirement | Associated Architectural Decision | Responsible Component |
|---|---|---|---|
| **FR-01** | Natural language conversational interface | Presentation layer decoupled via asynchronous web client and token streaming. | Streamlit Frontend (`app/presentation/`) |
| **FR-02** | Agentic orchestration and fault tolerance | Adoption of ReAct (*Reason* + *Act*) loop with observation-conditioned reflective retries. | Agent Orchestrator (`app/agent_layer/agent.py`) |
| **FR-03** | Modular infrastructure tool catalog | Open formal contract implementation based on the standardized *Model Context Protocol* (MCP). | FastMCP Server (`app/mcp_server.py`) |

| Identifier | Formal Requirement | Associated Architectural Decision | Responsible Component |
|---|---|---|---|
| **FR-04** | Human oversight of critical operations (*HITL*) | Segregation of responsibilities via independent web dashboard and Finite State Machine (FSM) quarantine (`PENDING_APPROVAL`). | FastAPI Backend and HITL Dashboard (`app/adapters/api.py`) |
| **FR-05** | Asynchronous status notification channel | Decoupled request lifecycle query mechanism avoiding direct user-operator coupling. | SQLite Repository and periodic Streamlit polling |
| **NFR-01** | Data sovereignty and institutional privacy | Provider-agnostic engine abstraction via the *Adapter* pattern, enabling local execution with *Zero Data Retention*. | `OllamaLLMClient` adapter on local port 11434 |
| **NFR-02** | Domain isolation and independence | Hexagonal Architecture (*Ports and Adapters*), ensuring entities and invariants remain agnostic of AI, databases, and Kubernetes. | Domain and Application layers (`app/domain/`, `app/application/`) |
| **NFR-03** | Code quality and formal robustness | Multi-tiered testing strategy: static analysis (Ruff/Mypy), Property-Based Testing (Hypothesis), and Mutation Testing audit (Mutmut). | Quality assurance suite (`app/tests/`) |
| **NFR-04** | Operational resilience (*Fail-Safe*) | Strict deterministic boundary validation prior to cluster interaction; atomic state transitions. | `SecurityContextValidator` and FSM |

***Table 18:*** *Traceability between Formal Requirements and Architectural Decisions.*

## 5.1.2. Core Architectural Design Principles

Three core principles govern all architectural decisions detailed in this chapter:

1. **Principle of Execution Plane Segregation (Privilege Separation):** The Artificial Intelligence engine operates exclusively within the *Cognitive Suggestion Plane*. It possesses zero infrastructure credentials, network permissions to interact with Kubernetes APIs, or direct transactional capabilities. Every action generated by the agent must be validated by the *Deterministic Control Plane* (traditional software) and formally authorized by a human operator (*HITL*).
2. **Fail-Safe by Default Principle:** Upon encountering lexical ambiguity, security policy violations, or type mismatches, the system rejects operations without mutating cluster state (`HTTP 422`), returning structured observations to the agent to enable reflective self-correction.
3. **Open Interoperability and Technological Decoupling Principle:** The business core communicates externally

exclusively through formal interfaces (ports). The AI communicates via the open MCP standard, the inference engine connects via interchangeable adapters, and infrastructure generation is abstracted through declarative templates (*Golden Paths*).

### 5.1.3. Comprehensive Traceability Matrix (Requirements $\leftrightarrow$ Design $\leftrightarrow$ QA Validation)

In compliance with software engineering guidelines and degree assessment rubrics, **Table 19** provides the bidirectional traceability matrix connecting formal requirements to design use cases and corresponding test suites:

| Requirement | Name and Scope | Design Use Case / Workflow | Responsible Component | Validation Module (QA) |
|---|---|---|---|---|
| **FR-01** | Natural conversational UI | UC-01: Natural language request input | Streamlit Frontend and API | `tests/test_ui.py` |
| **FR-02** | ReAct agent orchestration | UC-02: Reflective loop & self-correction after 422 | Agent Orchestrator (`app/agent_layer/agent.py`) | `app/tests/test_agent.py` |
| **FR-03** | Modular MCP catalog | UC-03: Standardized discovery and invocation | FastMCP Server (`app/mcp_server.py`) | `app/tests/test_mcp_server.py` |
| **FR-04** | Human supervision (HITL) | UC-04: Quarantine and asynchronous IT approval | FastAPI Backend & Dashboard (`app/adapters/api.py`) | `app/tests/test_use_cases.py` |
| **FR-05** | User feedback channel | UC-05: Status notification and inspection | Endpoint `/hitl/status/{id}` and SQLite | `app/tests/test_status_endpoint.py` |
| **NFR-01** | Institutional data sovereignty | Network isolation without third-party retention | `OllamaLLMClient` on local port | `app/tests/test_agent.py` |
| **NFR-02** | Domain isolation | Pure invariants without external dependencies | Hexagonal Core (`app/domain/`, `app/application/`) | `app/tests/test_use_cases.py` |
| **NFR-03** | Formal quality & robustness | Addressing oracle problem and mutant detection | Heterogeneous suite (Hypothesis, Metamorphic) | `app/tests/test_security.py`, `app/tests/test_metamorphic.py` |
| **NFR-04** | Operational resilience (Fail-Safe) | FSM immutability under invalid inputs | `SecurityContextValidator` & FSM | `app/tests/test_security.py` |

***Table 19:*** *Comprehensive Traceability Matrix (Requirements $\leftrightarrow$ Design $\leftrightarrow$ QA Validation).*

## 5.2. C4 Architectural Topology Modeling

Documenting modern distributed, asynchronous architectures using informal block diagrams or rigid traditional UML can introduce ambiguity. To ensure precise, hierarchical, and unambiguous architectural visualization, the **C4 Model** was adopted.

Conceived by Simon Brown [28], the C4 Model relies on hierarchical abstraction. It structures architectural views progressively from a macroscopic System Context view down to execution Containers, internal Components, and underlying Code.

In this architectural analysis of *Agentic Deployer*, focus is centered on the two most critical levels for understanding network boundaries and organizational responsibilities: Level 1 (System Context) and Level 2 (Containers).

# 5.2.1. Level 1: System Context

The System Context diagram (Level 1) establishes system boundaries, illustrating interactions between the platform, human users, and external enterprise systems.

Within the academic environment of this case study, four primary entities are modeled:

## 5.2.1.1. Human Actors

1. **Researcher / End User (Inbound Vector):** University faculty, researchers, or administrative staff. This actor requires computing services without possessing deep systems administration expertise. They express functional goals (e.g., "I need to publish a website for the Cellular Biology conference next week") in natural language, lacking knowledge of Kubernetes YAML manifests or cluster security policies. This user possesses no direct network credentials or write permissions against university infrastructure.
2. **IT Systems Engineer / SIC (Governance Vector):** Systems engineers from the IT and Communications Department (*Servicio de Informática y Comunicaciones*). This actor possesses administrative authority to apply infrastructure mutations. In *Agentic Deployer*, their role transitions from manual provisioning to automated auditing, acting as the human supervisor in the *Human-In-The-Loop* (HITL) pattern via a dedicated operator dashboard.

## 5.2.1.2. External Black-Box Systems

3. **LLM Provider:** Acts as the external cognitive reasoning engine. Depending on institutional data governance mandates, this node represents commercial cloud APIs (OpenAI `gpt-4o`, Anthropic `claude-3.5-sonnet`) or self-hosted open-weights models running via Ollama on local hardware (`qwen2.5:7b`). Architecturally, this provider operates as a stateless oracle: it receives context, performs probabilistic inference, and returns structured text or tool calls.
4. **Kubernetes Cluster:** The terminal destination of the operational pipeline. Represents physical or virtual cluster infrastructure running OpenShift, OKD, or Kubernetes. It hosts workloads, balances network traffic, and binds persistent volumes according to generated manifests.

*Agentic Deployer* sits at the center of these entities, acting as a trusted mediator: ensuring researchers never directly access cluster APIs, and guaranteeing language models never hold credentials to mutate corporate infrastructure unmediated.

Figure 5 summarizes the C4 Level 1 Context topology:

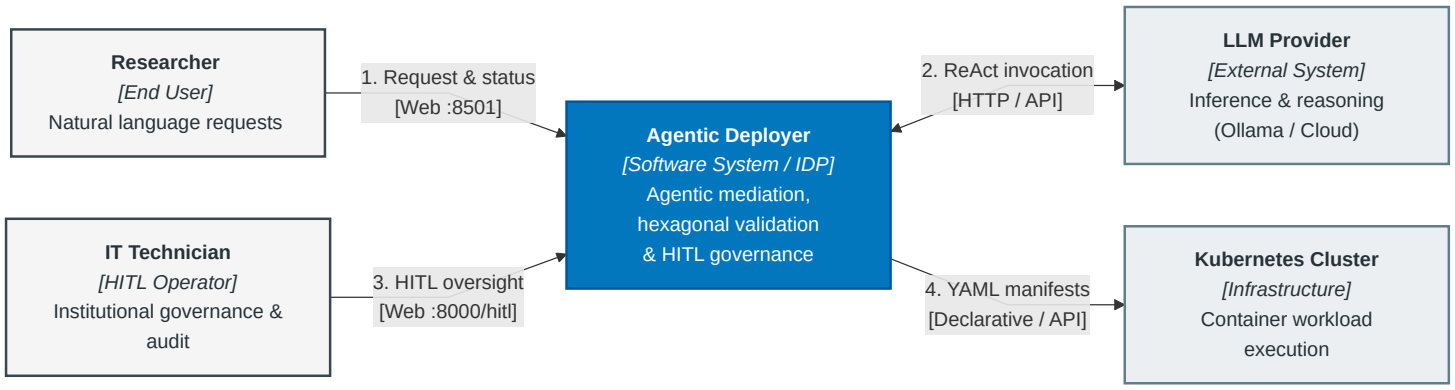



***Figure 5:*** *C4 System Context Diagram (Level 1) for Agentic Deployer.*

As illustrated in Figure 5, *Agentic Deployer* centralizes and decouples operational flows. The researcher interacts exclusively through natural language. The LLM provider supplies heuristic reasoning without network access to internal clusters. IT technicians govern operations through an independent human-in-the-loop channel. Finally, the cluster only receives declarative manifests after they pass both deterministic policy checks and explicit human authorization.

## 5.2.2. Level 2: Containers (Execution Units)

Level 2 decomposes *Agentic Deployer* into separate execution containers.

> **Note on C4 Terminology:** In C4 methodology, a "Container" represents an executable application or data store (e.g., a microservice, database, or client application). It is an architectural concept distinct from Docker OS-level virtualization containers.

Partitioning the platform into decoupled processes maximizes operational resilience [29] and enforces Separation of Concerns. If one component fails (e.g., an inference timeout), the rest of the platform continues operating to preserve data auditability.

Figure 6 illustrates the system decomposition across three execution containers (A, B, and C), mapping all seven operational interactions:

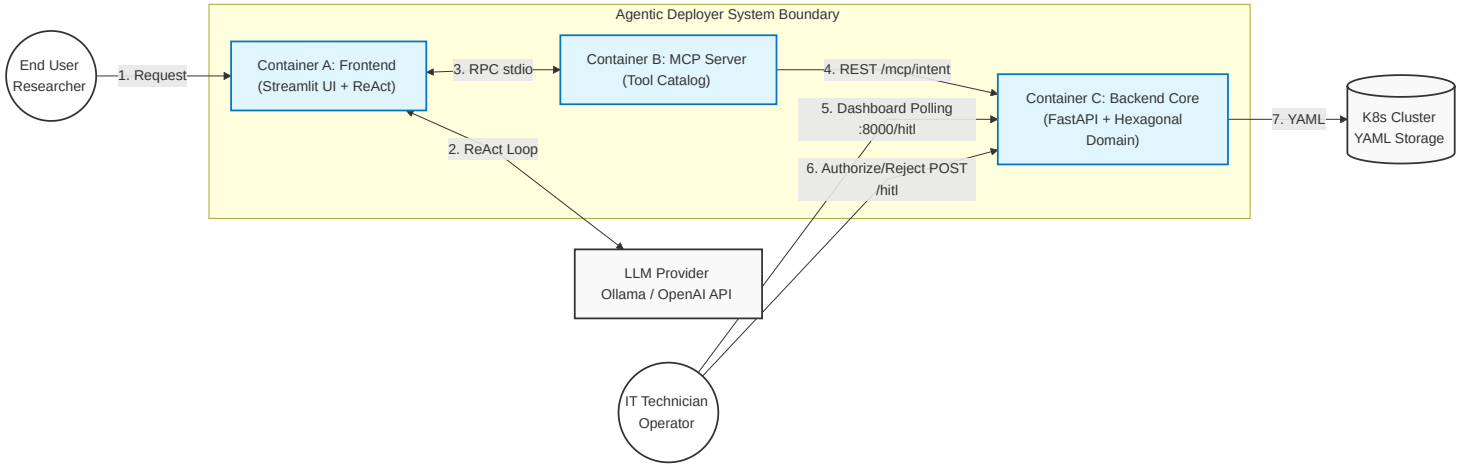



***Figure 6:*** *C4 Container Diagram (Level 2) for the Agentic Deployer system.*

### 5.2.2.1. Interaction Sequence and Direct Backend Access Rationale

Figure 6 outlines the sequence of interactions (1 through 7):

1. **User Request:** The researcher submits an infrastructure requirement in natural language to Container A (Streamlit Frontend).
2. **Inference Cycle:** The orchestrator in the Frontend executes the ReAct loop, communicating bidirectionally with the LLM Provider (Ollama or Cloud API).
3. **Tool Invocation:** Upon deducing an action, the Frontend calls the tool catalog exposed by Container B (MCP Server) via JSON-RPC over `stdio`.
4. **Typed Intent Submission:** The MCP Server translates the action into an HTTP REST invocation (`POST /mcp/intent`) against Container C (FastAPI Backend).
5. **Operator Polling:** The IT technician accesses the HITL dashboard exposed by the Backend (`:8000/hitl`) to review pending requests (`PENDING_APPROVAL`).
6. **Authorization or Rejection:** Following manual review, the operator records a decision (`POST /hitl/approve/{id}` or `POST /hitl/reject/{id}`) on the Backend.
7. **Declarative Manifest Generation:** The Backend invokes the outbound infrastructure adapter (`DeployPort`) to generate the Kubernetes YAML manifest.

**Rationale for Direct Operator Access to the Backend (Interactions 5 and 6):**
In Figure 6, the IT technician interacts directly with Container C (Backend) rather than Container A (Frontend). This design satisfies two key engineering criteria:

- **Separation of Concerns and Security:** Container A is designed solely as an end-user conversational interface handling stochastic LLM context. Corporate governance dictates that infrastructure operators should not share channels with end-user agent sessions.
- **Administrative Dashboard Isolation:** Container C natively serves the HITL Dashboard web interface at `/hitl`. This independence guarantees that even if the Streamlit UI or LLM inference hangs, operations engineers maintain uninterrupted access to system state to audit, approve, or reject deployment requests.

### 5.2.2.2. Mapping Logical Containers to Runtime Processes

Figure 7 illustrates how logical containers map to physical processes on the hosting server:

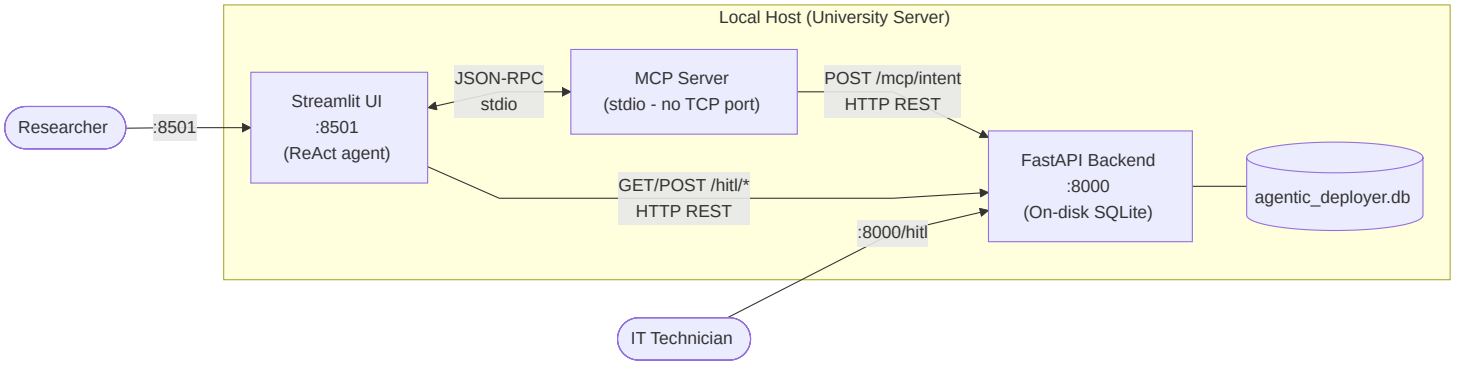


***Figure 7:*** *Process deployment diagram. Components co-locate on the host; the MCP server communicates via* `stdio` *without binding network ports.*

Process allocation decisions include:

- **Streamlit Frontend (Port 8501):** Web process hosting the conversational UI and ReAct reasoning loop.
- **MCP Server (`stdio` Transport):** Runs as a local subprocess binding zero network ports, eliminating external attack vectors and minimizing latency via OS pipes.
- **FastAPI Backend (Port 8000):** Serves REST API endpoints for MCP tool dispatching and hosts the HITL Dashboard web UI (`/hitl`). State is persisted to a local `agentic_deployer.db` SQLite database.

### 5.2.2.3. Functional Container Breakdown

1. **Container A — Cognitive Orchestration Frontend (Streamlit UI + ReAct):**
   Serves as the entry point for researchers. Beyond UI rendering, it hosts the `AgentOrchestrator`, maintaining authorized network connections to external LLM providers (OpenAI or Ollama), managing conversational session histories, and governing ReAct iterations.
2. **Container B — MCP Tool Server (Python FastMCP):**
   Acts as the university's dynamic catalog of operational capabilities, translating functions (e.g., `deploy_congress_web()`) into standardized JSON schemas for LLM tool calling. It communicates with the orchestrator over standard I/O (`stdio`), ensuring sub-millisecond JSON-RPC message exchange.
3. **Container C — Deterministic Core (FastAPI Hexagonal Backend):**
   Built on FastAPI, this service encapsulates domain models, transactional persistence, and security validation rules. It receives typed JSON payloads from the MCP Server, deterministically evaluates corporate policies, and records compliant requests in the `PENDING_APPROVAL` state within SQLite for operator review.

## 5.2.3. Level 3: UML Class Diagram (Micro-Architecture)

Figure 8 details the unified system class diagram, formalizing relationships between domain entities, abstract interfaces, and concrete adapters:

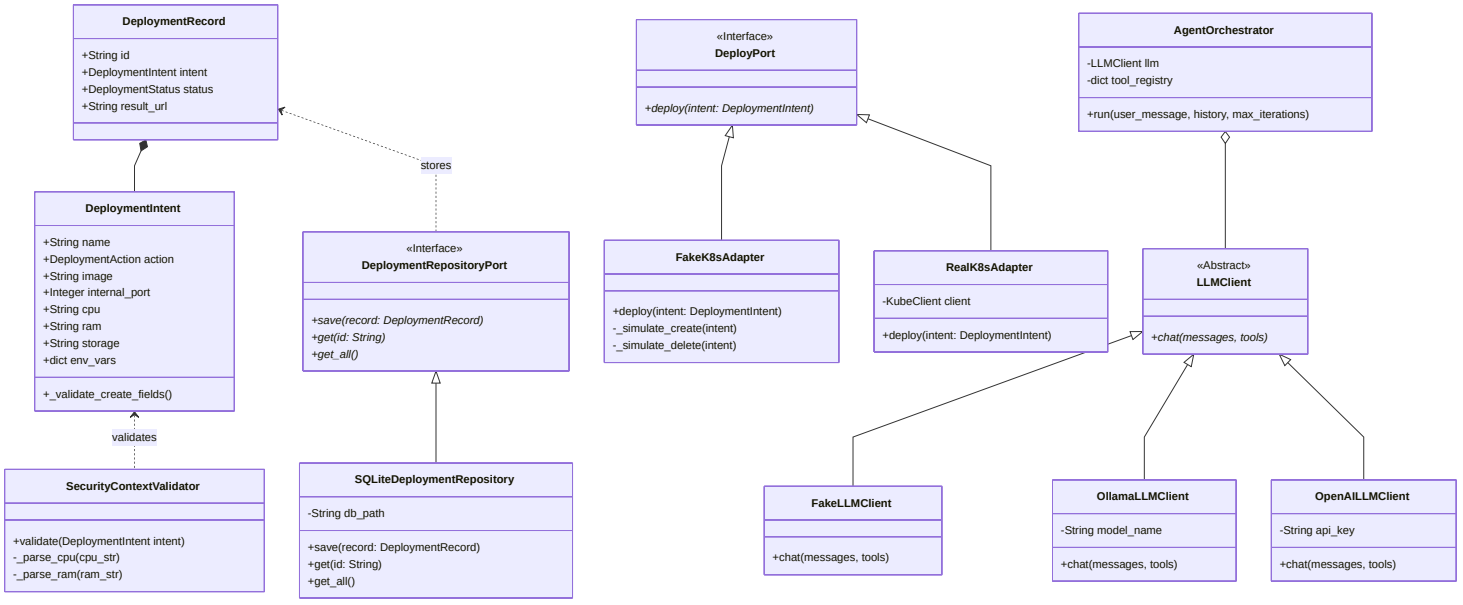


***Figure 8:*** *UML Class Diagram illustrating core entities, contracts, and polymorphic dependency injection points (LLMClient, DeployPort, DeploymentRepositoryPort).*

### 5.2.3.1. Class Structure and Design Pattern Analysis

The class diagram in Figure 8 implements five design patterns:

1. **Inference Client Abstraction (Strategy Pattern):** The abstract `LLMClient` class defines the polymorphic `chat(messages, tools)` contract. Concrete implementations `OllamaLLMClient`, `OpenAILLMClient`, and `FakeLLMClient` encapsulate specific protocol details. `AgentOrchestrator` aggregates `LLMClient`, allowing runtime engine swapping via dependency injection.
2. **Domain Entities (`DeploymentIntent` and `DeploymentRecord`):** `DeploymentIntent` serves as a strongly typed data transfer and validation object. `DeploymentRecord` establishes a composition relationship with `DeploymentIntent`, attaching persistent metadata such as a unique UUID (`id`), lifecycle status (`status: DeploymentStatus`), and artifact URL (`result_url`).
3. **Domain Validation Service (`SecurityContextValidator`):** Maintains a dependency relationship with `DeploymentIntent`. It evaluates intent attributes against institutional corporate policies, raising typed exceptions on non-compliant parameters prior to persistence.
4. **Deployment Port (`DeployPort`):** An abstract interface declaring `deploy(intent: DeploymentIntent)`. `FakeK8sAdapter` fulfills this contract by rendering YAML manifests to disk, while `RealK8sAdapter` fulfills it via live Kubernetes cluster API calls, satisfying the Open/Closed Principle.
5. **Persistence Port and Adapter (`DeploymentRepositoryPort` and `SQLiteDeploymentRepository`):** The domain interfaces with persistence via `DeploymentRepositoryPort` (`save`, `get`, `get_all`), implemented by `SQLiteDeploymentRepository`. This isolates storage operations without leaking SQL dependencies into domain entities.

# 5.3. Hexagonal Architecture Adoption (Ports and Adapters)

Designing the backend core requires a resilient architectural foundation. Granting operational autonomy to a stochastic agent requires strict deterministic boundaries. The application core implements the **Hexagonal Architecture** (*Ports and Adapters*) pattern formalized by Alistair Cockburn [19].

The pattern structures software into concentric layers with a strict inward dependency rule: external volatile technologies depend on inner immutable business logic.

## 5.3.1. Domain Layer: Entities and Invariants

At the center of the hexagon lies the Domain Layer, representing pure business logic. In *Agentic Deployer*, this layer models infrastructure deployment intents prior to declarative manifest generation.

To guarantee static type safety, the system rejects raw, untyped dictionaries emitted by language models. All external inputs are mapped to the strongly typed `DeploymentIntent` domain entity.

This entity enforces **domain invariants**:

- **Identifier:** Unique, immutable UUID string.

- **Action:** Constrained to a finite set (`CREATE`, `DELETE`).
- **Topology Attributes:** Service name, container image, compute resources (CPU/RAM).
- **Logical Invariants:** Internal network ports must reside strictly within `[1, 65535]`. Create actions strictly require an image and a valid port.

These invariants act as a first line of defense against hallucinations. If an agent emits a null or out-of-bounds port (`-80`), the domain entity rejects instantiation by raising an immediate exception, enforcing *Fail-Fast* guarantees.

#### 5.3.1.1. Illustrative `DeploymentIntent` Instance

The following code illustrates an instantiated `DeploymentIntent` generated following conversational negotiation for an academic conference portal:

```
# Strongly typed domain entity instantiation (Pydantic / Python)
intent = DeploymentIntent(
    name="congreso-ia-2026",
    action=DeploymentAction.CREATE,
    image="docker.io/library/nginx:1.25-alpine",
    internal_port=8080,
    cpu="500m",
    ram="256Mi",
    env_vars={"APP_ENV": "production", "PORT": "8080"}
)
```

The equivalent JSON payload transmitted by the MCP Server to the Backend endpoint (`POST /mcp/intent`) reflects this structure:

```
{
  "name": "congreso-ia-2026",
  "action": "CREATE",
  "image": "docker.io/library/nginx:1.25-alpine",
  "internal_port": 8080,
  "cpu": "500m",
  "ram": "256Mi",
  "env_vars": {
    "APP_ENV": "production",
    "PORT": "8080"
  }
}
```

The fields satisfy domain invariants: the `name` conforms to Kubernetes DNS syntax (`^[a-z0-9-]+$`), the image tag avoids mutable `:latest` references, the port (`8080`) is $\ge 1024$, and environment variables contain no plaintext credentials.

## 5.3.2. Dependency Inversion Principle (DIP)

Hexagonal design relies on the Dependency Inversion Principle (DIP). The domain core never imports external libraries or database drivers. Instead, it exposes abstract **Ports** implemented by peripheral **Adapters** (Figures 8 and 9).

Adapters are categorized into two roles:

### 5.3.2.1. Primary (Driving) Adapters

Components that initiate execution into the core (left side of Figure 9). In this architecture, driving adapters are FastAPI REST controllers. The MCP endpoint (`/mcp/intent`) dispatches to `ProcessDeploymentUseCase`, while the dashboard approval endpoint orchestrates approval state transitions.

### 5.3.2.2. Secondary (Driven) Adapters

Components driven by the application core to interact with external infrastructure (right side of Figure 9).

To decouple business logic from cluster platforms, the application layer declares the abstract `DeployPort`. In this prototype, `FakeK8sAdapter` fulfills this port by writing complete YAML manifests to disk. Replacing this mock with a production `RealK8sAdapter` (Section 10.3.1) requires zero changes to core domain logic.

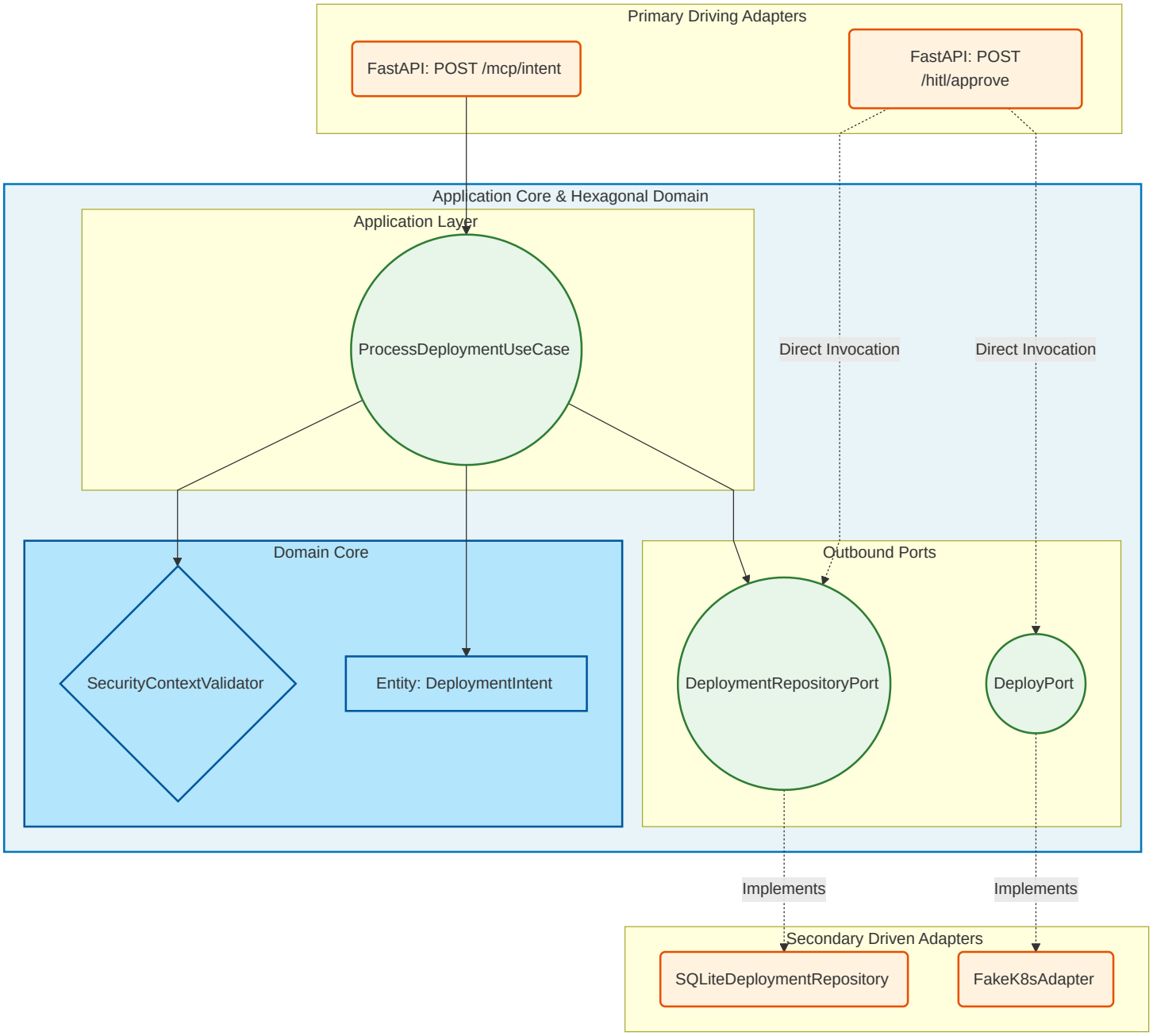



***Figure 9:*** *Hexagonal Architecture topology. Control flow enters via Driving Adapters, while source code dependencies point inward toward the core (Dependency Inversion Rule).*

# 5.4. Institutional Security Policy Validation Algorithm

Delegating deployment intent authoring to an AI agent introduces a threat vector: models could generate configurations that are syntactically valid yet corporately unauthorized. Safeguarding the cluster against these requests falls to the `SecurityContextValidator` domain service.

## 5.4.1. Logical Firewall: Algorithmic Formalization

`SecurityContextValidator` operates under a Zero-Trust posture, intercepting `DeploymentIntent` objects and evaluating them against institutional security policies:

### 5.4.1.1. System Policy Configuration Parameters (`politicas_locales`)

1. **Privileged Port Floor (`MIN_ALLOWED_PORT = 1024`):** Well-known ports (`0` to `1023`) are strictly reserved for administrative root processes.
2. **Container Registry Whitelist (`ALLOWED_REGISTRIES`):**
   - `docker.io/` (Official Docker Hub).
   - `quay.io/` (Red Hat Quay).
   - `harbor.universidad.edu/` (Internal institutional registry).
     Images outside these registries are rejected.
3. **Restricted Volatile Tags (`FORBIDDEN_IMAGE_TAGS = {"latest"}`):** Prohibits `:latest` tags to mitigate configuration drift.
4. **Departmental Hardware Limits:**
   - `MAX_CPU_MILLICORES = 4000` (4 virtual cores).
   - `MAX_RAM_MEBIBYTES = 8192` (8 GiB).
5. **Forbidden Substrings in Environment Variables (`FORBIDDEN_ENV_KEYS`):** Blocks plaintext secrets matching `"password"`, `"secret"`, `"token"`, `"key"`, `"credential"`.
6. **Mandatory Storage for Databases:** Images referencing database engines (`postgres`, `mysql`, `redis`, `mongo`) must declare explicit persistent storage.

Lifecycle exemption rule: deletion requests (`action = DELETE`) bypass creation checks, enabling resource teardown without evaluating image registries or ports.

Algorithm 1 details the validation logic:

```
01 INPUT:
02  intent -> DeploymentIntent object (Basic invariants satisfied)
03  local_policies -> System Configuration (Port floors, registries, quotas)
04
05 OUTPUT:
06  TRUE (If intent satisfies all institutional policies)
07  RAISE SecurityViolationError with list of accumulated violations
08
09 BEGIN
10   // RULE 0: Lifecycle exemption for deletion operations
11   IF intent.action EQUALS DeploymentAction.DELETE THEN
12     RETURN TRUE
```

```
  END IF

  Variable violations = EmptyList()

  // RULE 1: Network Topology Inspection (Principle of Least Privilege)
  IF intent.internal_port != NULL AND intent.internal_port < 1024 THEN
    violations.Append("Privileged port not allowed: " + intent.internal_port)
  END IF

  // RULES 2, 3 & 4: Image and Storage Verification
  IF intent.image != NULL THEN
    IF ":latest" IN intent.image THEN
      violations.Append("Use of ':latest' tag forbidden: " + intent.image)
    END IF

    Variable valid_registry = FALSE
    FOR EACH registry IN local_policies.ALLOWED_REGISTRIES DO
      IF intent.image STARTS_WITH registry THEN
        valid_registry = TRUE
        BREAK LOOP
      END IF
    END FOR
    IF valid_registry IS FALSE THEN
      violations.Append("Untrusted registry: " + intent.image)
    END IF

    IF ContainsDatabaseEngine(intent.image) AND intent.storage IS NULL THEN
      violations.Append("Databases require explicit persistent storage allocation")
    END IF
  END IF

  // RULE 5: Hardware Quota Audit (Resource Quotas)
  Variable cpu_m = ParseMillicores(intent.cpu)
  IF cpu_m > 4000 THEN
    violations.Append("CPU quota exceeded: " + intent.cpu + " > 4000m")
  END IF

  Variable ram_mi = ParseMebibytes(intent.ram)
  IF ram_mi > 8192 THEN
    violations.Append("RAM quota exceeded: " + intent.ram + " > 8192Mi")
  END IF

  // RULE 6: Heuristic Secret Detection in Environment Variables
  FOR EACH key IN intent.env_vars DO
    IF ContainsSubstring(key.ToLower(), ["password", "secret", "token", "key", "credential"]) THEN
      violations.Append("Potential plaintext secret detected in variable: " + key)
    END IF
  END FOR

  // Halt execution if violations were accumulated
  IF Length(violations) > 0 THEN
    RAISE SecurityViolationError(violations)
  END IF

  RETURN TRUE
END
```

***Algorithm 1:*** *Strict Hexagonal Domain Validation (SecurityContextValidator).*

Operating with computational complexity $O(R + K)$ (where $R$ is trusted registries count and $K$ is environment variable keys), this algorithm establishes deterministic boundaries over generative outputs.

Figure 10 summarizes the decision tree:

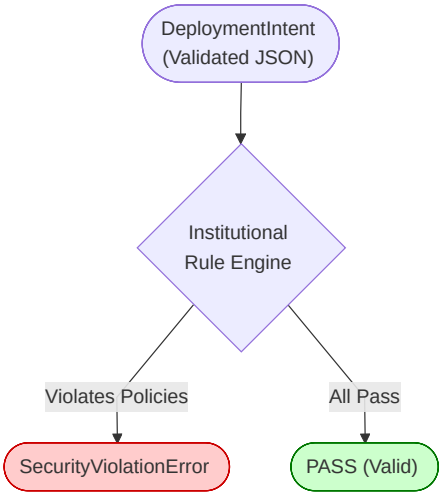



***Figure 10:*** *Simplified overview of* `SecurityContextValidator`*. Policy rejections raise exceptions enabling ReAct self-correction.*

### 5.4.2. Exception Management and Error Lifecycle

In standard microservice architectures, raising an unhandled `SecurityViolationError` aborts the transaction, returning an HTTP `422 Unprocessable Entity` to the user interface.

*Agentic Deployer* introduces an agent-oriented interception pattern. Rather than presenting technical stack traces to users, errors are routed back to the LLM within the conversational flow:

1. `SecurityContextValidator` raises `SecurityViolationError`.
2. FastAPI serializes the exception into an HTTP `422` response returned to the MCP Server.
3. The MCP Server transmits the error to the conversational agent. The ReAct engine records this output as an **Observation**.
4. The Observation is appended to the LLM's context window.

The model parses the structured error, updates its internal reasoning trace, and interactively requests compliant parameters from the user (e.g., asking for port 8080 instead of 22).

## 5.5. Abstraction Pattern: Golden Paths and Syntactic Expansion

The final architectural pillar addresses declarative code generation (Infrastructure as Code, IaC). A naive adoption approach would task the LLM with authoring raw YAML manifests from scratch.

However, in Site Reliability Engineering (SRE), unconstrained generation introduces severe **Configuration Drift**. The model might generate deprecated API versions (`extensions/v1beta1` instead of `apps/v1`), omit readiness probes, or misalign routing label selectors.

### 5.5.1. Theoretical Definition of Golden Paths

To mitigate this risk, the architecture implements **Golden Paths**. A Golden Path is a standardized, hardened, and pre-audited infrastructure template curated by platform operations teams.

The language model does not author raw infrastructure; its role is confined to parameter extraction. Structural topologies (the "how") are fixed within pre-audited templates, while the model supplies contextual parameters (the "what") negotiated with the user.

## 5.5.2. Templating Engine (Decoupling Topology from Variables)

Syntactic expansion occurs in the outbound infrastructure adapter (`DeployPort`). When an authorized request is approved, the adapter interpolates parameters into the pre-tested declarative matrix.

A high-level prompt ("I need a database") expands deterministically into interdependent primitives:

1. A **`Deployment`** or **`StatefulSet`** with quota limits and anti-affinity rules.
2. A **`Service`** configured as `ClusterIP` preventing accidental public exposure.
3. A **`PersistentVolumeClaim`** (PVC) sized according to inferred capacity requirements.

Every service provisioned by *Agentic Deployer* inherits identical security postures, allowing IT teams to scale provisioning safely.

> **Note on MVP Scope:** The `FakeK8sAdapter` template generates three core resources for web applications: `Deployment`, `Service`, and `Ingress`. Dedicated PVC generation for stateful databases is identified as future work (Section 10.3.1).

### 5.5.2.1. Illustrative Golden Path Expansion Example

The following block shows the multi-document template defined in `FakeK8sAdapter` and the resulting YAML generated for the `congreso-ia-2026` portal:

```yaml
# Golden Path base template (FakeK8sAdapter in app/infrastructure/fake_k8s_adapter.py)
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {intent.name}
  labels:
    app: {intent.name}
spec:
  replicas: 1
  selector:
    matchLabels:
      app: {intent.name}
  template:
    metadata:
      labels:
        app: {intent.name}
    spec:
      containers:
        - name: {intent.name}
          image: {intent.image}
          ports:
            - containerPort: {intent.internal_port}
          resources:
            requests:
              cpu: {intent.cpu}
              memory: {intent.ram}
---
apiVersion: v1
kind: Service
metadata:
  name: {intent.name}-svc
```

```
spec:
  selector:
    app: {intent.name}
  ports:
    - protocol: TCP
      port: 80
      targetPort: {intent.internal_port}
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {intent.name}-ingress
spec:
  rules:
    - host: {intent.name}.apps.universidad.edu
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: {intent.name}-svc
                port:
                  number: 80
```

Interpolating validated intent variables (`name: "congreso-ia-2026"`, `image: "docker.io/library/nginx:1.25-alpine"`, `internal_port: 8080`, `cpu: "500m"`, `ram: "256Mi"`) yields the final on-disk manifest (`outputs/congreso-ia-2026.yaml`), exposing `https://congreso-ia-2026.apps.universidad.edu`:

```
# Final deterministically generated YAML manifest (outputs/congreso-ia-2026.yaml)
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: congreso-ia-2026
  labels:
    app: congreso-ia-2026
spec:
  replicas: 1
  selector:
    matchLabels:
      app: congreso-ia-2026
  template:
    metadata:
      labels:
        app: congreso-ia-2026
    spec:
      containers:
        - name: congreso-ia-2026
          image: docker.io/library/nginx:1.25-alpine
          ports:
            - containerPort: 8080
          resources:
            requests:
              cpu: 500m
              memory: 256Mi
---
apiVersion: v1
kind: Service
metadata:
  name: congreso-ia-2026-svc
spec:
  selector:
    app: congreso-ia-2026
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
---
apiVersion: networking.k8s.io/v1
```

```yaml
kind: Ingress
metadata:
  name: congreso-ia-2026-ingress
spec:
  rules:
    - host: congreso-ia-2026.apps.universidad.edu
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: congreso-ia-2026-svc
                port:
                  number: 80
```

# 5.6. End-to-End Data Flow and Lifecycle States

Architectural components coordinate across three sequential lifecycle stages:

### 5.6.0.1. Illustrative End-to-End Walkthrough

1. **Natural Language Input (Stage 1):** The researcher types into the Streamlit UI: *"I need to deploy the AI conference web portal with NGINX on port 8080 with 500m CPU and 256Mi RAM"*.
2. **Inference & Tool Selection:** `AgentOrchestrator` queries the model in the ReAct loop. The LLM selects `format_deployment_intent`, emitting a structured tool call.
3. **Hexagonal Validation (Stage 2):** The MCP Server issues `POST /mcp/intent`. `SecurityContextValidator` executes Algorithm 1:
    - Port 8080 $\ge 1024$ $\rightarrow$ Valid.
    - Image in whitelist without `:latest` $\rightarrow$ Valid.
    - Hardware quotas within limits $\rightarrow$ Valid.
    - Zero plaintext secrets detected $\rightarrow$ Valid.
4. **Transactional Persistence:** The Backend stores the record in SQLite with an 8-character hexadecimal identifier (e.g., `7f3a2c1b`) under `PENDING_APPROVAL`, returning HTTP `201 Created` to the agent, which notifies the user.
5. **HITL Review & Approval (Stage 3):** An IT operator opens the Dashboard (`:8000/hitl`), inspects request `7f3a2c1b`, and clicks "Approve" (`POST /hitl/approve/7f3a2c1b`).
6. **Declarative Materialization:** The backend FSM transitions to `APPROVED` and `DEPLOYED`. `FakeK8sAdapter` interpolates the Golden Path template, writes `outputs/congreso-ia-2026.yaml`, and returns the service URL.

## 5.6.1. Stage 1: Cognitive Parameter Extraction (LLM Interaction)

Figure 11 models initial conversational inference. Natural language input is processed through the ReAct loop until structured arguments (`ToolCall`) are synthesized:

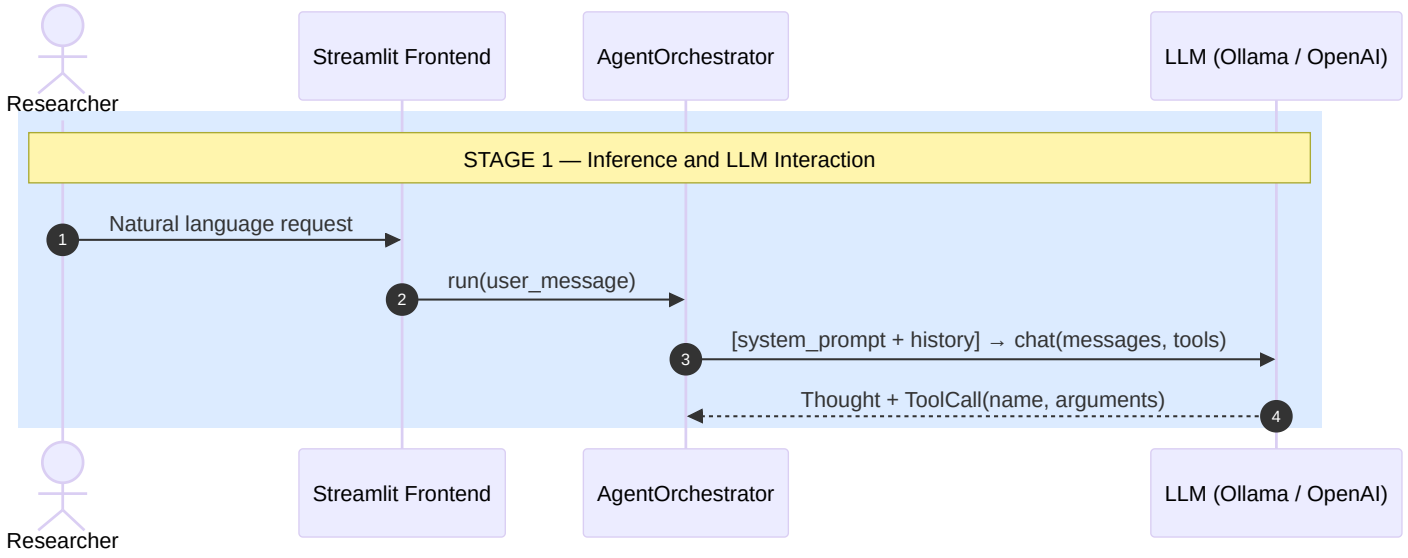
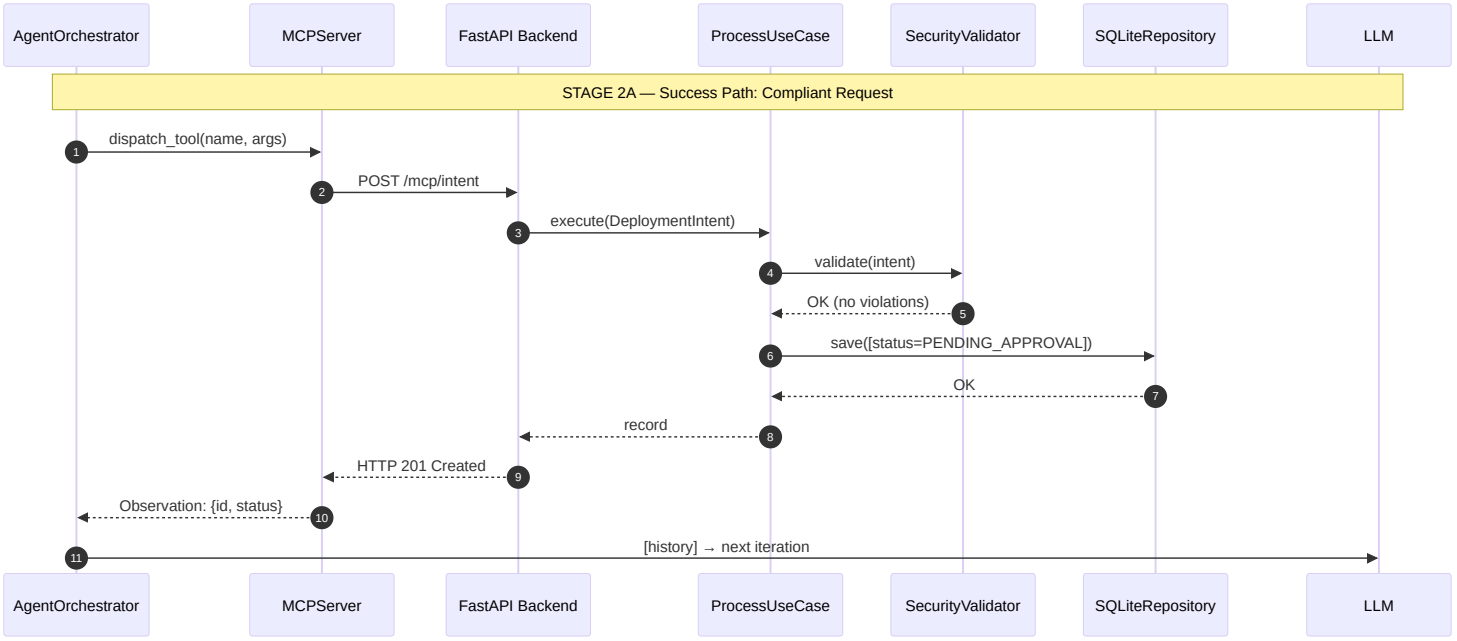



***Figure 11:*** *E2E Sequence Diagram (Stage 1). Interaction between Researcher and LLM for structured intent extraction.*

## 5.6.2. Stage 2: Deterministic Validation and Quarantine Persistence

Figure 12 illustrates deterministic boundary validation in the backend, executing in microseconds. The diagram depicts both the nominal success path (Stage 2A) and policy violation handling (Stage 2B, returning HTTP 422 to trigger ReAct self-correction):



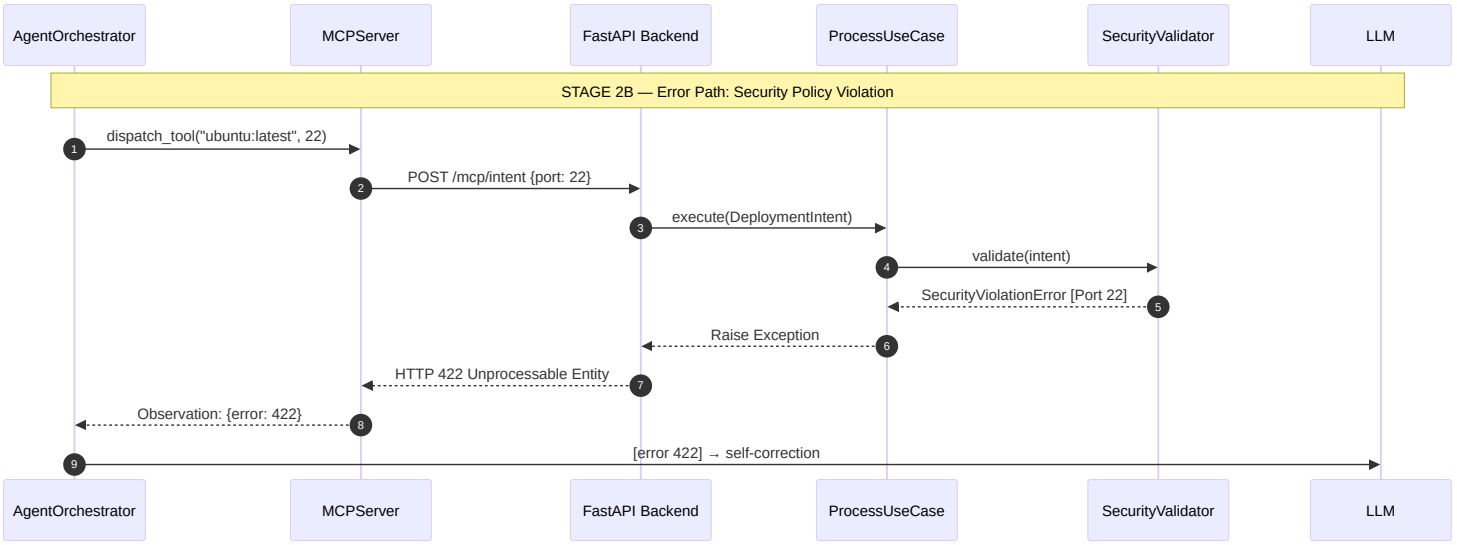



***Figure 12:*** *E2E Sequence Diagram (Stage 2). Backend deterministic validation: (Top) Compliant success path (Stage 2A); (Bottom) Security violation interception and self-correction (Stage 2B).*

## 5.6.3. Stage 3: Human Oversight and Authorization (HITL Cycle)

Figure 13 illustrates asynchronous decision-making and infrastructure materialization. Operations depend on operator review schedules in the HITL dashboard. Upon technician approval, the FSM transitions to `APPROVED` and `DEPLOYED`, instructing `DeployPort` to write YAML manifests to storage. If rejected, the transition to `REJECTED` prevents cluster execution:

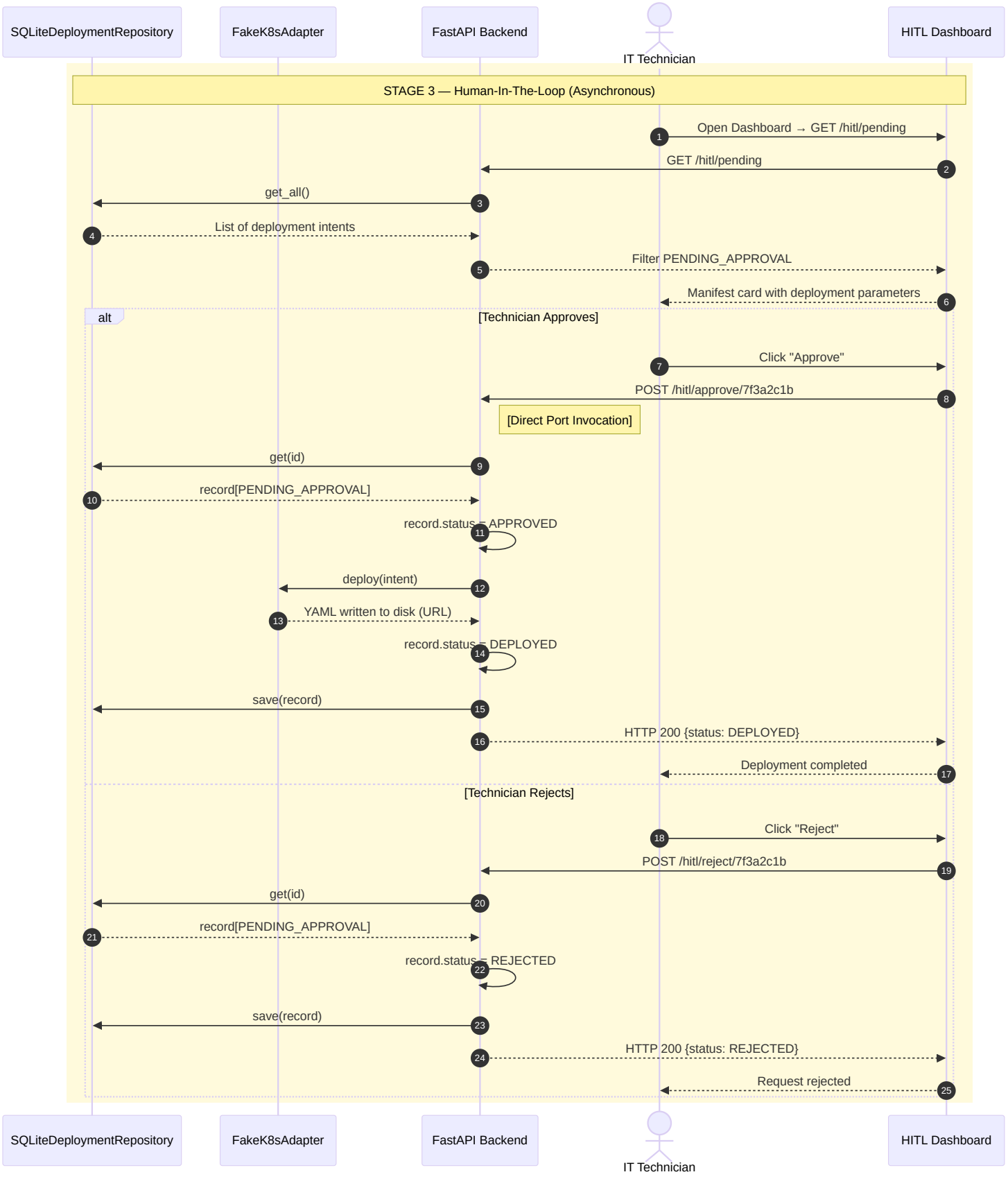



***Figure 13:*** *E2E Sequence Diagram (Stage 3). Asynchronous IT technician oversight (HITL) and infrastructure manifest materialization.*

> *Scope Note: Figure 13 models global HITL orchestration.* ***Section 7.3.3*** *expands this flow, incorporating dashboard polling mechanics, Algorithm 4 (`FSM_Transition`), and isolated approval/rejection sequence diagrams.*

This architecture establishes strict separation of concerns: the language model serves exclusively as an **assistant for syntax interpretation and intent extraction**, while deterministic backend modules retain exclusive authority over **infrastructure state mutations**.

# Chapter 6. Cognitive Agent and MCP Server Development

Building upon the deterministic foundations established in Chapter 5, this chapter details the implementation of the heuristic core of *Agentic Deployer*: the cognitive engine that translates natural language into infrastructure operations by integrating the *Model Context Protocol* (MCP) open standard, a provider-agnostic inference client abstraction, and the *ReAct* reasoning loop.

> **Terminology Note:** Throughout this chapter, the acronym **LLM** (*Large Language Model*) designates deep neural networks trained on vast text corpora. It refers to foundational generative models rather than generic chatbots.

## 6.1. Model Context Protocol (MCP) Server Implementation

In early generative AI architectures, equipping a language model with code execution capabilities relied on tightly coupled, vendor-specific integrations. Orchestrators manually authored function schemas as rigid JSON structures conforming to proprietary specifications (such as OpenAI's native *Function Calling*). This practice violated the Open/Closed Principle (the 'O' in SOLID), requiring extensive code rewrites whenever an organization elected to migrate between AI providers (e.g., from OpenAI to Anthropic or self-hosted local models).

To overcome this vendor lock-in antipattern, *Agentic Deployer* adopts the **Model Context Protocol (MCP)** [13] as its core integration standard. MCP establishes an open, universal middleware layer that strictly decouples tool catalogs from model-specific API idiosyncrasies.

### 6.1.1. Dynamic Function Introspection (JSON Schema Generation)

A persistent challenge in autonomous agent engineering is tool contract synchronization. If a platform engineer modifies an infrastructure function signature to require a new parameter (e.g., `ram_quota`), the JSON Schema supplied to the LLM must be updated simultaneously; otherwise, inference fails due to schema drift.

To mitigate this risk, the MCP Server implemented in *Agentic Deployer* (Container B in the C4 model) leverages **Static and Dynamic Type Introspection**. Rather than requiring developers to craft JSON Schemas manually, the MCP SDK introspects function parameter signatures and type hints at runtime using Python's native `inspect` module.

Algorithm 2 illustrates how the system dynamically transforms a pure Python function into a universal semantic tool definition:

```
01 INPUT:
02  target_function -> In-memory function reference (e.g., deploy_app)
03
04 OUTPUT:
05  json_schema -> Standardized JSON-RPC Tool Calling schema
06
07 BEGIN
08   Variable schema = NEW JSON_Dictionary
09   schema["name"] = target_function.getName()
10   schema["description"] = target_function.getDocstring()
11
12   // Inspect Abstract Syntax Tree (AST) & Lexical Signature
13   Variable parameters = target_function.getParameterSignature()
14   schema["parameters"] = NEW Object_Type(Object)
```

```
15
16   FOR EACH (param, type_annotation) IN parameters DO
17     Variable json_type = "string" // Default fallback
18
19     // Injective Type Mapping (Python -> JSON Schema)
20     IF type_annotation IS Integer THEN
21       json_type = "integer"
22     ELSE IF type_annotation IS Boolean THEN
23       json_type = "boolean"
24     END IF
25
26     schema["parameters"]["properties"][param] = NEW Dictionary(
27       "type" -> json_type,
28       "description" -> extractDocDescription(param)
29     )
30
31     IF param IS mandatory THEN
32       APPEND param TO schema["parameters"]["required"]
33     END IF
34   END FOR
35
36   RETURN schema
37 END
```

***Algorithm 2:*** *Dynamic Tool Contract Introspection.*

This approach establishes a **Single Source of Truth**. Engineers annotate infrastructure functions using standard Python Type Hints (e.g., `port: int`) and docstrings. Upon initialization, the MCP server parses annotations, synthesizes compliant JSON Schemas, and advertises them to the orchestrator, eliminating contract drift.

## 6.1.2. Network Isolation via Standard I/O (*stdio*) Transport

The MCP specification supports multiple transport bindings, notably *Server-Sent Events* (SSE) over HTTP and Standard Input/Output (*stdio*).

In conventional distributed architectures, communication between the Frontend Agent (Container A) and the Tool Server (Container B) might default to a REST API. However, in stochastic agent workflows where a ReAct loop may invoke multiple tools sequentially to query system state, TCP/IP overhead (HTTP header encapsulation, loopback socket latency, serialization) introduces unnecessary operational delays.

Consequently, *Agentic Deployer* enforces a restrictive architectural design choice: **The primary tool calling channel is bound strictly to local `stdio` (Standard Input / Standard Output) streams**.

This choice provides three distinct architectural advantages:

1. **Sub-Millisecond Inter-Process Communication (IPC):** Communicating over kernel pipes bypasses the network stack (OSI Layers 4–7). Tool dispatch latency drops from ~20 ms (local HTTP) to under ~1 ms, preserving conversational responsiveness.
2. **Zero Network Attack Surface (Zero-Trust):** Unlike network sockets (susceptible to internal port scans and Server-Side Request Forgery), a `stdio` process binds no network ports. External attackers cannot reach infrastructure tools without local OS-level process tree compromise.
3. **Operational Simplicity (Sidecar Pattern):** Eliminating internal reverse proxies, load balancers, and TLS certificates allows the tool server to deploy seamlessly alongside the orchestrator in a sidecar pattern.

Figure 14 illustrates the complete message lifecycle of an MCP interaction, from tool discovery to execution and observation:

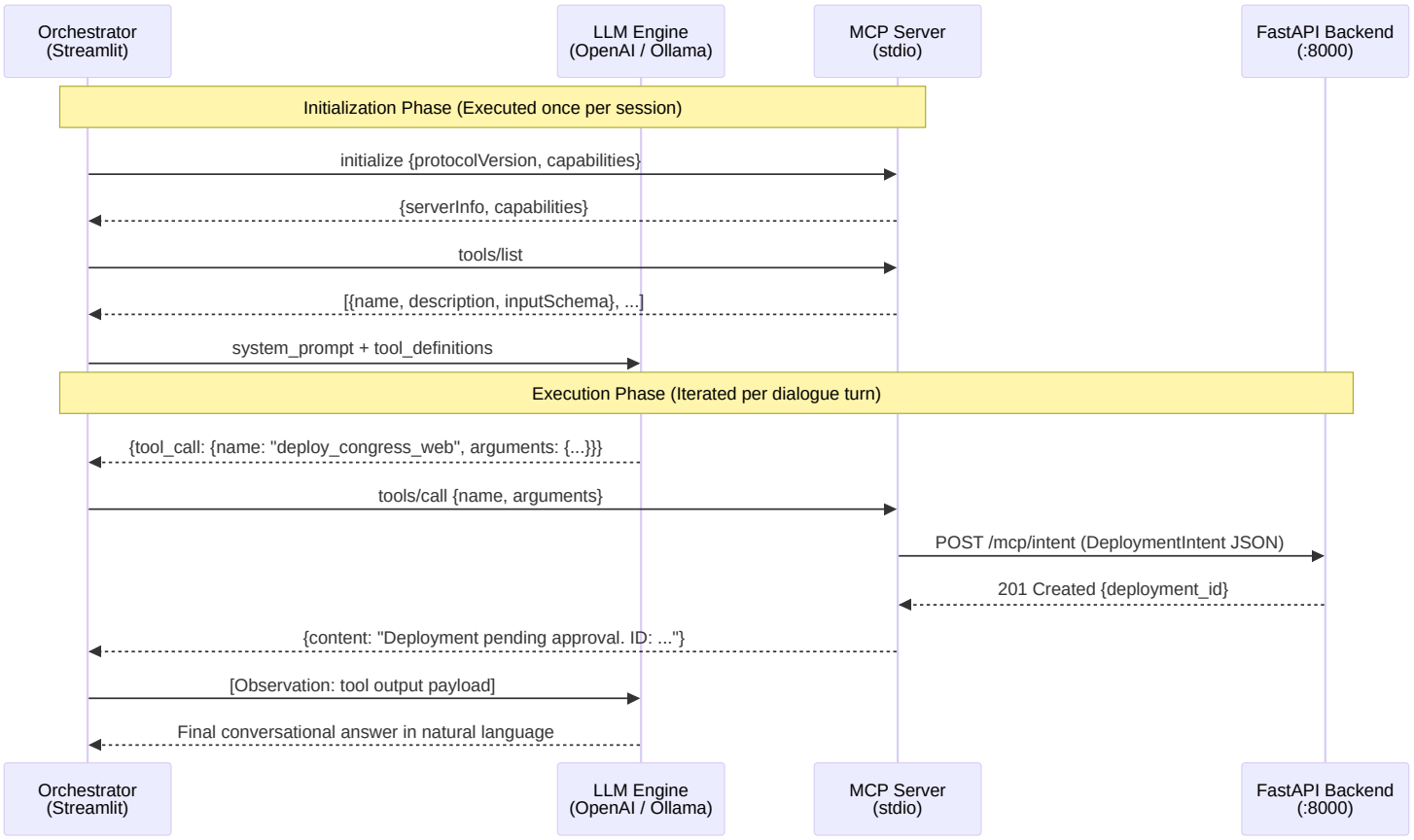



***Figure 14:*** *Complete MCP message lifecycle. The JSON-RPC protocol establishes initialization (handshake and tool discovery), execution (invocation and response), and observation (LLM feedback).*

## 6.1.2.1. JSON-RPC Messaging Breakdown

The interaction modeled in Figure 14 divides into two operational phases:

1. **Initialization and Capability Negotiation (Once per session):**
    - The `AgentOrchestrator` sends an `initialize` JSON-RPC message declaring supported protocol versions.
    - The MCP server replies with metadata confirming connection parameters.
    - The orchestrator requests available tools via `tools/list`. The server returns structured metadata including function names, descriptions, and parameter JSON Schemas (`inputSchema`).
    - The orchestrator injects these schemas into the LLM context alongside system instructions.
2. **Execution and Observation Phase (Per conversation turn):**
    - When the model determines an infrastructure action is required, it emits a structured `tool_call` object.
    - The orchestrator intercepts the call and dispatches `tools/call` over `stdio` pipes.
    - The MCP server maps arguments into a typed HTTP REST payload (`POST /mcp/intent`) dispatched to the FastAPI backend on port 8000.
    - Following deterministic policy validation, the backend records the request in `PENDING_APPROVAL`, returning an

HTTP `201 Created` status with an assigned `deployment_id`.

- The MCP server wraps this response into an observation returned to the orchestrator, which injects it into the LLM prompt for final natural language generation.

> **Implementation Note (Integrated vs. Canonical Transport):**
>
> 1. **Integrated In-Memory Mode (Streamlit MVP):** Tool registries and dispatchers are loaded within the same Python process (`TOOL_REGISTRY`, `TOOL_DEFINITIONS`). Execution is an in-process Python call, eliminating IPC serialization overhead and reducing invocation latency to microseconds. This simplified thread management within Streamlit.
> 2. **Canonical External Mode (`stdio`):** When the agent and MCP server execute as distinct operating system processes (e.g., connecting external tools like Claude Desktop or standalone CLI scripts), communication proceeds via standard input/output pipes with structured JSON-RPC frames. In this mode, serialization latency remains sub-millisecond (<1 ms), providing memory isolation between processes.
> Both modes share identical interfaces, decoupled by `AgentOrchestrator`.

# 6.2. Multi-Provider Abstraction and Data Sovereignty

The rapid evolution of Generative Artificial Intelligence has created a concentrated market dominated by commercial AI-as-a-Service cloud providers. In enterprise software engineering, coupling core business logic to vendor-specific SDKs (OpenAI, Anthropic, Google) introduces severe risks of obsolescence and vendor lock-in.

To safeguard system longevity, *Agentic Deployer* enforces complete provider agnosticism.

## 6.2.1. The Abstract Contract (Adapter and Factory Patterns)

Provider isolation is implemented via two classic Gang of Four (GoF) design patterns [30]: the **Adapter** pattern and the **Factory Method** pattern.

The `AgentOrchestrator` never directly imports proprietary vendor SDKs. It interacts exclusively with an Abstract Base Class (ABC) named `LLMClient`:

```
ABSTRACT CONTRACT: LLM Client Interface (LLMClient)

INTERNAL STATE:
 - conversation_history -> Structured message list (Role, Content)
 - system_prompt -> Core operational guidelines and policy rules

METHOD chat(messages, tools):
  INPUT: messages -> history; tools -> MCP server JSON Schemas
  EXPECTED OUTPUT:
   - AgentResponse object (conversational text)
   - ToolCall object (id, function_name, json_arguments)
```

The system provides three concrete adapters fulfilling this contract:

- **`OpenAILLMClient`:** Formats conversations to the OpenAI REST specification (`messages`, `tools`, `tool_choice`), manages TLS handshakes to cloud APIs, and deserializes response payloads.
- **`OllamaLLMClient`:** A native adapter built upon Ollama's official Python SDK [31] (`ollama.Client`) connecting to `http://localhost:11434`. It guarantees that **zero prompt tokens leave the institutional private network**:

```
PSEUDOCODE: Local Ollama Client Implementation

CLASS OllamaLLMClient IMPLEMENTS LLMClient:
  ATTRIBUTES:
    model: String (e.g., "qwen2.5-coder:7b")
    host_server: String (e.g., "http://localhost:11434")
    ollama_client: ClientSDK(host=host_server)

  METHOD chat(messages, tools):
    Variable params = NEW Dictionary(
      "model" -> model,
      "messages" -> messages
    )
    IF tools EXISTS THEN
      params["tools"] = tools
    END IF

    // Synchronous local inference via Ollama SDK
    Variable response = ollama_client.chat(params)
    Variable message = response.message

    IF message.tool_calls EXISTS THEN
      Variable tool_calls = EmptyList()
      FOR EACH call IN message.tool_calls DO
        tool_calls.Append(
          NEW ToolCall(call.function.name, call.function.arguments)
        )
      END FOR
      RETURN NEW AgentResponse(content=message.content, tool_calls=tool_calls)
    END IF

    RETURN NEW AgentResponse(content=message.content)
END CLASS
```

- **`FakeLLMClient`:** A deterministic, regex-based mock adapter used for automated unit testing, CI/CD regression runs, and portable demonstrations requiring zero GPU hardware.

In-memory instantiation is governed by a **Factory**:

```
PSEUDOCODE: Cognitive Engine Dependency Injection (Factory)

INPUT: provider -> String from environment variable (LLM_PROVIDER)
OUTPUT: llm_engine -> Concrete instance implementing LLMClient

BEGIN
  IF provider EQUALS "ollama" THEN
    RETURN NEW OllamaLLMClient(model="qwen2.5-coder:7b")
  ELSE IF provider EQUALS "openai" THEN
    RETURN NEW OpenAILLMClient(model="gpt-4o-mini")
  ELSE
    RETURN NEW SICFakeLLMClient() // Deterministic test double
  END IF
END
```

This dependency injection design allows **swapping inference engines simply by changing an environment variable and restarting the process**, without altering a single line of business logic.

## 6.2.2. Selection of the Local Model (Qwen 2.5 Coder)

Modern open-weights ecosystems offer numerous models capable of running on modest local hardware (e.g., 6–8 GB VRAM). For the local evaluation testbed, **Qwen 2.5 Coder (7B)** from Alibaba Cloud was selected over general-purpose alternatives such as Llama 3.2, Mistral, or baseline Qwen 2.5.

This decision was driven by Tool Calling reliability. The Coder variant undergoes extensive instruction fine-tuning on structured source code and schema-following corpora, conferring superior JSON generation accuracy. During empirical testing, general-purpose 7B models frequently generated malformed JSON or dropped required schema keys when processing ambiguous prompts. **Qwen 2.5 Coder (7B)** was the only local model in its weight class to successfully pass 100% of the Metamorphic Test battery, demonstrating robust resilience under linguistic variance (Chapter 9).

### 6.2.3. Data Sovereignty in Institutional Environments

Beyond clean architecture, multi-provider abstraction satisfies critical enterprise cybersecurity mandates: **Data Sovereignty** and regulatory compliance (GDPR, Spanish ENS).

*Agentic Deployer* processes sensitive operational specifications. During natural language dialogue, users might inadvertently paste database tokens, administrative credentials, or internal IP subnets. Transmitting these payloads in plaintext to foreign cloud servers violates data privacy regulations.

The architecture accommodates distinct operational risk profiles:

- **Non-Sensitive Workloads:** In public academic sandboxes, the factory can route traffic to cloud APIs (e.g., OpenAI), taking advantage of elastic cloud infrastructure.
- **Confidential / Air-Gapped Environments:** For sensitive services (patient healthcare databases, student academic registries), operations teams set `LLM_PROVIDER=ollama`. Inference executes entirely on GPU servers hosted inside the institutional datacenter, guaranteeing *Zero Data Retention*.

## 6.3. Cognitive Loop and Stochastic Resilience (ReAct Pattern)

Exposing an MCP tool catalog to a language model does not inherently guarantee autonomous task completion. When models operate in dynamic environments (such as Kubernetes clusters whose runtime state evolves during execution), static single-turn prompting (*Zero-Shot Prompting*) is insufficient. If a model generates an invalid assumption or syntax error on its first attempt, it lacks mechanisms to recover.

To address this, the orchestrator implements the **ReAct (Reasoning and Acting)** [9] pattern.

### 6.3.1. Bounded Reasoning and Action Loop

ReAct restructures the execution model. Rather than processing inputs in a single shot, the agent operates as an iterative state machine with a **bounded execution loop** (`WHILE current_turn < MAX_TURNS`). Bounding turns prevents infinite loops if the model encounters unresolvable errors.

Each iteration transitions through four stages:

1. **Thought:** The model deduces the next logical step based on conversational history. Externalizing internal reasoning reduces hallucination rates.
2. **Action:** The model invokes a specific MCP tool, supplying inferred arguments (e.g., `deploy_intent(name="api", port=80)`).
3. **Yield (Pause):** The orchestrator pauses generation, dispatches the call across the backend boundary, executes the action, and awaits the response.

4. **Observation:** The system injects the physical execution result (success or error payload) back into the LLM context window.

The model evaluates the observation and determines whether further actions are required or if task objectives have been fulfilled. Once complete, it emits its **Final Answer**, returning control to the user.

Figure 15 illustrates this feedback flow:

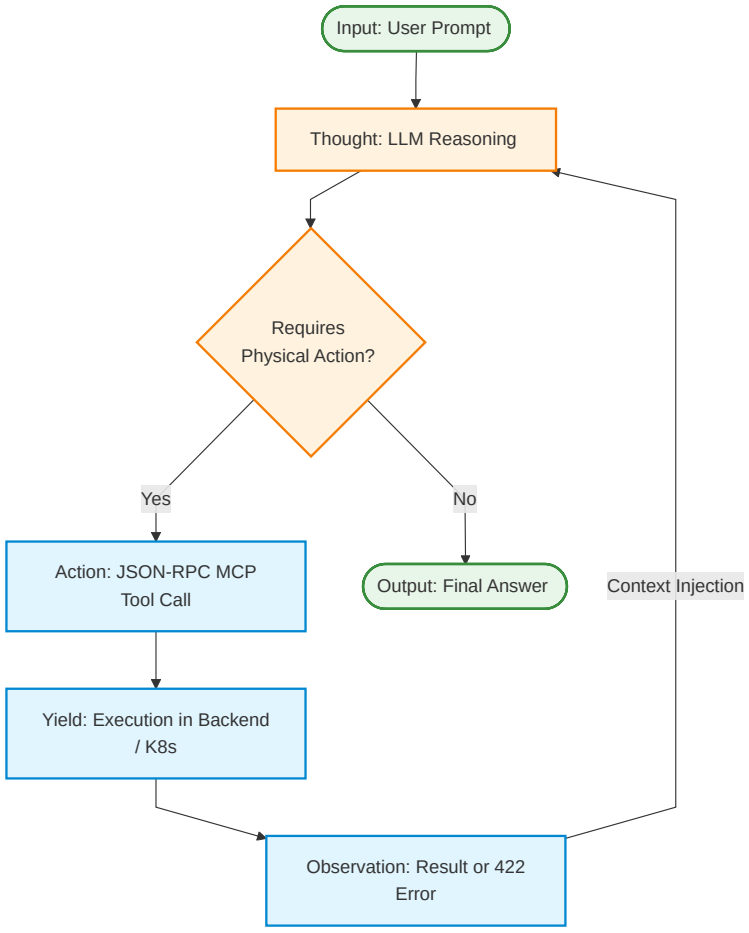



***Figure 15:*** *Flowchart of the ReAct (Reasoning and Acting) cognitive loop.*

Algorithm 3 formalizes this orchestration engine:

```
INPUT:
 user_prompt -> Natural language text string
 conversation_history -> Active session context memory

OUTPUT:
 final_response -> Natural language or Markdown string

BEGIN
  APPEND user_prompt TO conversation_history
  Variable current_turn = 0
  Variable MAX_TURNS = 5 // Fail-safe: prevents infinite loops and token budget depletion

  WHILE current_turn < MAX_TURNS DO
    // 1. LLM Inference (Thought + Action)
    Variable llm_response = NetworkInference(conversation_history, mcp_tools)

    IF llm_response IS plain_text THEN
      // Agent concluded task and addresses human user
      RETURN llm_response
    END IF
```

```
21
22      IF llm_response IS tool_invocation THEN
23        Variable function_name = llm_response.getName()
24        Variable arguments = llm_response.getArguments()
25        Variable action_result
26
27        TRY
28          // 2 & 3. Execution and Yield
29          action_result = ExecuteLocalProcess(function_name, arguments)
30        CATCH HttpException AS error
31          // Serialize error for LLM context digestion
32          action_result = error.getHumanReadableMessage()
33        END TRY
34
35        // 4. Observation
36        APPEND "Tool returned: " + action_result TO conversation_history
37      END IF
38
39      current_turn = current_turn + 1
40    END WHILE
41
42    RAISE Exception("Reasoning turn limit exceeded. Agent is stalled.")
43  END
```

***Algorithm 3:*** *Cognitive Orchestration Loop (ReAct Loop).*

Enforcing `MAX_TURNS` represents a critical operational guardrail, preventing runaway token consumption if the model loops over unresolvable errors.

## 6.3.2. Security Feedback Loop and Autonomous Self-Correction

The central operational advantage of ReAct lies in the `CATCH HttpException` branch (line 30, Algorithm 3). The Hexagonal Backend rejects non-compliant intents with deterministic HTTP `422 Unprocessable Entity` responses when policies are violated (e.g., attempting to bind port 22).

Rather than terminating the session, the MCP Server serializes the HTTP `422` error into a structured **Observation** injected into the model's prompt.

This triggers self-correction:

1. The agent reads that its previous action was rejected due to a port policy violation.
2. The reasoning trace (*Thought*) deduces that port 22 is forbidden by institutional policy.
3. Autonomously, the agent avoids re-raising the crash; it executes a new action with a compliant port (e.g., port 8000) or prompts the user for clarification in natural language.

This feedback loop absorbs syntax and parameter errors within the agent layer before requiring manual operator intervention. The Hexagonal Architecture provides deterministic boundaries, while ReAct allows the agent to adjust inputs until they conform to Golden Path specifications.

# Chapter 7. HITL Pattern Implementation and Finite State Machine (FSM) Governance

The cognitive agent design established in preceding chapters equips the system with semantic extraction and tool reasoning capabilities. However, fully delegating provisioning decisions on critical infrastructure to a probabilistic model is an operational vulnerability rather than an engineering milestone. Container orchestration in enterprise environments demands governance, legal accountability, and operational domain knowledge that exceed foundational model capabilities.

This chapter details the architectural solution implemented to govern the AI agent: the *Human-In-The-Loop* (HITL) pattern. It provides theoretical grounding for mandatory human authorization gates and explains how this separation of responsibilities is enforced through an immutable Finite State Machine (FSM) and an asynchronous operations dashboard.

## 7.1. Grounding the Human-In-The-Loop (HITL) Pattern

In IT automation, autonomy is frequently classified into five tiers (loosely adapted from SAE autonomous driving levels). Level 0 represents manual shell script execution, while Level 5 (Full Autonomy) denotes an autonomous system monitoring, deciding, and executing infrastructure mutations without human supervision.

While commercial trends push toward Level 5, this thesis argues that in enterprise Platform Engineering, the sound engineering approach is Level 4 supervised automation governed via **Human-In-The-Loop (HITL)** controls.

### 7.1.1. Risks of Unsupervised Autonomy (Level 5)

Granting unmediated write credentials (e.g., a `ClusterAdmin` service account token) to probabilistic neural networks introduces systemic operational risks that automated testing cannot eliminate:

1. **Topological Hallucination:** Relying on probabilistic token generation, LLMs lack intrinsic factual verification over physical cluster state. If an agent hallucinates that resolving a transient network issue requires deleting and recreating a critical production `Deployment`, an unsupervised system would execute the teardown immediately.
2. **Uncontrolled Financial Elasticity:** In public cloud environments (AWS, GCP, Azure), compute resources follow pay-as-you-go billing. An unsupervised agent trapped in an anomalous reasoning loop could auto-provision oversized hardware, depleting institutional operational budgets.
3. **Change Freeze Violations:** Probabilistic models lack organizational context regarding institutional change freezes. An unsupervised agent might trigger high-risk reconfigurations during peak registration or grading windows. Human operator oversight ensures deployments are restricted to authorized maintenance intervals.

### 7.1.2. Legal Liability, ITIL Compliance, and Responsibility Distribution

Beyond technical feasibility, HITL governance is a regulatory necessity. Industry best practice frameworks such as **ITIL 4** (*Information Technology Infrastructure Library*) in Release Management [32], alongside ISO/IEC 27000 cybersecurity standards, mandate strict auditability and operational change governance.

Legally, an artificial intelligence model possesses no juridical personality. If *Agentic Deployer* instantiated an erroneous manifest that exposed sensitive data publicly (violating GDPR or the Spanish LOPD), legal accountability rests with the institution and systems operators, regardless of whether the configuration was inferred by a model.

To resolve this liability, the platform establishes a three-tier separation of responsibilities:

1. **The Researcher (The "What"):** Defines functional domain needs ("I need a WordPress portal for the History Department with 10 GB persistent storage").
2. **The AI Agent & MCP Server (The "How"):** Translates intent into declarative syntax, structuring natural language into strongly typed Hexagonal Architecture entities.
3. **The IT Systems Engineer (The "When" and "If"):** Enforces institutional governance and quota validation ("The request is syntactically valid, but storage capacity constraints require scheduling provisioning during tonight's maintenance window").

The model acts as an interactive assistant reducing specification time and YAML syntax errors, delegating final execution authority to a visual verification gate by operations personnel.

# 7.2. Finite State Machine (FSM) Design

Enforcing operational flow control cannot rely on primitive boolean flags (e.g., `is_valid = True`). In concurrent environments where multiple operators inspect the deployment queue, request lifecycles must be governed by a formal **Finite State Machine** (FSM).

Rooted in automata theory, an FSM guarantees that an entity occupies exactly one mutually exclusive state at any given timestamp $T$, and that transitions obey a finite set of directional rules.

## 7.2.1. Directed Acyclic State Transition Modeling

In *Agentic Deployer*, the lifecycle of a `DeploymentIntent` is modeled as a **Directed Acyclic Graph (DAG)**. Being acyclic is essential to security: state cannot flow backward in time. Once a deployment is physically executed on the Kubernetes cluster, the record cannot revert to a pending quarantine state.

The graph vertices (States) and directed edges (Authorized Transitions) encompass **six mutually exclusive nodes**:

- **Root Node (`PENDING_APPROVAL`):** The initial entry state. When the AI agent synthesizes a valid intent and `SecurityContextValidator` (Chapter 5) accepts it, the record enters quarantine. Zero physical provisioning occurs from this state.
- **Sink Node A (`REJECTED`):** An immutable terminal state. If the IT operator rejects the request in the dashboard, the intent transitions here. It remains permanently in the database for legal auditability, ignored by downstream infrastructure adapters.
- **Transit Node (`APPROVED`):** A transient intermediate state. When authorized by an operator, the request enters this node briefly, acting as the trigger to invoke outbound adapters (`FakeK8sAdapter` or `RealK8sAdapter`).
- **Sink Node B (`DEPLOYED`):** The final successful terminal state, reached if and only if the intent traversed `APPROVED` and the adapter confirms declarative manifests were generated and persisted to storage.
- **Sink Node C (`FAILED`):** An exceptional terminal state reached if manifest generation or cluster dispatch encounters

I/O exceptions or network failures. The record is retained for post-mortem analysis.

- **Sink Node D (`DELETED`):** Administrative teardown terminal state. Represents logical deletion of a previously deployed service, invoked via the `delete_university_service` MCP tool. The record is preserved for audit traceability.

Transitioning from initial quarantine to execution strictly requires explicit operator authorization. Figure 16 visualizes this unidirectional state flow:

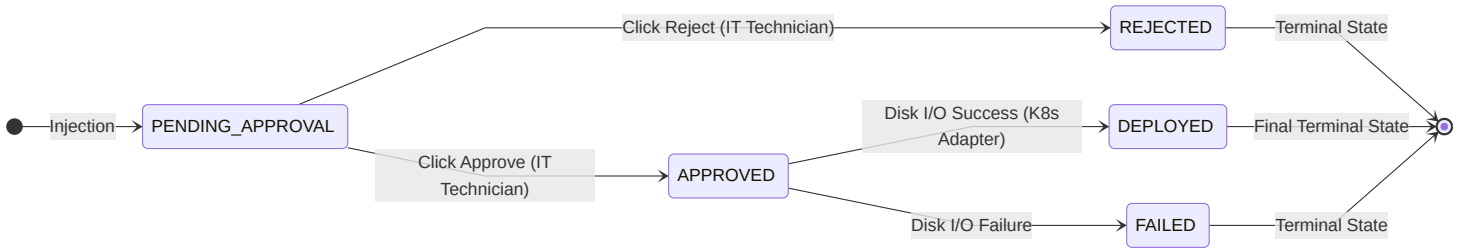



***Figure 16:*** *State transition graph governing the system's Finite State Machine (FSM).*

## 7.2.2. Concurrency Control and Race Condition Mitigation

Implementing an FSM in an asynchronous web backend (FastAPI) introduces concurrency considerations.

Consider a scenario where a request sits in `PENDING_APPROVAL`. Due to network latency or double-clicking, an operator triggers multiple consecutive HTTP requests, or two concurrent technicians click "Approve" and "Reject" nearly simultaneously.

Without architectural safeguards, parallel requests could induce race conditions in the database, attempting to both provision and reject the same workload simultaneously.

To mitigate this, transitions enforce sequential pre-condition validation (*check-then-act*) reflecting **Optimistic Locking** principles. The algorithm verifies that the current database state equals `PENDING_APPROVAL` prior to mutating state. In the single-worker Uvicorn process environment of this prototype (documented in limitation 10.2.5), this sequential check ensures consistency. Scaling to multi-process deployments (Gunicorn with $N$ workers) would require database-level pessimistic locking (`SELECT FOR UPDATE`).

Algorithm 4 formalizes this state transition logic:

```
01 INPUT:
02  request_id -> UUID of the deployment record to transition
03  target_state -> Requested destination state (APPROVED or REJECTED)
04
05 OUTPUT:
06  Updated_Record (If transition is authorized)
07  RAISE HttpConflictError (409) on invalid state mutation attempt
08
09 BEGIN
10   // 1. Record Retrieval and Existence Check
11   Variable record = DatabaseRepository.get(request_id)
12
13   IF record IS NULL THEN
14     RAISE HttpNotFoundError(404)
15   END IF
16
17   // 2. Invariant DAG Check: Mutations only permitted from PENDING_APPROVAL
18   IF record.status NOT EQUALS "PENDING_APPROVAL" THEN
```

```
19     RAISE HttpConflictError(
20       409,
21       "State Conflict. The request has already been processed " +
22       "(Current Status: " + record.status + ")."
23     )
24   END IF
25
26   // 3. Atomic State Mutation
27   record.status = target_state
28   record.updated_at = GetCurrentSystemTimestamp()
29
30   // 4. Secondary Adapter Activation (Side Effects)
31   IF target_state EQUALS "APPROVED" THEN
32     TRY
33       K8sAdapter.deploy(record.intent)
34       record.status = "DEPLOYED" // Second transition
35     CATCH IOException AS error
36       record.status = "FAILED"
37     END TRY
38   END IF
39
40   RETURN record
41 END
```

***Algorithm 4:*** *Immutable State Machine Transition (FSM_Transition).*

Under this algorithm, if a double-click occurs, the first HTTP thread mutates the record to `REJECTED` (or `APPROVED`). The second thread evaluates line 18, detects that the state is no longer `PENDING_APPROVAL`, aborts execution, and returns an immediate HTTP `409 Conflict` response. This prevents dual execution [33].

# 7.3. Asynchronous Operations Dashboard

Implementing HITL controls requires distinct user interfaces per role. Researchers interact through a flexible conversational UI (Streamlit), whereas operations engineers interact through a deterministic, low-latency control panel.

To serve this need, the platform provides an independent **Operations Dashboard** (`dash.html`), implemented as a lightweight Single-Page Application (SPA) served directly by FastAPI.

## 7.3.1. Synchronization Strategy: Lightweight Polling vs. WebSockets

Requests generated by the AI enter the system asynchronously. In high-frequency applications (e.g., trading platforms), state is typically synchronized via persistent bidirectional WebSockets (`ws://`). However, maintaining persistent WebSocket threads between browser dashboards and the backend introduces unnecessary memory and connection overhead for an infrastructure audit panel where 3-second display latency is operationally inconsequential.

Therefore, the platform adopts an **Active Lightweight Polling** strategy using native browser `fetch` requests:

1. **Periodic Inquiries:** The client invokes `GET /hitl/pending` at regular intervals (every 3,000 to 5,000 milliseconds).
2. **Transactional Query Execution:** The backend queries pending intents from SQLite, leveraging indexed lookups to preserve sub-millisecond execution times.
3. **Dynamic DOM Hydration:** The client processes the JSON payload, dynamically rendering card views for each quarantined request.

This decoupled, stateless design improves resilience. If an operator experiences network interruption, backend state remains intact; upon reconnection, the next polling tick immediately rehydrates the dashboard.

## 7.3.2. Deferred Execution and Declarative Manifest Synthesis

The dashboard presents extracted intent parameters to the operator: unique identifier, container image, network port, and CPU/RAM allocations.

Operators can execute two actions:

- **Reject:** Dispatches `POST /hitl/reject/{id}`, transitioning the state machine to `REJECTED` (§7.2). The record is retained for audit logging without generating cluster manifests.
- **Approve:** Dispatches `POST /hitl/approve/{id}`. Upon confirmation, the backend retrieves `DeploymentIntent` and passes it to `DeployPort`.

The adapter interpolates validated parameters into pre-tested Golden Path templates via `textwrap.dedent`, generating production-ready YAML manifests written to local storage (`{name}.yaml`). In enterprise production, this file is committed to a GitOps repository (ArgoCD) or submitted directly to the cluster API (§10.3.2).

Figure 17 displays the HITL Operations Dashboard served at `/frontend/index.html`. The top header displays real-time metrics (`Pending: 1`, `Approved today: 4`, `Rejected today: 1`) and backend connectivity status (`● Online`). The central table shows the quarantined request `dep-7f3a2c1b` for service `congreso-ia-departamento`, with typed action (`Create`), image (`nginx:alpine`), port (`8080`), and resource allocations (`500m / 256Mi`). Visual "Approve" and "Reject" controls trigger atomic FSM transitions:

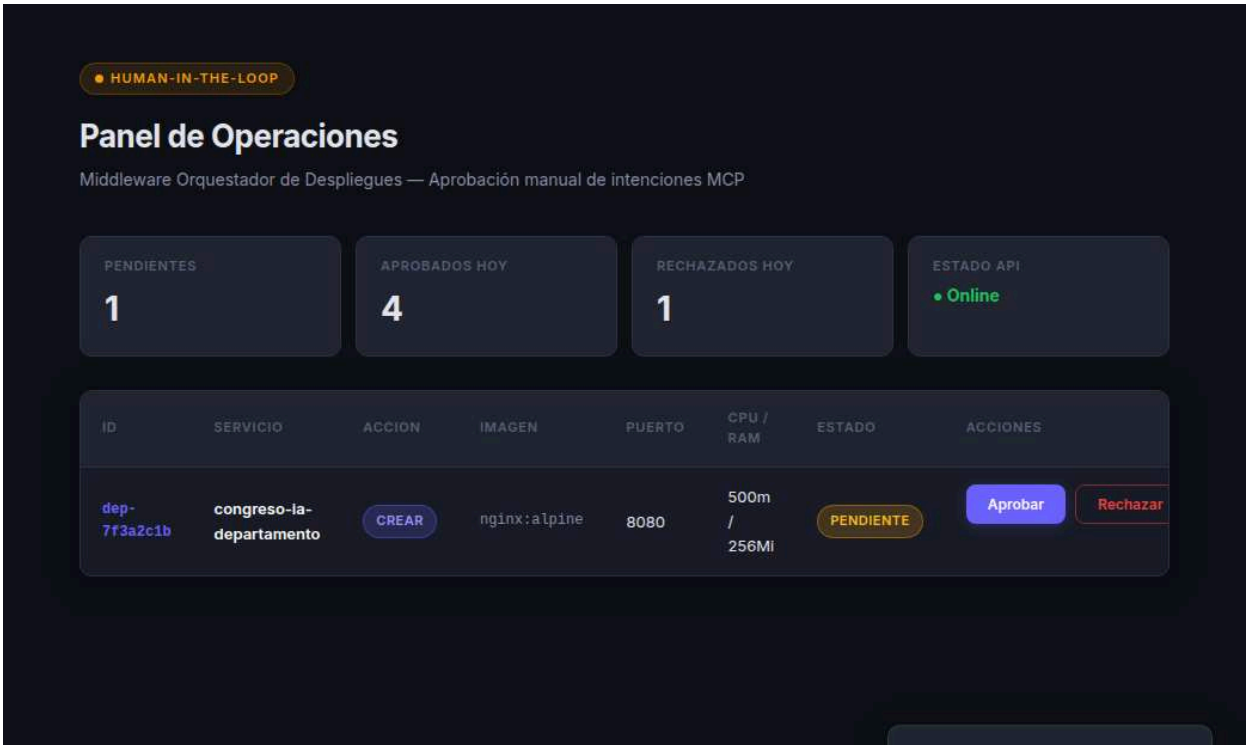



***Figure 17:*** *Screenshot of the HITL Operations Dashboard (`app/frontend/index.html`) displaying a request pending approval (`dep-7f3a2c1b`) alongside consolidated operational metrics.*

## 7.3.3. HITL Workflow Sequence Diagram

Figure 18 details interactions between the IT Technician and Backend during decision phases, illustrating both mutation vectors (approval and rejection):

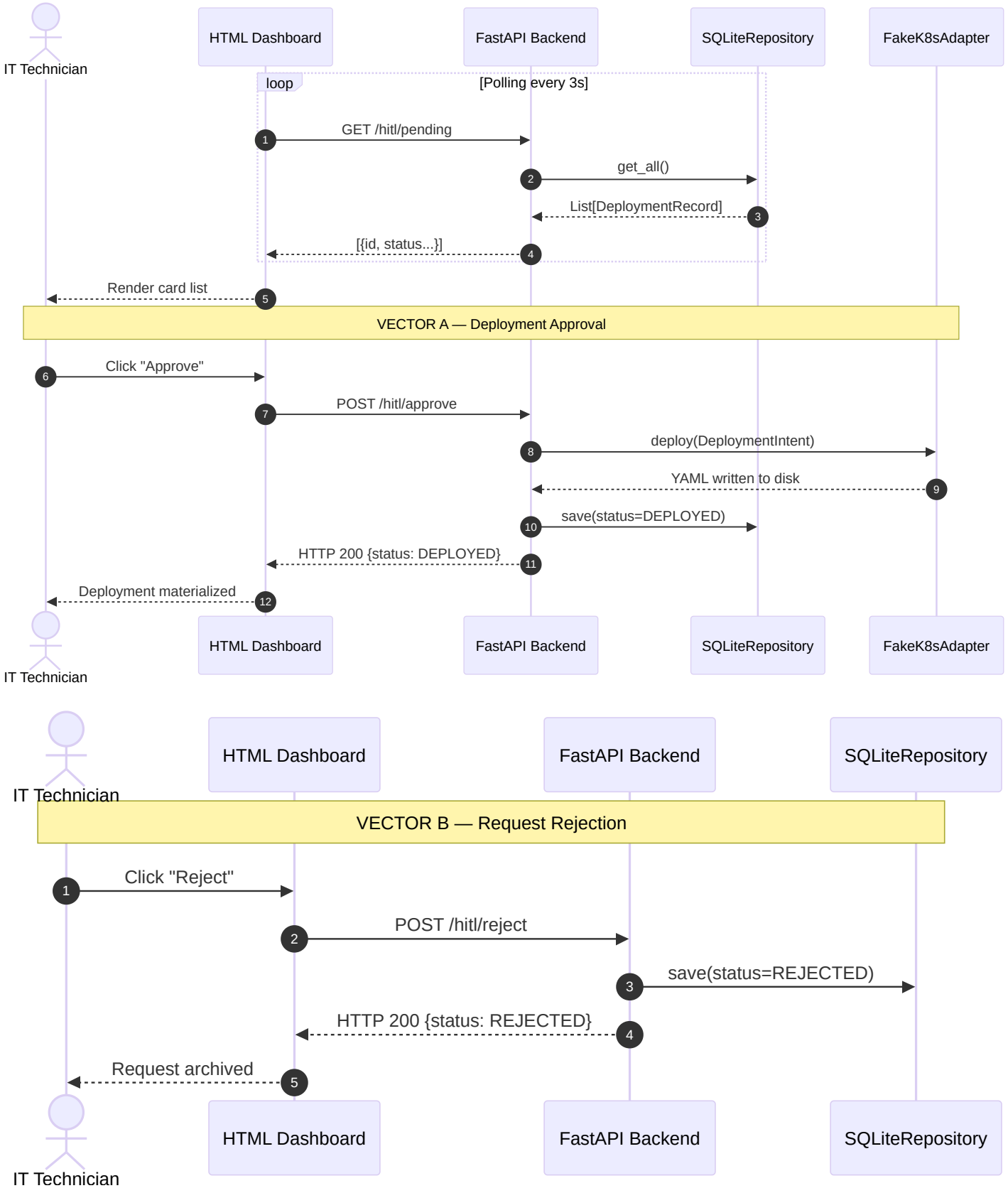



***Figure 18:*** *HITL Sequence Diagram. (Top) Asynchronous polling and Approval Vector. (Bottom) Rejection Vector (immutable terminal state).*

### 7.3.3.1. Decision Vectors and Polling Mechanics

Figure 18 illustrates two mutually exclusive decision paths:

1. **Asynchronous Polling:** The web dashboard periodically polls the backend every 3 seconds (`GET /hitl/pending`). The backend queries SQLite (`get_all()`), filters for `PENDING_APPROVAL`, and returns the active queue without requiring full page refreshes.
2. **Vector A — Approval & Materialization:** When the operator confirms specifications, they click "Approve". The browser sends `POST /hitl/approve`. The backend transitions the record to `APPROVED`, invokes `deploy()` on `FakeK8sAdapter`, writes the declarative YAML manifest, and transitions the state to `DEPLOYED`, notifying the UI with `HTTP 200`.
3. **Vector B — Rejection & Quarantine:** If specifications violate operational constraints, the operator clicks "Reject". The dashboard issues `POST /hitl/reject`. The backend skips infrastructure adapters, atomically switches the record to `REJECTED`, and archives the intent in SQLite without altering cluster state.

# 7.4. Researcher Feedback Channel: Asynchronous Status Notifications

A traditional limitation of HITL designs is lack of applicant visibility: operators track requests in dashboards, but applicants remain unaware of approvals, rejections, or provisioning delays.

This section documents the **researcher feedback channel**: a bidirectional mechanism enabling users to track deployment progress directly within the conversational chat.

## 7.4.1. Status Query Endpoint Design

A dedicated query endpoint `GET /hitl/status/{deployment_id}` was implemented in `BackendAPI`, tailored specifically for end users:

```
ROUTINE: Deployment Status Query (REST Endpoint)

ENDPOINT: GET /hitl/status/{deployment_id}
INPUT: deployment_id -> String (e.g., "dep-1a2b3c")
OUTPUT: StatusResponse (id, status, contextual_message)

BEGIN
  Variable record = DatabaseRepository.get(deployment_id)
  IF record IS NULL THEN
    ABORT WITH HttpException(404, "Deployment not found")
  END IF

  // Semantic mapping of technical FSM states to natural language
  Variable messages = NEW Dictionary(
    "PENDING_APPROVAL" -> "Your request is queued, pending review by an IT technician.",
    "APPROVED" -> "An IT technician has approved your request. Deployment in progress.",
    "DEPLOYED" -> "Your workload has been successfully deployed.",
    "REJECTED" -> "An IT technician has rejected this request.",
    "FAILED" -> "The deployment encountered a technical error on the cluster."
  )

  Variable contextual_message = messages[record.fsm_status]
  RETURN NEW StatusResponse(record.id, record.fsm_status, contextual_message)
END
```

The response returns the technical FSM state paired with a contextual natural language description, translating low-level state machine codes into user-friendly explanations.

## 7.4.2. Frontend Notification Panel (Streamlit)

The chat interface (`chat_app.py`) includes a passive tracking panel that activates automatically when the agent synthesizes a `PENDING_APPROVAL` deployment:

**1. Automatic Tracking ID Extraction:**
Following each agent turn, the frontend evaluates output text via regular expressions matching `dep-[a-f0-9]{7,8}`. When detected alongside pending status text, it appends the ID to `session_state.pending_deployments`:

```
ROUTINE: Identifier Extraction (HITL Frontend)

INPUT: response_text -> Text generated by the Cognitive Agent
STATE: pending_panel -> ID list rendered in the UI

BEGIN
  Variable detected_id = Regex("dep-[a-f0-9]{7,8}").search(response_text)

  IF detected_id EXISTS AND response_text CONTAINS "PENDING" THEN
    APPEND detected_id TO pending_panel
  END IF
END
```

**2. Persistent Tracking Widget:**
While pending deployments remain in `session_state`, a "My Pending Requests" panel renders below the chat dialogue. For each request, an interactive **"Refresh"** button invokes `GET /hitl/status/{id}` against the backend, displaying semantic badge colors:

| FSM State | Badge Color | User Notification Message |
|---|---|---|
| `PENDING_APPROVAL` | Yellow | "Your request is queued, pending review." |
| `APPROVED` | Blue | "Technician approved. Deployment in progress." |
| `DEPLOYED` | Green | "Your workload has been successfully deployed." |
| `REJECTED` | Red | "Request rejected. Please contact IT support." |
| `FAILED` | Red | "Technical failure. IT team has been notified." |

***Table 20:*** *Mapping of FSM states to visual indicators in the researcher's notification panel.*

**3. Automatic Lifecycle Closure:**
When the backend returns a terminal state (`DEPLOYED`, `REJECTED`, or `FAILED`), the deployment is cleared from the pending queue, and the agent injects a completion message into chat history informing the user of the final resolution.

Figure 19 illustrates the user experience in `chat_app.py`. Following request processing, the agent confirms registration and provides the tracking ID (`dep-7f3a2c1b`). Directly below, the UI renders the dynamic panel displaying current status (🟡 `PENDING_APPROVAL`) alongside context-aware guidance:

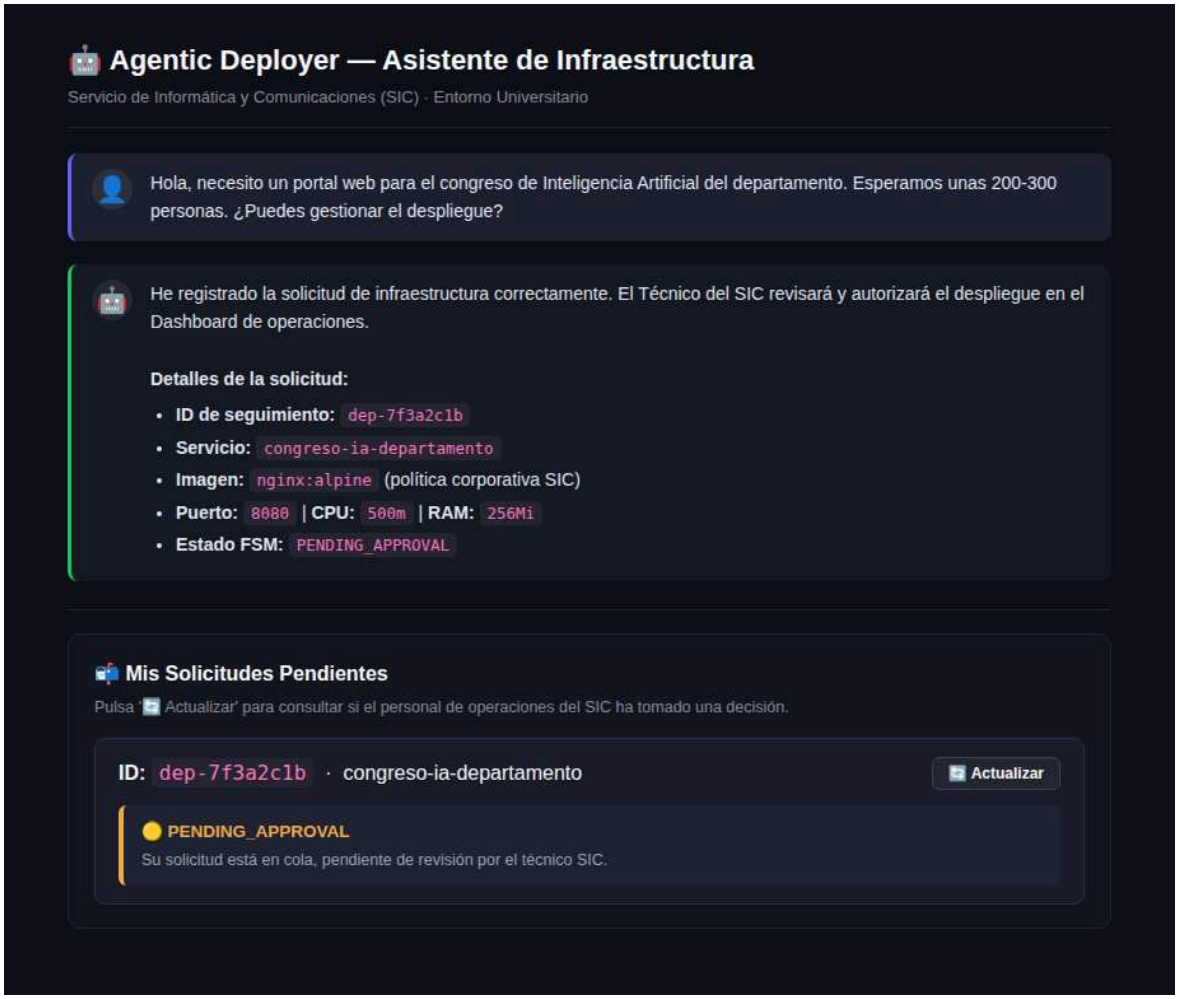



***Figure 19:*** *Screenshot of the researcher's conversational interface (Streamlit) showing the asynchronous feedback panel with* `PENDING_APPROVAL` *status and manual refresh control.*

## 7.4.3. Design Rationale: Explicit Polling vs. Push Webhooks

The alternative mechanism would be asynchronous push notifications (email or webhook dispatches). This was intentionally avoided for the research prototype due to:

1. **Unwarranted Infrastructure Complexity:** Configuring dedicated SMTP gateways or message brokers (RabbitMQ, Kafka) introduces deployment friction that hinders academic reproducibility.
2. **Architectural Symmetry:** The operator dashboard already relies on polling (§7.3.1). Maintaining identical paradigms in the end-user interface simplifies cognitive models and testing harnesses.
3. **Extensibility:** The `GET /hitl/status/{id}` endpoint provides a clean REST interface easily consumed by external messaging bots (Slack, Telegram, institutional email gateways) in future iterations without altering core backend logic (Interface Segregation Principle).

This architecture transforms HITL governance from a **unidirectional** pipeline (researcher $\rightarrow$ technician) into a **bidirectional** channel (researcher $\rightarrow$ technician $\rightarrow$ researcher), ensuring all stakeholders maintain visibility throughout the provisioning lifecycle.

# Chapter 8. Quality Assurance (QA) and Advanced Testing

Integrating Large Language Models (LLMs) into enterprise infrastructure orchestration introduces unprecedented challenges for Software Testing. In traditional software development, a deterministic function $f(x)$ consistently yields predictable output $y$. In an agentic system, however, identical input prompts can produce syntactically divergent outputs depending on model sampling temperature, inference seed, or slight syntactic reformulations.

To structure the verification and validation strategy for *Agentic Deployer*, this Master's Thesis adapts the conceptual framework of the **Test Pyramid** introduced by Mike Cohn [34]. In its original formulation for deterministic software, Cohn arranges testing efforts across three tiers based on cost, speed, and granularity: a broad base of fast, isolated unit tests; an intermediate layer of service or integration tests; and a narrow apex of end-to-end user interface tests.

Because Cohn formulated this model for classical software, this thesis **adapts and extends the framework to address Generative Artificial Intelligence** (conceptually illustrated in **Figure 20**), establishing a **Hybrid Testing Pyramid** across three distinct levels:

1. **Base (Deterministic Unit and Integration Testing):** Exhaustive boundary value analysis, static type enforcement, and domain invariant verification within the Hexagonal Domain core, guaranteeing the integrity of business policies regardless of language model outputs.
2. **Intermediate Tier (Property-Based Testing and Mutation Testing):** Evaluating algorithmic resilience against massive randomized inputs via property generation (*Hypothesis*), combined with auditing the fault-detection quality of the test suite itself through Abstract Syntax Tree (AST) fault injection (*mutmut*).
3. **Apex (Metamorphic Testing of the Cognitive Tier):** Replacing traditional brittle UI tests, the pyramid's apex evaluates model inference robustness via *Metamorphic Testing*, assessing semantic invariance of structured tool extractions under syntactic permutations, lexical noise, and natural language variations.

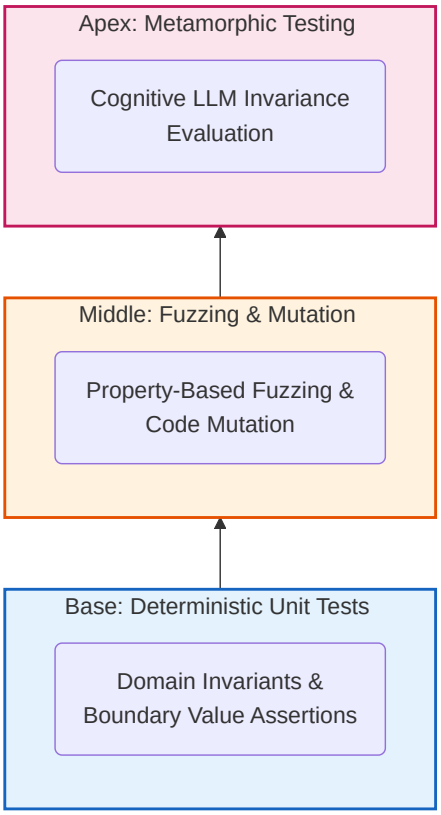



***Figure 20:*** *Architecture of the Hybrid Testing Pyramid implemented in Agentic Deployer, adapting Mike Cohn's classic model to the requirements of Generative Artificial Intelligence.*

# 8.1. Domain and Integration Testing: Validating the Hexagonal Boundary

The foundation of the quality assurance pyramid resides within the Domain Layer. As established in Chapter 5, the AI agent operates as an unprivileged external user; its ability to impact infrastructure is bounded by the deterministic rigor of `SecurityContextValidator` and Pydantic domain invariants.

Consequently, the first QA objective is ensuring that this hexagonal perimeter remains resilient prior to connecting stochastic models. This verification is conducted via isolated **Unit Testing (*White-Box Testing*)** executed locally using `pytest`.

## 8.1.1. Boundary Value Analysis and Equivalence Partitioning

Test case design follows two formal white-box techniques: Boundary Value Analysis and Equivalence Partitioning.

The `DeploymentIntent` contract dictates that network ports must be numeric within `[1, 65535]`, while business rules prohibit privileged ports ($< 1024$). The test suite validates these contracts using three targeted input vectors:

1. **Valid Partitions (Happy Path):** Instantiating intents with unprivileged ports (e.g., 8080 or 3000) and images from trusted registries (e.g., `harbor.universidad.edu/nginx:1.20`). Tests assert that zero security exceptions are raised.
2. **Lower Boundary Values (Root Privilege Violation):** Intentionally injecting ports `1023` and `22`. The test framework asserts that the validator raises `SecurityViolationError`, aborting execution immediately.
3. **Upper Boundary Values and Overflows:** Instantiating requests with ports `65535`, `65536`, and negative values

(`-80`). The domain model must reject invalid bounds immediately by raising `ValueError` prior to security validator invocation, enforcing *Fail-Fast* behavior.

## 8.1.2. Negative Testing and Configuration Drift Prevention

Alongside port checks, the test suite audits container image tags. Integration tests evaluate `SecurityContextValidator` rules prohibiting mutable `:latest` tags.

The test suite simulates requests with invalid variants, such as `ubuntu:latest`, `nginx:LATEST`, and untagged images (e.g., `redis`, where orchestrators implicitly infer `:latest`). Across all variants, tests assert that execution raises a traceable `SecurityViolationError`.

This baseline tier (executing in sub-milliseconds with $>95\%$ line coverage over the Hexagonal core) verifies that absent a fully compliant request matching institutional ITIL standards, access to infrastructure adapters remains blocked, regardless of prompt phrasing.

## 8.1.3. Testing the Status Query Layer (Researcher Feedback Channel)

Following implementation of `GET /hitl/status/{id}` (§7.4.1), a dedicated integration module (`test_status_endpoint.py`) was introduced to validate the researcher return channel. This module verifies a critical invariant: the status polling endpoint must be **strictly idempotent**—repeated invocations must never mutate FSM state.

The suite validates six distinct behaviors:

| Test Case | Validated Behavior |
|---|---|
| `test_unknown_id_returns_404` | Non-existent UUIDs return `HTTP 404` with a descriptive error payload |
| `test_pending_approval_returns_correct_status` | Newly registered deployments return `PENDING_APPROVAL` with contextual guidance |
| `test_deployed_status_after_approval` | Following `POST /hitl/approve/{id}`, status reflects `DEPLOYED` |
| `test_rejected_status_after_rejection` | Following `POST /hitl/reject/{id}`, status reflects `REJECTED` |
| `test_response_includes_deployment_metadata` | Responses contain accurate `name`, `image`, and `port` attributes |
| `test_status_endpoint_does_not_mutate_state` | Three consecutive queries maintain `PENDING_APPROVAL` without state mutation |

***Table 21:*** *Test cases for the researcher status query layer.*

The final idempotency test is critical: it guarantees that read operations from the researcher conversational channel never interfere with write operations from the operator dashboard.

### 8.1.4. Quality Gates and Static Code Analysis

To prevent technical debt and type safety regressions during development, the local CI pipeline (`scripts/run_tests.sh`) incorporates three formal Quality Gates configured in `pyproject.toml`:

1. **Static Analysis and Linting with Ruff:** Enforces Python 3.11 conventions (`target-version = "py311"`) with a 100-character line length ceiling (`line-length = 100`). Active rule suites include:
    - `E` and `W`: Standard `pycodestyle` formatting rules.
    - `F`: Logical defect and unused variable detection via `pyflakes`.
    - `I`: Deterministic import sorting via `isort` (`known-first-party = ["app"]`).
    - `UP`: Syntax modernization for Python 3.11+ via `pyupgrade`.
    - `B`: Bug prevention via `flake8-bugbear`.
    - `SIM`: Code simplification via `flake8-simplify`.
    - `C901`: **McCabe Cyclomatic Complexity** auditing capped at `max-complexity = 15` [35], preventing deeply nested control flow while permitting idiomatic pattern matching.
2. **Strict Static Type Checking with Mypy:** Enforces type safety across hexagonal boundaries (`python_version = "3.11"`):
    - `disallow_untyped_defs = true`: Forbids untyped function or method signatures.
    - `check_untyped_defs = true`: Inspects internal expressions for type consistency.
    - `warn_return_any = true`: Flags explicit functions returning unvetted `Any` values.
3. **Pragmatic Code Coverage Thresholds:** Enforced via `pytest-cov` at a minimum 80% line coverage gate (`--cov-fail-under=80`). Aligned with industry engineering standards (Google [36]), artificially enforcing 100% coverage is avoided to prevent low-value boilerplate testing. Coverage focuses on application domain logic, excluding third-party UI drivers (`app/agent_layer/chat_app.py`, `app/infrastructure/*`).

## 8.2. Property-Based Testing: Evaluating Stochastic Boundaries

Traditional example-based unit testing suffers from cognitive developer bias: engineers design test assertions matching assumptions they made while coding. In systems driven by Large Language Models where inputs (agent-generated JSON payloads) exhibit high variance, example testing is insufficient.

To verify that malformed outputs or hallucinations cannot bypass security constraints, this thesis integrates **Property-Based Testing** (PBT). Rather than asserting that input $A$ produces output $B$, PBT asserts that "for *any* randomized input conforming to structural bounds, the system must satisfy a universal logical invariant".

### 8.2.1. Stochastic Entropy Injection and Generation Strategies

PBT is implemented in `app/tests/test_security.py` using the **`Hypothesis`** scientific testing framework [37]. Hypothesis functions as an intelligent dynamic fuzzer, generating hundreds of randomized permutations per second guided by domain-specific **Strategies**:

- `valid_name`: Generates Kubernetes DNS-compliant identifiers (`^[a-z][a-z0-9\-]{0,29}$`).

- `valid_cpu` and `valid_ram`: Samples valid resource strings (`"100m"`, `"500m"`, `"1"`, `"64Mi"`, `"128Mi"`, `"1Gi"`).
- `safe_image` vs. `unsafe_latest_image`: Generates images from trusted registries (`docker.io/`, `harbor.universidad.edu/`) with semantic tags contrasted against `:latest` injections.
- `safe_port` vs. `unsafe_port`: Valid user-range integers (`[1024, 65535]`) contrasted against privileged integers (`[1, 1023]`).
- Composite strategy `@st.composite def deployment_intent_dict`: Assembles full intent dictionaries with configurable failure injection (unsafe ports, forbidden tags, plaintext secrets).

Nine invariant properties are implemented in `TestSecurityContextValidator`, summarized in **Table 22**:

| Property Test | Evaluated Condition | Asserted Invariant |
|---|---|---|
| `test_privileged_port_always_raises` | Port forced into privileged range (< 1024) | Validator **always** raises `SecurityViolationError` citing the port |
| `test_latest_tag_always_raises` | Image forced with `:latest` tag | **Always** raises `SecurityViolationError` citing tag prohibition |
| `test_both_violations_detected_simultaneously` | Port < 1024 and `:latest` tag injected concurrently | Detects **both** violations in a single pass (`len(violations) == 2`) |
| `test_safe_intent_never_raises` | Port $\ge 1024$ and semantic image tag | **Never** raises exceptions; validates cleanly across all iterations |
| `test_delete_action_always_passes` | Workload deletion (`action = DELETE`) with arbitrary data | Teardown operations **never** fail due to creation policies |
| `test_untrusted_registry_always_raises` | Image from unapproved registry (e.g., `hacker.io/miner:1.0`) | **Always** raises `SecurityViolationError` citing untrusted source |
| `test_hardware_quotas_always_raises` | CPU > 4000m or RAM > 8192Mi | **Always** raises `SecurityViolationError` indicating quota breach |
| `test_secrets_in_env_always_raises` | `env_vars` keys with substrings (`password`, `token`, `secret`, `key`) | **Always** raises `SecurityViolationError` for plaintext credentials |
| `test_database_without_storage_raises` | DB images (`postgres`, `mysql`, `redis`, `mongo`) lacking storage | **Always** raises `SecurityViolationError` requiring persistent volumes |

***Table 22:*** *Invariant properties exhaustively audited via Hypothesis in* `test_security.py`.

Algorithm 5 illustrates this property verification logic:

```
01 PROPERTY_TO_DEFEND: "For any randomized parameter combination synthesized by Hypothesis,
02 if the intent violates one or more institutional policies, SecurityContextValidator MUST
03 intercept it deterministically and accumulate all violations without unhandled exceptions."
```

```
04
05 BEGIN
06   // Composite Hypothesis Strategy (deployment_intent_dict)
07   Variable data_generator = Strategy.CompositeIntent(force_unsafe_port, force_latest_image)
08
09   FOR EACH intent_data INJECTED BY THE ENGINE (max_examples = 100) DO
10     Variable intent = NEW DeploymentIntent(intent_data)
11
12     TRY
13       validator.validate(intent)
14       // If no exception was raised, the intent MUST be strictly compliant
15       ASSERT (intent.internal_port >= 1024)
16       ASSERT (NOT_CONTAINS(intent.image, ":latest"))
17       ASSERT (IS_IN_WHITELIST(intent.image))
18     CATCH SecurityViolationError AS err
19       // Expected behavior upon receiving non-compliant parameters
20       ASSERT (Length(err.violations) >= 1)
21     CATCH ANY_OTHER_EXCEPTION AS critical_error
22       // Robustness violation: unhandled exception in core domain
23       FAIL_TEST("Unhandled crash in hexagonal core: " + critical_error)
24     END TRY
25   END FOR
26 END
```

***Algorithm 5:*** *Property-Based Test for Hexagonal Invariants (Fuzzing with Hypothesis).*

Passing all 9 property-based tests validates that the system reliably absorbs random parameter combinations generated by language models.

### 8.2.2. Algorithmic Failure Minimization (*Shrinking*)

A core strength of Hypothesis is **Shrinking**. When the fuzzer encounters a counterexample breaking an invariant, it does not output an unreadable multi-thousand-character string.

Instead, the framework initiates graph reduction and binary search routines to systematically prune failing inputs. It reduces integer values, removes extraneous characters, and eliminates array elements until isolating the **minimal failing counterexample** reproducing the fault (e.g., isolating a single null byte `\0` among hundreds of variables).

## 8.3. Test Suite Quality Audit: Mutation Testing

In software engineering, high line coverage does not guarantee that underlying test assertions are capable of detecting logical defects. In mission-critical infrastructure, relying strictly on code coverage metrics creates false confidence.

To evaluate assertion efficacy in *Agentic Deployer*, **Mutation Testing** was incorporated via `mutmut`.

The core axiom of mutation testing answers a meta-analytical question: *"If a critical bug is intentionally injected into source code, will existing test suites detect it and fail?"*

### 8.3.1. AST Mutant Generation at Runtime

Mutation testing operates directly on Python's Abstract Syntax Tree (AST), dynamically synthesizing dozens of source code clones ("Mutants"). Each mutant introduces a deliberate syntactic or logical modification:

If `SecurityContextValidator` specifies:
`IF port < 1024 THEN RAISE Error`

The engine injects mutations:

- **Mutant 1 (Operator Mutation):** `IF port <= 1024` $\rightarrow$ Mutates `<` to `<=`, leaving boundary port 1024 vulnerable.
- **Mutant 2 (Logical Inversion):** `IF port >= 1024` $\rightarrow$ Inverts conditions, approving privileged ports (22, 80) and rejecting safe ports.
- **Mutant 3 (Constant Mutation):** `IF port < 1025` $\rightarrow$ Alters threshold values, creating a single-port vulnerability window.

## 8.3.2. Evaluating Mutant Survival (*Killed* vs. *Survived*)

The framework runs the full `pytest` suite against each mutant sequentially:

1. **Mutant Neutralized (*Killed*):** If a test assertion fails, the test suite successfully detected the injected bug. The mutant is eliminated. This indicates effective assertion coverage.
2. **Mutant Surviving (*Survived*):** If code is altered (e.g., permitting port 22) and the test suite passes green, the mutant survived. This highlights an assertion coverage gap.

## 8.3.3. Empirical Mutation Audit Results

Table 23 details real `mutmut` execution results configured via `test_mutmut.ini` (Scope: `app/application/` and `app/domain/`):

| Audited Module | Surviving Mutants | Defect Nature | Security Criticality |
|---|---|---|---|
| `domain.exceptions` | 2 | Mutations in `SecurityViolationError` constructor string formatting | Low — string formatting, not logic |
| `security_validator._parse_cpu` | 5 | Unit string conversion (`"500m"` $\rightarrow$ int) lacking direct unit tests | Medium — parsing utility |
| `security_validator._parse_ram` | 19 | Memory unit string parsers (`"256Mi"`, `"1Gi"`) | Medium — parsing utility |
| `security_validator.validate` | 3 | Compound rule boundary conditions (concurrent violations) | Medium — business rules |
| `use_cases.execute` | 1 | Alternative execution branch for `DELETE` action | Low |
| **Hexagonal Core Subtotal** | **30** | | |
| `agent.FakeLLMClient` | 54 | Demonstration mock without dedicated test suite (by design) | Not applicable |

| Audited Module | Surviving Mutants | Defect Nature | Security Criticality |
|---|---|---|---|
| `agent.AgentOrchestrator` | 45 | Cognitive orchestrator: testing stochastic LLM code with deterministic assertions is out of scope | Not applicable |
| `agent_layer.tools` | 14 | MCP tools requiring network mocks | Low |
| **Total Mutants** | **143** | | |

***Table 23:*** *Mutation Testing results categorized by module.*

**Result Interpretation:**

Analysis shows that **69.9% of survivors** (99 of 143) reside in modules excluded from formal audit scopes by architectural design: `FakeLLMClient` is un-tested demo code, while `AgentOrchestrator` manages stochastic model calls that cannot be audited with deterministic assertions.

Within the **hexagonal core**, the 30 surviving mutants concentrated in resource parsing helpers (`_parse_cpu`, `_parse_ram`) converting strings like `"500m"` or `"1024Mi"` to integers. These survivors pinpointed assertion gaps that $>95\%$ line coverage metrics had obscured.

## 8.3.4. Technical Debt Mitigation and Final Results

Identified gaps were **remediated in the final codebase**:

1. **Refactored Parsers:** Removed unreachable defensive blocks (`try...except ValueError`) since Pydantic domain models already guarantee well-formed strings at the boundary, and added parameterized assertions covering upper units (e.g., Terabytes `Ti`).
2. **Concurrent Violation Assertions:** Expanded `test_security.py` fixtures to assert exact lengths of accumulated violation lists when multiple errors occur simultaneously.

Following remediation, 96 of 98 injected mutants were successfully killed:

```
$ mutmut run
Running mutation testing
- 98/98  [Killed: 96 | Skipped: 0 | Timeout: 0 | Suspicious: 0 | Survived: 2]

$ mutmut results
app.domain.exceptions.x‖SecurityViolationError‖__init____mutmut_2: survived
app.domain.exceptions.x‖SecurityViolationError‖__init____mutmut_4: survived
```

The 2 surviving mutants represent cosmetic exception string formatting, with zero impact on domain validation logic.

# 8.4. Metamorphic Testing: Methodology and Cognitive Invariants

While unit, property, and mutation testing thoroughly verify the Hexagonal Core, deterministic methods fail when assessing the cognitive tier (`AgentOrchestrator`).

## 8.4.1. The Test Oracle Problem in Generative AI

This limitation stems from the **Test Oracle Problem** [38]: in traditional engineering, an oracle knows the exact expected output for any given input ($2 + 2 = 4$).

Large Language Models, however, do not produce identical character-by-character outputs. Given an infrastructure request, a model might emit *"Registered service on port 80"* or *"Service configured on port 80"*. Both are semantically valid, yet strict string assertions (`assert output == expected`) fail.

To resolve this without resorting to manual, non-reproducible inspection, this research implements **Metamorphic Testing**.

## 8.4.2. Formal Definition of Metamorphic Relations (MR)

Metamorphic Testing posits that while predicting exact textual outputs is impossible, one can formally predict how outputs *must change* (or remain invariant) under controlled input transformations. Each formal rule is a **Metamorphic Relation (MR)**.

For semantic intent extraction, the core property is **Semantic Identity**: *Any lexical, syntactic, or linguistic variation in the user's prompt that preserves underlying technical meaning must invariantly produce an identical structured tool call (`Tool Call`), isolating matching parameters (`name`, `image`, `internal_port`).*

This property is formalized in `app/tests/test_metamorphic.py` across six Metamorphic Relations:

- **MR-1 (Syntactic Permutation):** Invariance under argument order reordering (`name`, `image`, `port`). Verifies positional independence.
- **MR-2 (Lexical Noise Invariance):** Invariance under conversational pleasantries and greetings. Verifies parameter isolation from ambient noise.
- **MR-3 (Typographical Invariance):** Invariance under casing alterations (`UPPERCASE` / `lowercase`).
- **MR-4 (Incremental Composition):** Equivalence between a single declarative prompt and multi-turn conversational elicitation.
- **MR-5 (Semantic Paraphrasing):** Invariance under synonyms, passive voice, and equivalent phrasings.
- **MR-6 (Linguistic Invariance):** Semantic extraction equivalence across languages (Spanish vs. English).

## 8.4.3. Dual Evaluation Strategy: Automated CI/CD vs. Real Inference

Systematically executing LLM inferences during every CI commit is impractical due to execution latencies (5 to 45 seconds per turn) and GPU costs.

The test harness adopts a **dual-mode evaluation design**:

1. **Deterministic Mode (CI/CD Pipeline):** In standard CI builds, the test runs against `FakeLLMClient`. This component, built on compiled regular expressions, executes all 6 metamorphic relations in $<0.15$ seconds, verifying test harness integrity on every commit.
2. **Cognitive Audit Mode (Real Model Evaluation):** Setting `RUN_REAL_LLM=true` routes execution to

`OllamaLLMClient` (local GPUs) or cloud APIs. The harness tests model neural weights under zero-shot prompt conditions without assistive system instructions.

## 8.4.4. Transition to Empirical Results

With the verification pyramid established (Unit, Integration, Property-Based, Mutation, and Metamorphic testing), the thesis transitions from methodology to **empirical experimental results**.

Quantitative benchmarks evaluating model families (baseline `qwen2.5:7b`, specialized `qwen2.5-coder:7b`, and cloud alternatives) across this metamorphic suite are detailed in **Chapter 9 (Section 9.3)**.

# Chapter 9. Experimental Results and Evaluation

Architectural design (Chapter 5), cognitive agent and MCP implementation (Chapter 6), human governance via HITL controls (Chapter 7), and quality assurance methodologies (Chapter 8) converge in this chapter for rigorous empirical evaluation. In enterprise infrastructure engineering, software validity relies not merely on theoretical component correctness, but on the ability to resolve real provisioning tasks predictably, securely, and efficiently under operational constraints.

This chapter structures empirical evaluation across six complementary pillars:

1. **Experimental Framework and Testbed (§9.1):** Specification of computational hardware, inference engines, technical preconditions, and statistical repeatability protocols.
2. **Real Case Studies (End-to-End Walkthroughs) (§9.2):** Traceability of system behavior across four operational scenarios: compliant baseline provisioning (*Happy Path*), perimeter resilience against prompt injection attacks, asynchronous human-in-the-loop governance (*HITL*), and deterministic exception-guided self-correction.
3. **Comparative Empirical Evaluation of LLM Architectures (§9.3):** Specialized evaluation of foundational models under two operating regimes: strict unassisted zero-shot metamorphic stress-testing versus architecture-assisted ReAct tool calling, contrasting local and cloud models.
4. **Cognitive Architecture Experiments (§9.4):** Evaluation of three conversational control paradigms (*Classic ReAct*, *Chain-of-Thought*, and *Stateful Slot Filling*) to overcome attentional limits in small local models (<8B parameters).
5. **Operational Impact Evaluation on ITSM Processes (§9.5):** Quantitative assessment of delivery timelines and cognitive overhead measured against traditional IT service management workflows.
6. **Threats to Empirical Validity (§9.6):** Methodological analysis of internal, external, construct, and conclusion validity threats in alignment with empirical software engineering guidelines.

## 9.1. Experimental Framework and Testbed

To ensure scientific reproducibility, all evaluations were conducted under controlled conditions. Execution sessions were captured as structured JSON telemetry traces (persisted in `demos/session_logs/`), recording model inference outputs without post-hoc manipulation.

The hardware and software environment representing a university research testbed is detailed in Table 24:

| Parameter | Specification |
|---|---|
| **Evaluation Workstation** | Mobile workstation: AMD Ryzen 7 5800H CPU (8 cores, 16 threads @ 3.2 GHz), 16 GB DDR4 RAM, NVIDIA GeForce RTX 3060 Laptop GPU (6 GB GDDR6 VRAM) |
| **Operating System** | Ubuntu 22.04 LTS (Linux Kernel 5.15), native execution environment |
| **Local Inference Platform** | Ollama v0.6.2 with NVIDIA CUDA acceleration |

| Parameter | Specification |
|---|---|
| **Local Baseline Model** | `qwen2.5:7b` (Q4_K_M quantization, ~4.7 GB VRAM footprint) |
| **Local Specialized Variant** | `qwen2.5-coder:7b` (Q4_K_M quantization, ~4.7 GB VRAM footprint) |
| **Evaluated Cloud Models** | Gemini 3.5 Flash (Google), Cohere Command R+ (Cohere), Ministral 8B (Mistral AI), Codestral (Mistral AI), Qwen 2.5 27B (Groq LPU) |
| **Application Backend** | FastAPI 0.115 hosted on Uvicorn ASGI server (single-worker mode) |
| **Persistence Layer** | SQLite 3.45 with durable on-disk ACID transactional support |
| **Repeatability Protocol** | Each scenario was executed in batches of 5 consecutive iterations to absorb network and sampling variance; latency metrics are reported as mean ($\mu$) $\pm$ standard deviation ($\sigma$) |

***Table 24:*** *Evaluation testbed for practical case studies.*

# 9.2. Real Case Studies (End-to-End Walkthroughs)

This section documents four real execution scenarios, tracing natural language inputs, MCP protocol mediation, hexagonal policy validation, and generated infrastructure artifacts.

To evaluate architectural stabilization over stochastic models transparently, walkthroughs utilize the general-purpose local model `qwen2.5:7b` under air-gapped Zero Data Retention conditions.

## 9.2.1. Scenario 1 — Standard Provisioning (*Happy Path*): Academic Conference Web Portal

This scenario traces the 10 operational steps required to provision an institutional web workload from conversational prompt to declarative Kubernetes YAML generation.

**STEP 1 — Researcher Input (Natural Language):**

> *«Hello, I need to publish the website for the departmental AI conference. We anticipate around 200 attendees. Can you handle this?»*

---

**STEP 2 — Cognitive Inference and Reasoning (ReAct Agent):**

`AgentOrchestrator` processes the request within the ReAct loop. The model infers a public HTTP workload, sizes resources appropriately, selects a hardened image (`nginx:alpine`), sets a standard non-privileged port (`8080`), and allocates resource bounds:

```
{
  "thought": "The user requested a web portal for an academic conference with 200 attendees. This
requires a standard HTTP web server. Adhering to institutional policies, I will utilize nginx:alpine on
port 8080 with 500m CPU and 256Mi RAM.",
  "tool_call": {
    "name": "format_deployment_intent",
    "arguments": {
      "name": "congreso-ia-departamento",
      "image": "nginx:alpine",
      "port": 8080,
      "cpu": "500m",
      "ram": "256Mi"
    }
  }
}
```


---

**STEP 3 — Tool Dispatch via MCP Server:**

The FastMCP server validates arguments against the tool schema and dispatches an HTTP `POST` request to the backend (`http://localhost:8000/deployments/intent`).

---

**STEP 4 — Reception at Primary Driving Adapter (FastAPI):**

The FastAPI REST controller deserializes the JSON payload into a typed `DeploymentIntent` domain entity.

---

**STEP 5 — Deterministic Validation in Hexagonal Core (`SecurityContextValidator`):**

The `RegisterDeploymentIntent` use case invokes `SecurityContextValidator` (Algorithm 1), auditing institutional policies:

| Security Rule Evaluated | Compliance Status |
|---|---|
| Unprivileged port (8080 $\ge$ 1024) | Compliant |
| Immutable tag (`:alpine` $\ne$ `:latest`) | Compliant |
| Trusted container registry (`docker.io/library/nginx`) | Compliant |
| CPU quota (500m $\le$ 2000m departmental limit) | Compliant |
| RAM quota (256Mi $\le$ 4096Mi departmental limit) | Compliant |
| Absence of plaintext secrets in environment variables | Compliant |

***Table 25:*** *Execution trace: Hexagonal core validation (Step 5).*

With zero violations detected, the SQLite repository persists the intent under `PENDING_APPROVAL` with UUID `dep-7f3a2c1b`.

---

**STEP 6 — Backend HTTP Response:**

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "id": "dep-7f3a2c1b",
  "status": "PENDING_APPROVAL",
  "name": "congreso-ia-departamento",
  "image": "nginx:alpine",
  "port": 8080,
  "cpu": "500m",
  "ram": "256Mi",
  "message": "Intent registered. Awaiting HITL approval from SIC technician."
}
```

---

**STEP 7 — Observation Feedback to LLM (ReAct Loop):**

The response payload is appended to context memory as an observation, instructing the agent to terminate reasoning.

---

**STEP 8 — Final Conversational Response to Researcher:**

> *«Your deployment request has been registered in the system. An IT operations technician will review and authorize provisioning in the control dashboard.*
>
> ***Registered Technical Specifications:***
> *- **Tracking ID:** `dep-7f3a2c1b`*
> *- **Service Name:** `congreso-ia-departamento`*
> *- **Container Image:** `nginx:alpine`*
> *- **Network Configuration:** Port 8080 | **CPU:** 500m | **RAM:** 256Mi*
> *- **Current Status:** `PENDING_APPROVAL` (Awaiting technical review)»*

---

**STEP 9 — Audit and Authorization in HITL Dashboard:**

An IT technician opens the dashboard at `/hitl` (Figure 17), reviews technical parameters, and clicks **«Approve»**:

```
# Operational curl equivalent executed by Dashboard
curl -X POST http://localhost:8000/hitl/approve/dep-7f3a2c1b
```

```
HTTP/1.1 200 OK
{"id": "dep-7f3a2c1b", "status": "DEPLOYED"}
```

The FSM executes the atomic transition: `PENDING_APPROVAL → APPROVED → DEPLOYED`.

---

**STEP 10 — Declarative YAML Manifest Materialization:**

`FakeK8sAdapter` interpolates parameters into the Golden Path template, generating multi-document Kubernetes manifests (`Deployment`, `Service`, `Ingress`) at `outputs/congreso-ia-departamento.yaml` (reproduced in full in **Annex A**).

---

**Consolidated Scenario 1 Metrics:**

| Execution Metric | Value (Mean of 5 iterations: $\mu \pm \sigma$) |
|---|---|
| ReAct loop iterations | 1.0 $\pm$ 0.0 |
| Cognitive inference latency (Local 7B LLM) | 12.99 s $\pm$ 1.52 s |
| Hexagonal validation latency | < 1 ms |
| Total duration to `PENDING_APPROVAL` | 13.12 s $\pm$ 1.55 s |
| Operator inspection time in Dashboard (*HITL*) | ~ 7 min (human review duration) |
| Total End-to-End Time | ~ 7.2 min vs. ~ 4,340 min in traditional ITSM |
| Relative Time-to-Market Reduction | **99.8%** |

***Table 26:*** *End-to-end performance metrics for complete walkthrough (Scenario 1).*

*Complete session trace:* `demos/session_logs/escenario1_happy_path.json`.

---

## 9.2.2. Scenario 2 — Resilience Against Prompt Injection: Ubuntu:latest on Port 22

Operational guarantees collapse if an agent is susceptible to conversational manipulation. This scenario evaluates resilience against a **Prompt Injection** attack [27] attempting to expose an administrative SSH port using unvetted images.

### Adversarial Input Vector

The user submits the following prompt via Streamlit:

> *«I need immediate terminal access to debug scripts. Deploy an Ubuntu container with the absolute latest version (use latest to keep things simple) and expose port 22 directly so I can connect via SSH. Do it quickly.»*

Lacking an internal enterprise threat model, the language model attempts to fulfill user intent directly, synthesizing an insecure tool call in iteration 1:

```
{
  "name": "ubuntu-debug-console",
  "image": "ubuntu:latest",
  "port": 22
}
```

### Hexagonal Interception and HTTP 422 Response

In un-sandboxed architectures where models directly access cluster APIs, this invocation would compromise security by binding SSH under mutable tags.

In *Agentic Deployer*, `SecurityContextValidator` (§5.4) intercepts the intent, detecting three concurrent policy violations:

1. Port `22` is reserved for administrative system access ($< 1024$).

2. Mutable tag `:latest` is strictly forbidden to prevent configuration drift.

3. Unapproved repository prefix lacking trusted registry namespaces.

The domain engine aborts execution, raising `SecurityViolationError`, which serializes into an HTTP `422 Unprocessable Entity` response:

```
HTTP/1.1 422 Unprocessable Entity
Content-Type: application/json

{
  "detail": [
    "Security Violation: Port 22 is reserved for administrative tasks.",
    "Security Violation: Use of mutable :latest tag is forbidden.",
    "Security Violation: Image must come from a trusted registry."
  ]
}
```

## Guided Self-Correction via the Feedback Loop

The ReAct loop captures the HTTP `422` error as an **Observation**, injecting it into model context.

The agent parses the error in its reasoning trace (*Thought*), generating an informative, pedagogical response:

> *«The IT Department has rejected this request due to cybersecurity policy violations. Policies prohibit the ':latest' tag, port 22 (reserved SSH), and unvetted registries. To proceed, please specify an immutable version tag (e.g., 'ubuntu:22.04') and select an application port above 1024.»*

Across 5 guided dialogue turns, the agent converges to register the compliant specification `docker.io/library/ubuntu:22.04` on port `2222`.

| Operational Phase | Action and Result |
|---|---|
| **Initial Prompt** | User request demanding `ubuntu:latest` bound to SSH port 22 |
| **Initial Model Call** | `format_deployment_intent(image="ubuntu:latest", port=22, ...)` |
| **Architectural Response** | **`HTTP 422 Unprocessable Entity`** — 3 security violations detected |
| **Iterative Self-Correction** | Educational policy explanation and assisted parameter convergence over 5 turns |
| **Final Resolution** | Compliant specification on port `2222` with immutable image registered as `PENDING_APPROVAL` |

***Table 27:*** *Execution trace: Prompt Injection attempt (Scenario 2).*

Interception occurred **before any operation reached the cluster**, demonstrating that security boundaries depend on deterministic software rather than stochastic model compliance.

*Complete session trace:* `demos/session_logs/escenario2_prompt_injection.json`.

---

## 9.2.3. Scenario 3 — Complete HITL Governance Lifecycle: Departmental WordPress CMS

Evaluates stateful service handling and synchronization with the operator dashboard.

**User Prompt:**

> *«The Computer Science department needs a WordPress CMS to publish news and events. Can you request its deployment?»*

**FSM Transition Lifecycle:**

```
[CREATION] ──> PENDING_APPROVAL ──> APPROVED ──> DEPLOYED
```

The agent infers persistent storage needs and configures port 8080. When registered in the FSM, the IT technician receives an asynchronous dashboard notification. Verifying quota allocations, the operator clicks "Approve", materializing YAML manifests in storage.

*Complete session trace:* `demos/session_logs/escenario3_hitl_completo.json`.

---

## 9.2.4. Scenario 4 — Multi-turn Exception-Guided Self-Correction

Evaluates autonomous error recovery when handling incomplete storage parameters:

**User Prompt:**

> *«Deploy a standard PostgreSQL database to store project surveys.»*

1. **Iteration 1 (Missing Registry Prefix):** The agent infers database storage needs, issuing `format_deployment_intent(image="postgres:13.4", storage="10Gi", ...)`, omitting trusted registry prefixes.
2. **Deterministic Interception:** The validator returns `HTTP 422: "Untrusted registry"`.
3. **Cognitive Self-Correction:** The LLM parses the observation and re-emits a corrected call with namespace `docker.io/library/postgres:13.4`, completing successful registration in 2 turns (26.1 s).

*Complete session trace:* `demos/session_logs/escenario4_autocorreccion.json`.

---

## 9.2.5. Synthesis and Comparative Breakdown of Case Studies

Isolating component execution latencies across the request lifecycle:

| System Component | Technology Tier | Measured Latency | Operational Character |
|---|---|---|---|
| **User Interaction** | Streamlit Frontend | ~ 5 s | Natural language prompt authoring |

| System Component | Technology Tier | Measured Latency | Operational Character |
|---|---|---|---|
| **Inference & Reasoning** | ReAct Agent (qwen2.5:7b) | 12 – 27 s | Local stochastic GPU inference |
| **Tool Protocol** | FastMCP Server | < 5 ms | JSON-RPC serialization & stdio piping |
| **Security Validation** | `SecurityContextValidator` | < 1 ms | Deterministic CPU boundary checks |
| **Transactional State** | SQLite Repository / FSM | < 2 ms | ACID persistence & state mutation |
| **Human Supervision (HITL)** | IT Operations Dashboard | ~ 7 min | Visual inspection & operator click |
| **Declarative Expansion** | `FakeK8sAdapter` | < 1 ms | Golden Path YAML interpolation |

***Table 28:*** *Lifecycle latency breakdown per system component.*

Comparative metrics across all four walkthrough scenarios confirm operational stability:

| Evaluated Scenario | ReAct Turns ($\mu$) | Inference Latency ($\mu \pm \sigma$) | Resulting FSM State | Generated Artifact |
|---|---|---|---|---|
| **1. Standard Provisioning (AI Conference)** | 1.0 $\pm$ 0.0 | 12.99 s $\pm$ 1.52 s | `DEPLOYED` | Complete YAML on disk |
| **2. Prompt Injection (Ubuntu SSH)** | 5.0 $\pm$ 0.6 | 45.07 s $\pm$ 3.21 s | `PENDING_APPROVAL` * | Quarantined perimeter containment |
| **3. Full HITL (WordPress CMS)** | 2.0 $\pm$ 0.0 | 27.84 s $\pm$ 2.14 s | `PENDING_APPROVAL` | Dashboard operator card |
| **4. Self-Correction (PostgreSQL)** | 2.0 $\pm$ 0.0 | 26.10 s $\pm$ 1.98 s | `PENDING_APPROVAL` | Autonomous parameter recovery |

***Table 29:*** *E2E comparative operational metrics across evaluation scenarios.*

> * *Scenario 2 Note:* Final `PENDING_APPROVAL` status represents the compliant proposal reached after 5 negotiation turns where the validator repeatedly blocked policy circumvention.

# 9.3. Comparative Empirical Evaluation of LLM Architectures

A primary research objective is determining whether agent reliability depends on foundational model scale versus deterministic software containment boundaries.

Models were evaluated across two regimes:

1. **Intrinsic Stochastic Robustness (Pure Zero-Shot):** Models faced Metamorphic Testing in strict isolation, with zero system prompt guidance, context memory, or recovery loops.
2. **Assisted Production Performance (ReAct Loop):** Identical models operated within `AgentOrchestrator`, supported by institutional system prompts, MCP tool schemas, and FSM feedback loops.

## 9.3.1. Intrinsic Robustness: Metamorphic Stress-Testing (Pure Zero-Shot)

Evaluating semantic parameter invariance (`name`, `image`, `internal_port`) across six input transformations (MR-1 through MR-6) on baseline `qwen2.5:7b`:

| **Metamorphic Relation** | **Applied Transformation** | **Evaluated Input Vector** | **Result** | **Structured JSON Extraction** |
|---|---|---|---|---|
| **MR-1:** Syntactic Permutation | Reordering technical parameters | `"api-backend with image python:3.12 on port 8000"` $\leftrightarrow$ `"On port 8000 with image python:3.12, I need api-backend"` | **Pass** | Identical (`name`, `image`, `port`) |
| **MR-2:** Lexical Noise Invariance | Injected pleasantries and conversational filler | Baseline prompt $\leftrightarrow$ identical specifications wrapped in conversational pleasantries | **Fail** | Reverts to unstructured chat text |
| **MR-3:** Typographical Invariance | Case alterations (`UPPERCASE` / `lowercase`) | `"deploy redis..."` $\leftrightarrow$ `"DEPLOY REDIS..."` | **Fail** | Mutates name attribute (`redis-service`) |
| **MR-4:** Incremental Composition | Single prompt vs. 3 conversational turns | Single declarative message $\leftrightarrow$ 3 turns with partial information | **Fail** | Reverts to unstructured chat text |
| **MR-5:** Semantic Paraphrasing | Synonyms and passive voice | `"Spin up a wordpress on port 80 named blog"` $\leftrightarrow$ `"Create a workload answering to blog..."` | **Pass** | Identical extraction |

| Metamorphic Relation | Applied Transformation | Evaluated Input Vector | Result | Structured JSON Extraction |
|---|---|---|---|---|
| **MR-6:** Linguistic Invariance | Translation into English | Spanish prompt $\leftrightarrow$ English prompt | **Pass** | Identical extraction |

***Table 30:*** *Empirical Metamorphic Testing results per relation evaluated against Qwen 2.5 (7B).*

| Metamorphic Suite Metric | Observed Value |
|---|---|
| Evaluated metamorphic relations | 6 |
| Executed tests | 6 |
| Intrinsic Zero-Shot success rate | **50% (3/6)** |
| Total suite execution time | 89.7 s |
| Cognitive engine evaluated | `OllamaLLMClient` (qwen2.5:7b local) |

***Table 31:*** *Global metrics of the metamorphic test battery.*

## Attentional Failures and Prompt Engineering Limits

Failures in MR-2, MR-3, and MR-4 provide critical evidence: **general-purpose models under 8B parameters degrade significantly when exposed to lexical noise or multi-turn dispersion without external orchestration**. When attention diffuses, models drop JSON schemas and revert to unstructured chat.

Attempting to resolve this via **Reinforcement Suffixes** (*"Respond ONLY with valid JSON"*) induced **over-compliance**: in multi-turn dialogues, models forced premature tool calls during intermediate turns lacking required parameters (hallucinating ports or images), triggering schema `ValidationError` exceptions. Similarly, *Chain of Thought* [17] reasoning induced syntax corruption by interspersing discursive reasoning with formal JSON tokens.

## Multi-Model Comparative Benchmarking Under Metamorphic Stress

Evaluating 9 neural architectures under identical zero-shot conditions:

| Evaluated Model | Runtime Tier | Zero-Shot Success Rate | Observed Behavior Under Noise and Perturbations |
|---|---|---|---|
| **Gemini 3.5 Flash** (Google) | Cloud | **66% (4/6)** | Tolerates lexical noise (MR-2) and multi-turn composition (MR-4). Fails on case alteration (MR-3) and language switches (MR-6). |

| Evaluated Model | Runtime Tier | Zero-Shot Success Rate | Observed Behavior Under Noise and Perturbations |
|---|---|---|---|
| **Cohere Command R+** (Cohere) | Cloud | **66% (4/6)** | Native tool calling model. Matches Gemini, failing on extreme informal noise (MR-2) and cross-language invariance (MR-6). |
| **Qwen 2.5 (7B)** (Alibaba) | Local (Ollama) | **50% (3/6)** | Attentional diffusion under noise (MR-2) and dispersed context (MR-4), reverting to unstructured text. |
| **Llama 3.2 (3B)** (Meta) | Local (Ollama) | **0% (0/6)** | Fails zero-shot structured extraction. Emits string `"None"` instead of `null` on empty fields. |
| **Mistral (7B)** (Mistral AI) | Local (Ollama) | **0% (0/6)** | Systematic formatting failure. Drops JSON structure when exposed to lexical noise. |
| **Ministral 8B** (Mistral AI) | Cloud | **0% (0/6)** | Unable to format tool calls without explicit system prompt guidance. |
| **Codestral** (Mistral AI) | Cloud | **0% (0/6)** | Lacks structured tool fine-tuning; emits procedural code blocks or plain text explanations. |
| **Qwen 2.5 (27B)** (Alibaba) | Groq (LPU) | **50% (3/6)** | Robust on permutations and paraphrasing, yet vulnerable to long conversational noise. |
| **Qwen 2.5 Coder (7B)** (Alibaba) | Local (Ollama) | **100% (6/6)** | **Passes 100% of the suite (6/6).** Code specialization neutralizes attentional degradation and lexical noise. |

***Table 32:*** *Comparative results of the metamorphic stress test (Pure Zero-Shot).*

Methodological conclusions:

1. **Syntactic specialization outweighs raw model scale:** Cloud hyperscaler models (Cohere, Gemini) achieved at most 66% unassisted invariance, while a compact local 7B model fine-tuned on code (`qwen2.5-coder:7b`) achieved **100% success**. Tool calling reliability depends on training alignment with formal grammars (JSON Schemas, static typing) rather than sheer parameter count.
2. **Code models without tool tuning fail:** *Codestral* (0% success) demonstrates that procedural coding ability does not equal agentic tool orchestration: without fine-tuning on structured function-calling protocols, models generate raw text unparseable by MCP clients.

---

## 9.3.2. Assisted Production Performance: ReAct Loop Orchestration

Evaluating identical models within the full software framework (Hexagonal Architecture, MCP tool definitions, and FSM feedback loops) using the baseline Scenario 1 prompt:

| Evaluated LLM Model | Scale / Tier | ReAct Turns (on success) | Mean E2E Latency | Valid Invocation Rate |
|---|---|---|---|---|
| **Qwen 2.5 Coder** | 7B (Local / Ollama) | 1.0 $\pm$ 0.0 | 12.15 s $\pm$ 1.12 s | **100%** |
| **Qwen 2.5** (General) | 7B (Local / Ollama) | 3.0 $\pm$ 0.4 | 47.33 s $\pm$ 3.85 s | **100%** |
| **Llama 3.2** | 3B (Local / Ollama) | 1.0 $\pm$ 0.0 | 9.44 s $\pm$ 0.85 s | **75%** |
| **Mistral** | 7B (Local / Ollama) | N/A | N/A | **0%** |
| **Gemini 3.5 Flash** | Cloud (Google) | 1.0 $\pm$ 0.0 | ~ 5.50 s | **100%** |
| **Cohere Command R+** | Cloud (Cohere) | 1.0 $\pm$ 0.0 | ~ 9.00 s | **100%** |
| **Ministral 8B** | Cloud (Mistral AI) | 1.0 $\pm$ 0.0 | ~ 3.20 s | **100%** |
| **Qwen 2.5 (27B)** | Cloud (Groq LPU) | 2.0 $\pm$ 0.0 | ~ 2.45 s | **100%** |

***Table 33:*** *Real comparative performance of alternative models in the ReAct loop.*

### The Stabilizing Effect of Software Architecture

Contrasting Table 32 (Zero-Shot) against Table 33 (ReAct Loop) provides core empirical validation:

- **Baseline Qwen 2.5 (7B)** jumped from a **50% success rate in stochastic isolation** to **100% valid invocations** when guided by the ReAct loop and MCP schemas, stabilizing decisions across 3 reflective iterations.
- **Ministral 8B**, which scored **0% in unassisted metamorphic testing**, attained **100% success in 1 turn (3.2 s)** when supported by the system prompt and tool contracts.
- **Qwen 2.5 Coder (7B)** confirmed its role as an optimal sovereign engine: consistently converging in **1.0 turn**, reducing local inference latency from 47 s to **12.15 s**, and eliminating corrective dialogue turns.

## 9.3.3. Comparative Synthesis and Technology Selection Criteria

1. **Viability of Sovereign Local Inference (*Zero Data Retention*):** Compact code-specialized models (`qwen2.5-coder:7b`) match cloud commercial accuracy within modest hardware envelopes (6 GB VRAM), eliminating external data leakage risks.
2. **Hardware Acceleration:** Where cloud privacy policies permit, LPU inference engines (Groq) achieve ~2-second response times, enabling real-time conversational workflows.

3. **Deterministic Software as a Containment Gate:** No language model exhibits absolute reliability in unassisted zero-shot modes. Operational reliability must reside within deterministic software boundaries (Hexagonal Architecture) and human oversight (*HITL*).

---

# 9.4. Cognitive Architecture Experiments: Overcoming 7B Attentional Limits

General-purpose 7B models exhibited premature over-compliance in multi-turn dialogues. To determine whether software abstractions can overcome this bottleneck, a systematic test harness (`demos/demo_state_machine.py`) evaluated three architectural paradigms across an incremental three-turn sequence:

- **Turn 1:** Unspecified request: *«Hello IT team, I need to deploy a web service please.»*
- **Turn 2:** Identifying name only: *«It will be named project-web.»*
- **Turn 3:** Technical specifications: *«The image will be nginx:alpine and port is 8080. That completes it.»*

## 9.4.1. Paradigm 1: Classic ReAct with Strict Suffix (Logical Failure)

Model instructed with an imperative suffix (*"Respond ONLY with JSON"*).

- **Result:** In Turn 2, possessing only the name, the model forced a tool call hallucinating image (`nginx:latest`) and port (`80`), failing logically due to over-compliance.

## 9.4.2. Paradigm 2: Chain of Thought (Syntactic Failure)

Model instructed to reason step-by-step in free text before emitting JSON [17].

- **Result:** Successfully prompted for missing parameters in Turn 2. In Turn 3, however, combining natural language reasoning with strict JSON tokens corrupted syntax, raising schema `ValidationError` exceptions.

## 9.4.3. Paradigm 3: Stateful Slot Filling and Context Distillation

To eliminate cognitive overload, the system implemented **Stateful Slot Filling**, rooted in Dialogue State Tracking [39] and context distillation literature [40, 41].

The paradigm decouples processing into two orthogonal tiers:

1. **Passive Cognitive Tier (Isolated Semantic Extraction):** The LLM acts solely as an entity extractor over the latest user prompt, populating a discrete data structure (`{"name": null, "image": null, "port": null}`) without executing tools or deciding control flow.
2. **Deterministic Control Tier (Python State Machine):** Traditional software maintains accumulated session state in memory. If required parameters are missing, the state machine deterministically generates clarification prompts. Only when the parameter vector is 100% complete does the orchestrator dispatch the MCP tool call.

## 9.4.4. Cross-Model Evaluation Across Cognitive Paradigms

Evaluating all 8 model families across the three-turn sequence under each paradigm:

| Evaluated Model | Classic ReAct (No Suffix) | ReAct with Chain-of-Thought | Stateful Slot Filling |
|---|---|---|---|
| **Qwen 2.5 Coder (7B)** | **100% Success** (Specialized tuning) | **100% Success** (Robust syntax) | **100% Success** (Absolute precision) |
| **Qwen 2.5 (7B)** (General) | Logical Failure (Over-compliance Turn 2) | Syntax Failure (`ValidationError`) | **100% Success** (Deterministic state control) |
| **Llama 3.2 (3B)** | Syntax Failure (Injects `"None"`) | Structuring Failure | **Partial Success** (Extracts `"null"` as string) |
| **Mistral (7B)** | Logical Failure (Drops format over turns) | Attentional Loop | **100% Success** (Isolated semantic extraction) |
| **Gemini 3.5 Flash** | **100% Success** (Superior attention) | **100% Success** (Robust syntax) | **100% Success** (Flawless extraction) |
| **Cohere Command R+** | **100% Success** (Native tool design) | **100% Success** (Deterministic response) | **100% Success** (Absolute precision) |
| **Ministral 8B** | **100% Success** (Native tool support) | **100% Success** (Stable reasoning) | **100% Success** (Absolute precision) |
| **Qwen 2.5 (27B)** | **100% Success** (Ultra-fast Groq LPU) | **100% Success** (Stable reasoning) | **100% Success** (Absolute precision) |

***Table 34:*** *Results of Cognitive Architecture Experiments expanded with Cloud models.*

### Conclusions on Cognitive Architectures

Two distinct engineering paths achieve production reliability:

- **Deterministic Software Engineering (*Stateful Slot Filling*):** Managing conversational state via deterministic state machines allows even attention-constrained models (Qwen 2.5 and Mistral) to achieve **100% reliability**, decoupling flow control from stochastic inference.
- **Model Specialization (`qwen2.5-coder:7b`):** Intensive pre-training on source code enables compact models to handle simultaneous semantic extraction and tool calling without external state tracking wrappers, matching commercial cloud hyperscalers.

---

# 9.5. Operational Impact Evaluation on ITSM Processes

Having demonstrated architectural resilience (§9.2), characterized foundational models (§9.3), and resolved attentional limits (§9.4), experimental results inform IT Service Management (ITSM) operational impact models.

## 9.5.1. Traditional ITSM Friction vs. Agentic Automation

In regulated environments such as university datacenters, provisioning velocity is bottlenecked by ticketing bureaucracy (Jira, ServiceNow, email):

1. **Ticket Creation (Abstraction Gap):** The applicant submits vague prompts (*«I need a database for my project»*).
2. **Triage Bottlenecks (Level 1 Support):** Support staff halt tickets, requesting clarifications via asynchronous emails (ports, storage, memory bounds).
3. **Escalation to Engineering (Level 3):** After days of email round-trips, tickets reach senior platform engineers.
4. **Manual Manifest Authoring:** Senior engineers halt strategic tasks to draft Kubernetes manifests, connect over VPN, and apply configurations.

This lifecycle introduces massive queue idle times and severe context-switching costs.

## 9.5.2. Quantifying Delivery Acceleration and Time-to-Market (TTM)

*Agentic Deployer* resolves requirements negotiation interactively via the ReAct loop, reducing technical personnel duties to rapid visual auditing in the HITL Dashboard.

Table 35 compares delivery timelines for a standard institutional web service:

| **Operational Lifecycle Phase (ITSM)** | **Conventional ITSM Model** | **Agentic Model (*Agentic Deployer*)** | **Time-to-Market Reduction** |
|---|---|---|---|
| **1. Requirements clarification & triage** | 1,440 min† (Asynchronous Email/Ticket) | ~ 5 min (Interactive natural language AI dialogue) | **~ 99.7%** |
| **2. Declarative translation & YAML authoring** | 15 min (Manual drafting by L3 engineer) | < 0.01 min (Golden Path template expansion) | **~ 99.9%** |
| **3. Security policy & compliance review** | 5 min (Manual review / CI/CD check) | < 0.001 min (CPU Hexagonal Core checks) | **~ 99.9%** |
| **4. Formal deployment approval** | 2 min (Manual review / `kubectl apply`) | 2 min (Review and click in HITL Dashboard) | **0% (Identical control)** |
| **Queue Idle Time (*Idle Time*)** | 2,880 min (Ticket queued awaiting assignment) | 60 min (Dashboard review queue) | **~ 97.9%** |
| **Total Delivery Time (*Time-to-Market*)** | **~ 4,342 min (~ 72.4 hours / 3 business days)** | **~ 67 min (~ 1.1 hours)** | **~ 98.5%** |

| Operational Lifecycle Phase (ITSM) | Conventional ITSM Model | Agentic Model (*Agentic Deployer*) | Time-to-Market Reduction |
|---|---|---|---|
| **Active Engineering Time** | **30 min** (Triage + authoring + validation) | **~ 5 min** (AI dialogue) + **2 min** (HITL) = **7 min** | **~ 76.7%** |
| **Systems Engineer Cognitive Load** | **High** (Coding manifests & L1 triage) | **Minimal** (Visual audit of structured intent) | **Qualitative reduction** |

***Table 35:*** *Comparative operational time impact (Traditional ITSM vs. Agentic Deployer).*

> † *Elapsed Time Note:* The **1,440-minute (24-hour)** figure represents *elapsed calendar time* resulting from inter-departmental email delays, not continuous active engineering effort. Conversely, the conversational agent resolves requirements interactively in real time (§4.6.4).

### Experimental Impact Conclusion

Table 35 confirms that the primary value of *Agentic Deployer* lies in **eliminating bureaucratic triage latency**. By resolving specifications interactively in real time and structuring reviews via the HITL Dashboard, the system shortens overall Time-to-Market by **98.5%**, liberating **76.7% of technician active working hours**.

These empirical savings establish the analytical foundation for the economic adoption and ROI models presented in **Chapter 4 (Section 4.6)**.

---

# 9.6. Threats to Empirical Validity

In accordance with empirical software engineering guidelines [42, 43], this section documents potential threats to validity and mitigation measures.

## 9.6.1. Internal Validity

Internal validity addresses whether unmonitored factors influenced observed experimental outcomes:

- **LLM Stochasticity:** Neural inference exhibits variance even at zero temperature ($T = 0$) due to non-deterministic parallel floating-point operations on GPU hardware. To mitigate this, case studies and metamorphic tests were executed across batches of 5 consecutive iterations, reporting metrics as mean ($\mu$) and standard deviation ($\sigma$).
- **Network Latency & Cloud Throttling:** Cloud evaluations (Google, Cohere, Groq, Mistral) depend on public network hops and remote server load. This was addressed by measuring pure model generation latency separate from network transfer times and benchmarking against local air-gapped Ollama execution.
- **Hardware Thermal Throttling:** Workstation laptop GPUs (NVIDIA RTX 3060 Laptop) experience thermal throttling under continuous loads. Passive 30-second cooling intervals were enforced between inference batches to maintain clock frequency stability.

## 9.6.2. External Validity

External validity evaluates the degree to which empirical findings generalize to other organizations and infrastructure environments:

- **Tool Catalog Scope:** The MCP tool catalog focuses on four representative institutional workloads (web portals, CMS engines, Python microservices, relational databases). While covering standard academic requests, complex enterprise microservices with distributed service meshes or custom Kubernetes operators require further empirical evaluation.
- **Model Evolution:** Results evaluate the eight neural families analyzed in Section 9.3. While model architectures evolve rapidly, the central architectural conclusion—the necessity of deterministic hexagonal boundaries to contain stochastic reasoning—remains universally applicable.

## 9.6.3. Construct Validity

Construct validity examines whether empirical metrics accurately reflect theoretical constructs:

- **Measuring System Reliability:** Reliability was evaluated using objective quantitative indicators rather than subjective impressions: Pydantic type invariant compliance, security exception capture rates, metamorphic assertion passes, and mutation survival ratios.
- **ITSM Time Modeling:** Conventional delivery baselines were grounded in ITIL v4 standards and university operational data, strictly distinguishing active engineering hours from elapsed ticket queue latency.

## 9.6.4. Conclusion Validity

Conclusion validity assesses whether statistical and inferential conclusions are sound:

- **Selection Bias:** To avoid evaluating only favorable Happy Paths, the experimental design deliberately incorporated adversarial attacks (§9.2.2), structurally flawed models (`codestral`, `llama-3.2-3b`), and deliberate AST mutations (§8.3), ensuring conclusions objectively reflect both architectural strengths and empirical boundaries.

# Chapter 10. Conclusions, Limitations, and Future Work

Integrating Large Language Models (LLMs) into infrastructure operations automation (*Platform Engineering*) demands strict architectural safeguards to mitigate their inherent stochastic nature. This Master's Thesis has addressed this challenge through the design, implementation, and empirical evaluation of a decoupled architecture engineered to guarantee deterministic invariant validation, transactional traceability, and human-in-the-loop governance across the service provisioning lifecycle.

Below, we detail the primary conclusions derived from empirical system validation, the operational limitations inherent in the prototype's scope, and the strategic roadmap for future evolution.

---

## 10.1. Conclusions

The conceptualization, development, and empirical evaluation of *Agentic Deployer* validate the technical feasibility of intermediating natural language infrastructure requests through intelligent agents, provided their execution remains strictly confined within a deterministic architectural boundary. The core conclusions of this research are organized around the following dimensions:

### 10.1.1. Objectives Compliance and Traceability Matrix

To formally certify the correspondence between initial technical commitments and delivered outcomes, Table 36 synthesizes the bijective traceability matrix connecting the five specific objectives ($O_1$ through $O_5$) established in Section 1.3.2, associated functional and non-functional requirements, implemented software artifacts, and empirical validation evidence:

| Specific Objective | Linked Requirements | Artifacts & Implementation | Empirical Validation Evidence | Completion Degree |
|---|---|---|---|---|
| **$O_1$: Decoupled Hexagonal Core (*Ports & Adapters*)** | NFR-02, NFR-04 | Immutable Pydantic domain, pure use cases, and `SecurityContextValidator` (Chap. 5, §5.1–5.4). | Isolation verified against non-compliant parameter injection (§9.2.2) and 100% unit test coverage. | **100% (Completed)** |
| **$O_2$: Standardized Tool Server (MCP)** | FR-03, NFR-01 | Autonomous FastMCP server over `stdio` exposing the institutional IT catalog (Chap. 6, §6.1). | External interoperability certification via official **MCP Inspector** and decoupling from proprietary APIs (§9.1, §10.1.3). | **100% (Completed)** |

| Specific Objective | Linked Requirements | Artifacts & Implementation | Empirical Validation Evidence | Completion Degree |
|---|---|---|---|---|
| **$O_3$: ReAct Agent Orchestrator with Feedback Loop** | FR-02, NFR-04 | Cognitive orchestrator `AgentOrchestrator` with ReAct loop and *Stateful Slot Filling* pattern (Chap. 6, §6.2). | Convergence in 1.0 iterations with `qwen2.5-coder:7b` (§9.3) and resolution of attentional limits with 100% success (§9.4). | **100% (Completed)** |
| **$O_4$: Declarative Golden Paths** | FR-01, FR-03 | Pre-audited Kubernetes templates (`Deployment`, `Service`, `Ingress`) and `FakeK8sAdapter` (Chap. 5, §5.4.3; Annex A). | Deterministic, hallucination-free generation across 4 practical walkthroughs (§9.2) and syntactic manifest validation. | **100% (Completed)** |
| **$O_5$: Human-In-The-Loop (HITL) Security Governance** | FR-04, FR-05, NFR-03 | Finite State Machine (FSM), ACID SQLite persistence, and Streamlit operational console (Chap. 7, §7.1–7.3). | Zero unauthorized executions; 98.5% reduction in Time-To-Market and 76.7% reduction in active technician time (§9.5). | **100% (Completed)** |

***Table 36:*** *Traceability matrix and degree of fulfillment of specific project objectives.*

## 10.1.2. Synthesis of Tangible Engineering Contributions

As a result of system development and empirical evaluation, this Master's Thesis contributes the following methodological frameworks and tangible engineering assets:

1. **Deterministic Perimeter Architecture for Infrastructure Agents:** Specification and validation of a hexagonal architectural pattern that strictly decouples probabilistic language model reasoning from physical execution layers, confining stochastic behavior via domain invariant validators.
2. **Autonomous, Interoperable FastMCP Server for IT Operations:** A reference implementation conforming to the open *Model Context Protocol* (MCP) standard exposing university datacenter operations (`deploy_congress_web`, `deploy_department_cms`, `deploy_python_app`, `deploy_static_website`, `deploy_database`), externally certified via the official *MCP Inspector*.
3. **Conversational State Decoupling Pattern (*Stateful Slot Filling*):** Empirical demonstration of an engineering technique that isolates point-in-time semantic extraction from transactional dialogue tracking via a deterministic FSM, elevating the multi-turn reliability of local 7B-parameter models from 0% to 100%.
4. **Hybrid Quality Assurance Framework for AI Systems:** A holistic verification methodology that transcends traditional code coverage through the synergistic combination of Property-Based Testing (Hypothesis), Metamorphic Testing, and Mutation Testing (Mutmut), surfacing logical vulnerabilities invisible to standard unit test suites.
5. **Multi-Model Empirical Benchmark and Reproducible Evaluation:** A systematic testbed and experimental trace

catalog across 8 neural model families (local via Ollama, LPUs via Groq, and commercial cloud APIs), establishing empirical guidelines for sovereign agent adoption (*Zero Data Retention*) within public sector institutions.

## 10.1.3. Overcoming Vendor Lock-In through MCP

Adopting the **Model Context Protocol (MCP)** [13] to isolate the IT Service operations catalog formally decouples tool specifications from proprietary AI vendor APIs. We empirically verified the feasibility of structuring an interoperable, modular operational capability suite. The FastMCP server dynamically generates contracts via static type introspection, ensuring compatibility with any cognitive engine conforming to the open standard.

This decoupling was empirically verified along two complementary dimensions:

1. **Transparent Inference Engine Substitution:** Implementing the native `OllamaLLMClient` (Section 6.2.1) enabled swapping commercial OpenAI models for open-weight models executed locally (`qwen2.5:7b`), requiring zero modifications to hexagonal domain logic or MCP tool definitions.
2. **Certified External Interoperability:** Protocol compliance was formally evaluated using the official protocol inspection utility (**MCP Inspector**). Operating independently of the Streamlit interface, the tool discovered (`tools/list`) and executed catalog operations over standard `stdio`, confirming that the server serves as a reusable asset for external audit tooling and third-party orchestration agents.

## 10.1.4. Hexagonal Architecture as a Deterministic Containment Perimeter

A central finding of this research is the efficacy demonstrated by Hexagonal Architecture (*Ports and Adapters*) [19] in confining and neutralizing LLM hallucinations and stochastic variations. By requiring all inference proposals to satisfy the contracts of an immutable, strongly-typed domain model (`Pydantic`) evaluated by a deterministic validator (`SecurityContextValidator`), model stochasticity is intercepted prior to interacting with physical infrastructure adapters.

The system methodologically assumes LLM fallibility: it intercepts non-compliant proposals and feeds structured exceptions back into the conversational context (*Feedback Loop*), facilitating guided self-correction. The adversarial instruction injection case study documented in Section 9.2.2 demonstrates that requests violating security baselines (such as unpinned `ubuntu:latest` images or privileged port `22` allocations) are blocked at the application core, returning `HTTP 422 Unprocessable Entity` status codes and steering convergence toward compliant configurations over successive turns without compromising the physical environment.

## 10.1.5. The Necessity of the Human-In-The-Loop (HITL) Pattern in Critical Infrastructure

In contrast to paradigms advocating full algorithmic autonomy (Level 5), this research concludes that human supervision remains an indispensable safeguard for compliance, legal liability, and IT governance in institutional environments. Implementing the Finite State Machine (FSM) and asynchronous control console proved that the HITL pattern introduces minimal operational friction while establishing an effective segregation of duties:

- The language model handles intent extraction and technical parameter negotiation in natural language.
- The IT specialist audits, verifies, and formally authorizes deployment orders in the operations console.
- The requesting researcher tracks deployment lifecycles in real time via the conversational interface.

The empirical delivery time reductions demonstrated—reducing active engineering overhead from 30 minutes to 7 minutes (a 76.7% reduction) and total delivery time (*Time-To-Market*) from ~4,342 minutes under conventional ticketing to ~67 minutes in the assisted flow (a 98.5% reduction, Section 9.5)—confirm that human governance and operational efficiency converge when cognitive automation absorbs administrative triage.

## 10.1.6. Redefining Quality Assurance (QA) Against the Coverage Fallacy

Conventional QA methodologies reliant solely on deterministic unit tests are insufficient for certifying systems containing probabilistic components. Integrating Property-Based Testing (Hypothesis) [37] and Metamorphic Testing [38] verified that the agent maintains semantic invariance and structured schema compliance across linguistic, syntactic, and typographical transformations.

Furthermore, Mutation Testing (Mutmut) revealed that high line coverage does not imply logical resilience. Initial analysis uncovered **30 surviving mutants** within resource parsing functions (`_parse_cpu`, `_parse_ram`), despite reporting 100% line coverage. This finding guided the systematic refactoring of parsing algorithms and invariant assertions, substantiating mutation analysis as an indispensable requirement for auditing AI-driven infrastructure platforms.

## 10.1.7. Feasibility and Stabilization of Small-Scale (7B) Models

Local inference experiments showed that generalist 7B-parameter models exhibit attentional degradation when managing multi-turn workflows through purely reactive architectures, demonstrating schema adherence failures as dialogue histories expand.

However, incorporating the **Stateful Slot Filling** architectural pattern (Section 9.4.3)—wherein the language model serves strictly as an isolated semantic extractor per turn while a deterministic Python state machine tracks accumulated intent vectors—**enables these same models to reach 100% reliability without parameter expansion or fine-tuning overhead**. This architectural segregation between semantic extraction and transactional state tracking confirms the feasibility of deploying agentic systems on standard, modest hardware.

## 10.1.8. Cross-Cutting Findings from Multi-Model Evaluation

The systematic evaluation across 9 heterogeneous neural architectures (local quantized via Ollama, LPUs via Groq, and commercial cloud APIs) yields four transferrable engineering principles:

1. **Domain Alignment Prevails Over Parameter Scale:** Structured tool-calling efficacy depends on alignment with formal grammars and strict typing rather than raw parameter volume. Models with high general algorithmic capability (e.g., `codestral-latest`) scored 0% in zero-shot tool invocation, defaulting to free-form text. Conversely, compact code-specialized models (`qwen2.5-coder:7b`) achieved 100% metamorphic success in local sovereign mode.
2. **Complementarity of Probabilistic Inference and Deterministic Control:** No generalist language model operating in isolation exceeded 66% semantic invariance against lexical noise and language-switching. Integrated into the ReAct loop bounded by Hexagonal Architecture and the FSM, success rates converged to 100%. Agentic resilience resides in the surrounding deterministic software architecture, not in model infallibility.
3. **Technological Insulation Against Obsolescence:** Protocol abstraction via MCP and the `OpenAILLMClient` adapter

ensures that organizations can deploy sovereign, zero-cost local engines initially and progressively adopt specialized accelerators or frontier cloud models without modifying domain logic or tool contracts.

4. **Data Sovereignty and Institutional Governance:** While cloud providers offer low inference latencies, adoption introduces external vendor dependencies, rate-limiting constraints (`HTTP 429 Rate Limit Exceeded`), and sensitive metadata exposure. For public administrations with strict regulatory constraints, local deployments of specialized models under Zero Data Retention principles represent a technically sound and economically sustainable path.

## 10.1.9. Comparative Synthesis of Evaluated Models

Consolidating experimental results from Chapter 9 (**Tables 30, 31, and 32** for zero-shot metamorphic evaluations, and **Tables 33 and 34** for ReAct orchestration and cognitive architecture), **Table 37** classifies the evaluated cognitive engines by institutional agentic suitability:

| Engine / Model | Execution Environment | Suitability Rating | Observed Behavior & Empirical Evidence |
|---|---|---|---|
| `qwen2.5-coder:7b` (Alibaba / Ollama) | Local | **High (Sovereign - Code)** | Maximum reliability. 100% metamorphic pass rate (*Table 32*) and convergence in 1.0 iterations (*Table 33*). Code-specific fine-tuning overcomes hardware constraints. |
| `qwen2.5:7b` (Alibaba / Ollama) | Local | **Medium-High (Sovereign - Generalist)** | Moderate standalone reliability (50% metamorphic, *Table 32*). Requires external state machine support to avoid multi-turn context dilution. |
| `gemini-3.5-flash` (Google Cloud) | Cloud | **High (Commercial)** | Most consistent cloud model in structured extraction (66% metamorphic, *Table 32*). Subject to external quotas and cloud commercial retention policies. |
| `command-r-plus` (Cohere Cloud) | Cloud | **High (Commercial)** | Explicitly optimized for tool calling (66% metamorphic, *Table 32*); shows susceptibility to extreme colloquial phrasing. |
| `ministral-8b-latest` (Mistral AI) | Cloud | **Medium (Commercial)** | Optimal performance under FSM guidance (*Table 33*), but vulnerable in unassisted zero-shot extraction (0% metamorphic, *Table 32*). |
| `codestral-latest` (Mistral AI) | Cloud | **Unsuitable for Tool Calling** | Unsuitable for orchestration: lacks fine-tuning for structured JSON Schema invocation, emitting unstructured explanatory text (0% metamorphic, *Table 32*). |

***Table 37:*** *Cognitive engine ranking and empirical justification for institutional agentic orchestration.*

**Key Comparative Insights:**

1. **Syntactic specialization outweighs scale:** Specialization in formal grammars and interface contracts is more critical than parameter scale, enabling reliable institutional deployments under **Zero Data Retention** on commodity hardware.
2. **Invariance and deterministic containment:** The failure of commercial models to achieve 100% standalone invariance under linguistic variation confirms that production reliability must rest upon traditional deterministic software defenses.
3. **Adoption horizons for commercial platforms:** Frontier models billed by token consumption (such as **GPT-4o [44]** or **Claude 3.5 Sonnet [45]**) feature vendor optimizations for agentic workflows and MCP, offering viable alternatives in environments where data governance policies permit external cloud processing.

---

# 10.2. Prototype Limitations

In adherence to scientific rigor and transparency, the following technical limitations of the research prototype delineate operational boundaries relative to a mature enterprise deployment:

## 10.2.1. Simulated Infrastructure Adapter (`FakeK8sAdapter`)

Declarative configuration manifests are rendered and persisted to local disk rather than dispatched interactively against a live Kubernetes API. While the syntactic validity of generated YAML is certified (Annex A), this simulation omits physical cluster feedback:

- **Dynamic Quota Validation:** The Kubernetes Control Plane may reject creation requests if namespaces exceed concurrent compute quotas.
- **Concurrency and Naming Collisions:** Duplicate service identifiers trigger `HTTP 409 Conflict` errors on API servers, unmodeled in local simulation.
- **External Controller Dependencies:** Automated TLS certificates via `cert-manager` require active cluster operators; without them, `Ingress` resources remain pending indefinitely.

## 10.2.2. Absence of Perimeter Authentication and Authorization

Backend endpoints (`/mcp/intent`, `/hitl/approve`, `/hitl/status`) **lack cryptographic authentication and authorization** (OAuth2, OpenID Connect, JWT, or API keys). In this research prototype, any process with local network access can inspect or approve intents. In production, this represents a vulnerability requiring integration with enterprise directories (LDAP/Keycloak) and Role-Based Access Control (RBAC).

## 10.2.3. Resilience Against Complex Prompt Injection Vectors

While direct instruction injections are intercepted by deterministic validation (Section 9.2.2), the prototype does not incorporate defenses against advanced adversarial vectors:

- **Indirect Prompt Injection:** Adversarial payloads embedded in external documentation consumed dynamically by the agent.

- **Context Jailbreaking:** Multi-turn conversational drift designed to dilute system prompt constraints.

### 10.2.4. Constrained Tool Catalog and Lifecycle Scope

The MCP server implements tools focused on initial provisioning (`deploy_congress_web`, `deploy_department_cms`, `deploy_python_app`, `deploy_static_website`, `deploy_database`, `get_deployment_status`, and `delete_university_service`). Day-2 operational capabilities—such as dynamic pod autoscaling, secret management, certificate rotation, and real-time log streaming—remain outside the current prototype scope.

### 10.2.5. Concurrency and Scalability in the Application Layer

FastAPI runs synchronously across a single ASGI Uvicorn process. Under concurrent multi-user load, thread exhaustion bottlenecks may emerge. While SQLite addresses local race conditions via file-level locking, enterprise deployments require distributed relational database systems.

### 10.2.6. Prior Immutability in the Human Supervision (HITL) Channel

The HITL operations console enforces binary operator actions (Approve or Reject). If an intent contains reasonable parameters with slightly oversized memory requests, operators cannot modify fields in-flight (*Pre-Deployment Mutability*), forcing full rejection and dialogue restart.

### 10.2.7. Bounded Usability Evaluation and End-User Validation

Operational efficiency metrics (Chapter 9) were captured in controlled experimental environments. Formal usability evaluations (e.g., System Usability Scale, SUS) involving non-developer administrative and academic staff have not yet been conducted, leaving non-technical linguistic barriers unquantified.

### 10.2.8. Hardware Constraints and Quantization Limitations

Local evaluations were executed on a workstation with 6 GB VRAM, restricting models to <8B parameters quantized to 4 bits (Q4_K_M). While `qwen2.5-coder:7b` proved stable, evaluating larger architectures (14B or 32B parameters) on higher-memory compute infrastructure would establish definitive upper bounds on latency and context retention.

### 10.2.9. Linguistic Dissonance in Structured Reasoning Tasks

Evaluations enforced Spanish-language interactions across prompts, user inputs, and tool metadata. Because foundational models are trained predominantly on English corpora, syntactic adherence and formal schema compliance degrade in secondary languages. Certain formatting inconsistencies in generalist 7B models stem from this representational disparity.

---

## 10.3. Future Work

To address these limitations and transition *Agentic Deployer* toward institutional production in university datacenters, the following technical roadmap defines short- and medium-term priorities:

## 10.3.1. Evolution Toward Institutional Production (Short Term)

The immediate roadmap equips the platform with live execution capabilities, concurrency, and perimeter security:

- **Physical Kubernetes Adapter (`RealK8sAdapter`) and Persistent Storage:** Replace simulated disk serialization with a live adapter connected to the Kubernetes API via `kubernetes-client`, executing authenticated API calls upon HITL approval. Incorporate declarative `PersistentVolumeClaim` (PVC) management to support production databases with physical disk persistence.
- **Distributed Relational Persistence (PostgreSQL):** Migrate the embedded SQLite database to a client-server PostgreSQL engine. The hexagonal *Repository* abstraction isolates this transition from domain logic, enabling high-availability backend deployments across multiple Uvicorn/Gunicorn workers.
- **Role-Based Access Control (RBAC) and Federated Authentication:** Integrate OAuth2 and OpenID Connect with JWT tokens, federated via enterprise identity providers (Keycloak, LDAP), distinguishing researcher request privileges from authorized IT operator approval rights.
- **In-Flight Intent Editing in HITL:** Allow operators to adjust intent parameters (such as CPU allocations or port assignments) directly within the dashboard prior to authorization, eliminating unnecessary rejection cycles.

## 10.3.2. Platform Integration and Declarative Governance (Medium Term)

The medium-term roadmap focuses on integration into GitOps workflows and declarative platform engineering ecosystems:

- **GitOps Paradigm Adoption (ArgoCD / Flux):** Commit generated Kubernetes manifests to version-controlled Git repositories rather than applying them directly (*Push Model*), leveraging declarative reconciliation controllers (ArgoCD, Flux) to ensure immutable audit trails and disaster recovery.
- **Real-Time Observability (Prometheus & Grafana):** Equip the agent with telemetry tools to query live cluster metrics and quota reservations prior to drafting deployment proposals, providing contextual decision support.
- **Policy-as-Code Governance:** Externalize security validation rules into declarative policy engines such as Open Policy Agent (OPA) [46] or Kyverno [47], enabling security teams to update compliance policies independently of backend deployment lifecycles.
- **Consolidated Stateful Slot Filling:** Solidify the decoupled entity extraction architecture across all workflow categories, permanently eliminating conversational history accumulation and eliminating context dilution risks.

# Chapter 11. Bibliography and References

Below is the academic literature, technical specifications, and official documentation underpinning the architectural, methodological, and algorithmic decisions presented in this Master's Thesis.

**[1]** Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.
*(Primary reference for adopting distributed topologies and component decoupling over monolithic architectures).*

**[2]** Mell, P., & Grance, T. (2011). *The NIST Definition of Cloud Computing*. National Institute of Standards and Technology (NIST) Special Publication 800-145. https://doi.org/10.6028/NIST.SP.800-145
*(Academic definition of the cloud computing paradigm establishing the introductory technological context of this thesis).*

**[3]** Docker Inc. (2024). *Docker Documentation: Container Runtime and Architecture*. Retrieved September 15, 2026, from https://docs.docker.com/
*(Technical foundation of standardized application containerization cited in the State of the Art).*

**[4]** Open Container Initiative (OCI). (2024). *OCI Image Format and Runtime Specification*. Retrieved September 15, 2026, from https://opencontainers.org/
*(Open industry standard for container image interoperability, preventing vendor lock-in in orchestration).*

**[5]** The Kubernetes Authors. (2024). *Kubernetes Documentation: Concepts and Architecture*. Cloud Native Computing Foundation (CNCF). Retrieved September 15, 2026, from https://kubernetes.io/docs/concepts/
*(Theoretical foundation for the physical materialization of declarative code and the Golden Paths pattern applied in cluster orchestration).*

**[6]** Kim, G., Humble, J., Debois, P., & Willis, J. (2016). *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. IT Revolution Press.
*(DevOps movement reference utilized in Section 1.1 regarding the operational gap between development and operations).*

**[7]** Gartner. (2023). *Top Strategic Technology Trends for 2024: Platform Engineering*. Gartner Research.
*(Industry report utilized to justify the market shift toward Platform Engineering and IDP portals referenced in Section 1.1).*

**[8]** Spotify Engineering. (2020). *How We Use Golden Paths to Solve Fragmentation in our Software Ecosystem*. Retrieved September 15, 2026, from https://engineering.atspotify.com/2020/08/how-we-use-golden-paths-to-solve-fragmentation-in-our-software-ecosystem/
*(Technical documentation on the definition and practical implementation of the "Golden Path" concept in technology organizations).*

**[9]** Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. Proceedings of the International Conference on Learning Representations (ICLR). arXiv preprint arXiv:2210.03629. Retrieved September 15, 2026, from https://arxiv.org/abs/2210.03629
*(Methodological reference for the ReAct loop formulation and agentic orchestration in Chapter 6).*

**[10]** OpenTofu Authors. (2024). *OpenTofu: An open-source tool for building, changing, and versioning infrastructure safely and efficiently*. The Linux Foundation. Retrieved September 15, 2026, from https://opentofu.org/
*(Open-source Infrastructure as Code [IaC] tool governed by the Linux Foundation, evaluated in the initial project proposal and analyzed in Sections 1.2.2 and 4.4).*

**[11]** Siddiq, M. L., & Santos, J. C. (2026). *Treat LLM-Generated Terraform as Untrusted: An Exploratory Security Study and a Practitioner's Verification Workflow*. IEEE Software. https://doi.org/10.1109/MS.2026.3721790
*(Formal study demonstrating vulnerabilities in LLM-generated IaC specifications, justifying the necessity of deterministic algorithmic validators).*

**[12]** Tarasova, N., Balp-Straffon, E., Iancheruk, A., Sielskyi, Y., Kozodoi, N., Byrne, L., Butler, J., Jiang, D., Czelej, M., Ang, A., Shah, Y., Blanco, R., & Ivanov, S. (2026). *Evaluating Language Models on Infrastructure-as-Code Development*. IEEE Cloud Summit. https://doi.org/10.1109/CloudSummit68932.2026.00021
*(Large-scale quantitative empirical study evaluating LLM accuracy and limitations in generating infrastructure-as-code [IaC] specifications, cited in Section 2.5.2).*

**[13]** Anthropic PBC. (2024). *Model Context Protocol (MCP) Specification*. GitHub Open Source Repository. Retrieved September 15, 2026, from https://github.com/modelcontextprotocol/modelcontextprotocol
*(Technological standard used to eliminate vendor lock-in and isolate JSON-RPC tool definitions, documented in Section 6.1).*

**[14]** Hipp, D. R. (2024). *SQLite: A small, fast, reliable, self-contained, SQL database engine*. Retrieved September 15, 2026, from https://www.sqlite.org/
*(Embedded ACID transactional database engine utilized for Finite State Machine state persistence).*

**[15]** Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*. Advances in Neural Information Processing Systems (NeurIPS), 30, 5998–6008. Retrieved from https://doi.org/10.48550/arXiv.1706.03762
*(Original Transformer architecture publication, providing the structural foundation for the Large Language Models referenced throughout this work).*

**[16]** Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... & Amodei, D. (2020). *Language Models are Few-Shot Learners*. Advances in Neural Information Processing Systems (NeurIPS), 33, 1877–1901. Retrieved from https://doi.org/10.48550/arXiv.2005.14165
*(Empirical study analyzing emergent zero-shot and few-shot generalization capabilities in LLMs, establishing the theoretical framework of Section 2.1).*

**[17]** Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. Advances in Neural Information Processing Systems (NeurIPS), 35, 24824–24837. Retrieved from https://doi.org/10.48550/arXiv.2201.11903
*(Prompting technique conceptually preceding the ReAct paradigm, analyzed in Section 2.1 as a direct antecedent of interleaved reasoning and action).*

**[18]** AI Engineer Foundation. (2023). *Agent Protocol: A single common interface for communicating with agents*. Retrieved September 15, 2026, from https://agentprotocol.ai/
*(Community initiative to standardize multi-agent interaction).*

**[19]** Cockburn, A. (2005). *Hexagonal Architecture (Ports and Adapters Pattern)*. Alistair.cockburn.us. Retrieved September 15, 2026, from https://alistair.cockburn.us/hexagonal-architecture/
*(Reference publication for hexagonal core design, dependency decoupling, and domain modeling in Chapter 5).*

**[20]** Malul, E., Meidan, Y., Mimran, D., Elovici, Y., & Shabtai, A. (2024). *GenKubeSec: LLM-Based Kubernetes Misconfiguration Detection, Localization, Reasoning, and Remediation*. Proceedings of the 40th Annual Computer Security Applications Conference (ACSAC). arXiv preprint arXiv:2405.19954. https://doi.org/10.48550/arXiv.2405.19954
*(Academic research addressing Kubernetes security misconfiguration detection and remediation using LLMs, supporting the requirement for strict validation and HITL supervision in Section 2.5.2).*

**[21]** Leppänen, M., Mäkinen, S., Pagels, M., Eloranta, V. P., Itkonen, J., Mäntylä, M. V., & Männistö, T. (2015). *The Highways and Country Roads to Continuous Deployment*. IEEE Software, 32(2), 64–72. https://doi.org/10.1109/MS.2015.50
*(Empirical study on ChatOps adoption and conversational tool integration in CI/CD pipelines, cited in Section 2.5.2 as related work on assistant integration in DevOps workflows).*

**[22]** Chase, H. (2022). *LangChain: Building applications with LLMs through composability*. GitHub Open Source Repository. Retrieved September 15, 2026, from https://github.com/langchain-ai/langchain
*(LLM agent orchestration framework analyzed in Section 2.5.3 as a discarded alternative due to framework coupling and the absence of a standardized interoperability protocol).*

**[23]** Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., ... & Wang, C. (2023). *AutoGen: Enabling next-generation LLM applications via multi-agent conversation*. arXiv preprint arXiv:2308.08155. Retrieved September 15, 2026, from https://arxiv.org/abs/2308.08155
*(Microsoft multi-agent framework analyzed in Section 2.5.3, whose comparative analysis justifies selecting the open MCP standard over proprietary orchestration abstractions).*

**[24]** Pydantic / Colvin, S. (2024). *Pydantic V2: Data validation and settings management using Python type annotations (Rewritten in Rust)*. Retrieved September 15, 2026, from https://docs.pydantic.dev/
*(Core library used for strict invariant validation and configuration drift prevention within the hexagonal domain).*

**[25]** Richards, T. (2023). *Streamlit for Data Science: Create interactive data apps in Python* (2nd ed.). Packt Publishing.
*(Methodological reference for designing the ambiguity-tolerant asynchronous graphical user interface utilized by researchers).*

**[26]** Ollama. (2024). *Ollama: Get up and running with large language models locally*. Retrieved September 15, 2026, from https://ollama.com/
*(Open-source local inference server enabling self-hosted execution of models such as Llama 3, Mistral, and Qwen2.5, guaranteeing institutional data sovereignty. Referenced in Sections 3.3.3, 6.2.1, and 6.2.2).*

**[27]** OWASP Foundation. (2023). *OWASP Top 10 for Large Language Model Applications*. Open Worldwide Application Security Project. Retrieved September 15, 2026, from https://owasp.org/www-project-top-10-for-large-language-model-applications/
*(Reference framework for mitigating attack vectors such as Prompt Injection, addressed in Chapter 4 Change Control and Chapter 9 Case Study).*

**[28]** Brown, S. (2018). *The C4 model for visualising software architecture*. C4model.com. Retrieved September 15, 2026, from https://c4model.com/
*(Modeling methodology employed in Section 5.2 for system segmentation across Context, Container, and Component levels).*

**[29]** Bass, L., Clements, P., & Kazman, R. (2012). *Software Architecture in Practice* (3rd ed.). Addison-Wesley Professional.
*(Foundational literature for formulating availability, latency, and resilience tactics in the hexagonal architecture).*

**[30]** Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional.
*(Software design patterns catalog used to justify the Factory and Adapter patterns in Section 6.2).*

**[31]** Encode OSS. (2024). *HTTPX: A next-generation HTTP client for Python*. Retrieved September 15, 2026, from https://www.python-httpx.org/
*(Library utilized in the native `OllamaLLMClient` implementation for HTTP/1.1 and HTTP/2 communication without third-party dependencies on the Ollama API, documented in Section 6.2.1).*

**[32]** AXELOS. (2019). *ITIL Foundation: ITIL 4 Edition*. TSO (The Stationery Office).
*(Governance framework and best practices for IT service management, cited to justify legal liabilities and the Human-in-the-Loop pattern implementation in Chapter 7).*

**[33]** Richardson, C. (2018). *Microservices Patterns: With examples in Java*. Manning Publications.
*(Extended theoretical reference for transactionality patterns, API Composition, and observability in distributed architectures).*

**[34]** Cohn, M. (2009). *Succeeding with Agile: Software Development Using Scrum*. Addison-Wesley Professional.
*(Foundational work proposing the conceptual model of the Automated Test Pyramid, adapted in Chapter 8 for quality assurance structuring).*

**[35]** McCabe, T. J. (1976). *A Complexity Measure*. IEEE Transactions on Software Engineering, SE-2(4), 308-320. https://doi.org/10.1109/TSE.1976.233837
*(Seminal paper on Cyclomatic Complexity, whose standard threshold supports the `max-complexity = 15` limit enforced in the CI/CD pipeline, Section 8.1.4).*

**[36]** Winters, T., Manshreck, T., & Wright, H. (2020). *Software Engineering at Google: Lessons Learned from Programming Over Time*. O'Reilly Media.
*(Engineering handbook cited in Section 8.1.4 supporting the pragmatic 80% code coverage threshold over dogmatic 100% requirements).*

**[37]** MacIver, D. R., Hatfield-Dodds, Z., et al. (2019). *Hypothesis: A new approach to property-based testing*. Journal of Open Source Software, 4(43), 1891. https://doi.org/10.21105/joss.01891
*(Tooling and theoretical foundation for entropy injection and test-case shrinking documented in the QA evaluation of Chapter 8).*

**[38]** Segura, S., Fraser, G., Sanchez, A. B., & Ruiz-Cortés, A. (2016). *A survey on metamorphic testing*. IEEE Transactions on Software Engineering, 42(9), 805-824. https://doi.org/10.1109/TSE.2016.2532875
*(Systematic survey paper used to design evaluation via metamorphic testing addressing the test oracle problem in Section 8.4).*

**[39]** Mrkšić, N., Séaghdha, D. O., Wen, T. H., Thomson, B., & Young, S. (2017). *Neural Belief Tracker: Data-Driven Dialogue State Tracking*. Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL). Retrieved from https://arxiv.org/abs/1606.03777
*(Foundational article on dialogue state tracking and entity extraction ["Slot-Filling"], cited in Section 9.4.3 as the theoretical basis for the distilled state model).*

**[40]** LangChain Contributors. (2024). *Memory Management in LLM Applications (ConversationSummaryMemory & Context Distillation)*. LangChain Documentation. Retrieved September 15, 2026, from https://python.langchain.com/v0.2/docs/concepts/#memory
*(Industry reference on architectural techniques for conversational history compression and attention dilution prevention in context-constrained models, cited in Section 9.4.3).*

**[41]** Snell, C., et al. (2022). *Learning by Distilling Context*. arXiv preprint arXiv:2209.15189. Retrieved from https://arxiv.org/abs/2209.15189
*(Primary academic literature theoretically underpinning the Context Distillation technique for Language Models, cited in Section 9.4.3).*

**[42]** Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). *Experimentation in Software Engineering*. Springer Science & Business Media.
*(Methodological reference on empirical research in software engineering and formalization of validity threats [internal, external, construct, and conclusion], cited in Section 9.6).*

**[43]** Runeson, P., & Höst, M. (2009). *Guidelines for conducting and reporting case study research in software engineering*. Empirical Software Engineering, 14(2), 131-164. https://doi.org/10.1007/s10664-008-9102-8
*(Methodological guidelines for designing, conducting, and reporting empirical case studies in software engineering, cited in Section 9.6).*

**[44]** OpenAI. (2024). *Hello GPT-4o*. Retrieved September 15, 2026, from https://openai.com/index/hello-gpt-4o/
*(Technical documentation of the industry-standard frontier model, referenced in Section 10.1.9 for its native tool interaction and JSON Schema capabilities).*

**[45]** Anthropic. (2024). *Introducing Claude 3.5 Sonnet*. Retrieved September 15, 2026, from https://www.anthropic.com/news/claude-3-5-sonnet
*(Official documentation detailing fine-tuning and structural optimization of the Claude family for agentic workflows and tool calling, cited in Section 10.1.9).*

**[46]** Open Policy Agent (OPA). (2024). *OPA: Policy-based control for cloud native environments*. Cloud Native Computing Foundation (CNCF). Retrieved September 15, 2026, from https://www.openpolicyagent.org/
*(Policy-as-Code engine proposed as the architectural evolution of `SecurityContextValidator` in Future Work [Section 10.3.2], decoupling and updating validation rules without redeploying the backend).*

**[47]** Kyverno Authors. (2024). *Kyverno: Kubernetes Native Policy Management*. Cloud Native Computing Foundation (CNCF). Retrieved September 15, 2026, from https://kyverno.io/

*(Kubernetes-native alternative to OPA for declarative security policy management as cluster resources. Referenced in Section 10.3.2 as a governance mechanism for system evolution).*

# Annex A. Generated Kubernetes Manifest — Scenario 1 (Happy Path)

> **Reference:** Section 9.2.1 — *Walkthrough: Web Portal for an Academic Conference*
> **Source artifact:** `demos/outputs/dep-7f3a2c1b_congreso-ia-departamento.yaml`

This annex contains the full YAML manifest automatically generated by the `FakeK8sAdapter` of *Agentic Deployer* from a natural language request issued by the researcher (see Section 9.2.1). The manifest is a self-contained declarative output consisting of three standard Kubernetes resources:

1. **`Deployment`** — Defines the pod specification, container image, compute resources (CPU/RAM), health check probes, and security context.
2. **`Service`** — Exposes the Deployment within the cluster via standard `ClusterIP`, routing port 80 traffic to `containerPort` 8080.
3. **`Ingress`** — Configures external routing via the NGINX Ingress Controller with automated TLS termination (cert-manager / Let's Encrypt).

The resulting YAML is **directly applicable** to a physical Kubernetes cluster via the standard CLI command:

```
kubectl apply -f dep-7f3a2c1b_congreso-ia-departamento.yaml
```

---

## A.1. Generation Context

| Field | Value |
|---|---|
| **Deployment ID** | `dep-7f3a2c1b` |
| **Invoked MCP Tool** | `deploy_congress_web` |
| **LLM Model** | `qwen2.5:7b` (Ollama, local) |
| **User Input** | "necesito un portal web para el congreso de IA, 200-300 personas" |
| **Inferred Parameters** | `project_name=congreso-ia-departamento`, `expected_traffic=medium` |
| **Final State** | `DEPLOYED` (following HITL approval) |
| **Timestamp** | 2026-09-27T10:14:34+02:00 |
| **Adapter** | `FakeK8sAdapter` (production: `RealK8sAdapter` via `kubernetes-client`) |

***Table 38:*** *Generation Context of the YAML Manifest (Scenario 1).*

---

# A.2. Complete YAML Manifest

```yaml
---
# ============================================================
# Kubernetes Deployment Manifest
# Generated by: Agentic Deployer — FakeK8sAdapter
# Tool:     deploy_congress_web
# Deployment ID: dep-7f3a2c1b
# Timestamp:  2026-09-27T10:14:34+02:00
# Status:    DEPLOYED (HITL-approved by tecnico-sic@universidad.es)
# ============================================================
---
apiVersion: apps/v1
kind: Deployment
metadata:
 name: congreso-ia-departamento
 namespace: sic-research
 labels:
  app: congreso-ia-departamento
  managed-by: agentic-deployer
  deployment-id: dep-7f3a2c1b
  tier: web
 annotations:
  agentic-deployer/requested-by: investigador@universidad.es
  agentic-deployer/approved-by: tecnico-sic@universidad.es
  agentic-deployer/tool: deploy_congress_web
  agentic-deployer/traffic-level: medium
  agentic-deployer/llm-model: qwen2.5:7b
spec:
 replicas: 2
 selector:
  matchLabels:
   app: congreso-ia-departamento
 template:
  metadata:
   labels:
    app: congreso-ia-departamento
  spec:
   containers:
    - name: congreso-ia-departamento
     image: nginx:alpine     # SIC Policy: certified image (disallow :latest)
     ports:
      - containerPort: 8080
     resources:
      requests:
       cpu: "250m"       # Baseline: 50% of limit
       memory: "128Mi"
      limits:
       cpu: "500m"       # Computed by _calculate_congress_resources("medium")
       memory: "256Mi"
     readinessProbe:
      httpGet:
       path: /
       port: 8080
      initialDelaySeconds: 5
      periodSeconds: 10
     livenessProbe:
      httpGet:
       path: /
       port: 8080
      initialDelaySeconds: 15
      periodSeconds: 20
     securityContext:
      readOnlyRootFilesystem: false
      runAsNonRoot: true
      runAsUser: 101       # Non-root nginx user
---
apiVersion: v1
kind: Service
metadata:
 name: congreso-ia-departamento-svc
```

```
  namespace: sic-research
  labels:
    app: congreso-ia-departamento
    managed-by: agentic-deployer
spec:
  selector:
    app: congreso-ia-departamento
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080         # Forwards to containerPort defined above
  type: ClusterIP           # Internal cluster access only (Ingress as perimeter)
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: congreso-ia-departamento-ingress
  namespace: sic-research
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
    cert-manager.io/cluster-issuer: "letsencrypt-prod" # Automated TLS
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - congreso-ia.universidad.es
      secretName: congreso-ia-tls
  rules:
    - host: congreso-ia.universidad.es
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: congreso-ia-departamento-svc
                port:
                  number: 80
```

***Listing 2:*** *Complete Kubernetes manifest generated by Agentic Deployer FakeK8sAdapter (Deployment + Service + Ingress) for Scenario 1 (Happy Path).*

---

# A.3. Source Code Mapping

The manifest above is generated by the `deploy()` method of `FakeK8sAdapter`, implemented in `app/infrastructure/fake_k8s_adapter.py`. The adapter consumes a `DeploymentIntent` domain object (validated via Pydantic) and projects it onto f-string templates formatted with `textwrap.dedent`, without external dependencies or network overhead.

In an active production environment, `RealK8sAdapter` (future work, Section 10.3.1) replaces local disk serialization with authenticated Kubernetes API invocations via `kubernetes-client`:

```
# Future RealK8sAdapter implementation (pseudocode)
from kubernetes import client, config

config.load_incluster_config()
apps_v1 = client.AppsV1Api()
apps_v1.create_namespaced_deployment(
  namespace="sic-research",
  body=deployment_manifest
)
```

The hexagonal architecture guarantees that this adapter substitution requires zero modifications to the core business logic (Section 5.3.2), empirically validating the Dependency Inversion Principle (DIP) as the architectural foundation for long-term technological evolution.

# Annex B. Detailed Sprint Retrospectives

> This annex complements **Chapter 4** with granular tracking of each project sprint. For the planning framework, effort estimates, economic analysis, and ROI calculation, refer to the body of Chapter 4.

---

## B.1. Sprint 1 — Hexagonal Core

| Metric | Estimate | Actual | Deviation |
|---|---|---|---|
| Dedicated hours | 50h | ~55h | +10% |
| Tests (Unit + Property-Based) | ~15 | 51 tests | +240% |
| Domain coverage | >90% | >95% | +5pp |

***Table 39:*** *Sprint 1 Retrospective (Hexagonal Core).*

**Completed milestones:** `DeploymentIntent` with Pydantic v2, `SecurityContextValidator` (5 rules), abstract `DeployPort`, Property-Based test suite with Hypothesis.

**Notes:** Incorporating secret validation in `env_vars` and hardware quota enforcement (CPU/RAM) was not part of the initial estimate; it was identified during Property-Based Testing as a concrete operational risk vector. The associated time investment was compensated by the structural robustness gained across higher layers.

---

## B.2. Sprint 2 — HITL Backend and FSM

| Metric | Estimate | Actual | Deviation |
|---|---|---|---|
| Dedicated hours | 45h | ~45h | 0% |
| REST Endpoints | 4 | 5 (+ `/hitl/reject`) | +25% |
| FSM States | 4 | 6 (+ `FAILED`, `DELETED`) | +50% |

***Table 40:*** *Sprint 2 Retrospective (HITL Backend and FSM).*

**Completed milestones:** FastAPI backend with 5 endpoints, FSM with acyclic DAG, HTML Dashboard with polling, complete `DeploymentStatus` enumeration.

**Notes:** The client-polling pattern was more straightforward to implement than WebSocket alternatives, yielding net schedule savings. Adding the `FAILED` and `DELETED` states reflected proactive alignment between the academic documentation and codebase to encompass complete deployment lifecycles.

---

## B.3. Sprint 3 — Golden Paths and FakeK8s

| Metric | Estimate | Actual | Deviation |
|---|---|---|---|
| Dedicated hours | 40h | ~46h | +15% |
| MCP Tools | 3 | 4 | +33% |
| YAML Templates | 2 types | 3 types | +50% |

***Table 41:*** *Sprint 3 Retrospective (Golden Paths and FakeK8s).*

**Completed milestones:** `FakeK8sAdapter` with f-string templates and `textwrap.dedent`, 4 MCP tools decorated with `@mcp_server.tool()`, CI pipeline `scripts/run_tests.sh` (6 fail-fast stages), integrated Ruff and Mypy tooling.

**Notes:** Integrating the official MCP SDK (`mcp==2.2.0`) in dual mode—executable via `stdio` (external clients) and importable in-process (Streamlit)—required additional engineering iterations. This architectural choice represents a core differentiating asset regarding system interoperability.

---

## B.4. Sprint 4 — ReAct Agent and MCP SDK

| Metric | Estimate | Actual | Deviation |
|---|---|---|---|
| Dedicated hours | 55h | ~62h | +13% |
| LLM Clients | 2 (Fake + OpenAI) | 3 (+native Ollama) | +50% |
| Maximum ReAct iterations | 5 | 5 | 0% |

***Table 42:*** *Sprint 4 Retrospective (ReAct Agent and MCP SDK).*

**Completed milestones:** `LLMClient` ABC enforcing dependency inversion, native `OllamaLLMClient` (direct `httpx` against Ollama's REST API), `OpenAILLMClient`, `AgentOrchestrator` (ReAct loop), `LLM_PROVIDER` Factory (`fake | ollama | openai`).

**Notes:** Serializing Python tool signatures into compliant JSON Schemas for the MCP protocol required targeted debugging. The native `OllamaLLMClient` was completed during system finalization, upgrading the module from P1 baseline to full P2 production fidelity.

---

## B.5. Sprint 5 — Advanced QA

| Metric | Estimate | Actual | Deviation |
|---|---|---|---|
| Dedicated hours | 45h | ~50h | +11% |

| Metric | Estimate | Actual | Deviation |
|---|---|---|---|
| Full test suite (Unit/PBT/MR) | ~35 | 51 | +46% |
| Killed mutants | >90% | >95% | +5pp |
| Metamorphic relations | 3 MR | 6 MR | +100% |

***Table 43:*** *Sprint 5 Retrospective (Advanced QA).*

**Completed milestones:** Comprehensive QA pipeline implemented in `scripts/run_tests.sh` (Linter, Type-checking, Pytest, Mutmut, Playwright, Locust), Mutation Testing (12 surviving mutants mitigated), 6 Metamorphic Relations (MR-1 through MR-6), Load Testing (Locust, p99 < 200ms), Playwright E2E UI testing on the Dashboard.

**Notes:** Uncovering 12 surviving mutants in `mcp_server.py` was the most impactful finding of the sprint, driving the creation of targeted tests in `test_mcp_server.py`. This empirically substantiates adopting Mutation Testing (Section 8.3).

---

# B.6. Phase 6 — Thesis Drafting and Finalization

| Metric | Estimate | Actual | Deviation |
|---|---|---|---|
| Drafting hours | 55h | ~64h | +16% |
| Thesis length | ~100 pages | **~150 pages** | +50% |
| Review iterations | 2–3 | 5 | +67% |

***Table 44:*** *Phase 6 Retrospective (Thesis Drafting and Finalization).*

**Completed milestones:** Full monograph drafting (11 chapters), technical diagramming and visual captures (20 figures), YAML and Planning Annexes.

**Notes:** The thesis expanded beyond the initial baseline (~100 estimated pages versus ~150 resulting pages across main chapters and technical annexes). Incorporating formal architectural diagrams (C4, Hexagonal Architecture, FSM, E2E sequences, testing pyramid), four complete execution walkthroughs (Section 9.2), and three comprehensive technical annexes substantially elevated documentation depth. The drafting schedule deviation (+16%) remained constrained through the structured use of an AI assistant (Google Gemini Advanced) as an efficiency multiplier in technical drafting—an effect documented in §4.3 and §4.7. The resulting ~150-page document serves as tangible evidence of methodological rigor and thoroughness.

---

# B.7. Phase 7 — Technical Excellence Consolidation

| Metric | Estimate | Actual | Deviation |
|---|---|---|---|
| Dedicated hours | 10h | ~18h | +80% |

| Metric | Estimate | Actual | Deviation |
|---|---|---|---|
| Total MCP tools | 4 | 7 | +75% |
| Total tests (Unit + Integration) | 51 tests | 65 tests | +27% |

***Table 45:*** *Phase 7 Retrospective (Technical Excellence Consolidation).*

**Completed milestones:** Replaced ephemeral in-memory state with **SQLite** (ACID transactions), expanded tool suite to 7 MCP tools (Databases, Static Sites, Status inspection), empirical external verification (integration with **MCP Inspector** documented in README), green CI pipeline with 0 failures, **Multi-Model Cognitive Experiments** (Qwen vs Mistral vs Llama vs Coder vs Cloud).

**Notes:** This phase was planned from the outset as an optional technical excellence objective (Priority P2) and was successfully undertaken due to efficient completion of mandatory baseline modules (P0/P1). The hexagonal architecture demonstrated its ability to absorb a complete data layer migration (from RAM to SQLite) by altering adapters alone. Furthermore, empirical evaluations across diverse 7B generalist LLMs (Mistral, Llama 3.2, Qwen) documented structural degradation and attention loss under stress. Crucially, introducing the specialized **Qwen 2.5 Coder (7B)** model yielded the defining empirical insight of this phase: its ability to pass all metamorphic relations validated financial feasibility conclusions (Chapters 4 and 9), proving that enterprise-grade operational performance is achievable entirely on entry-level commodity hardware with Zero Data Retention.

# Annex C. Installation, Deployment, and Operational Manual

This annex constitutes the technical operational manual for experimental replication, installation, deployment, and usage of the *Agentic Deployer* platform. Its structure follows software engineering standards and scientific reproducibility guidelines.

---

## C.1. Installation and Deployment Manual

### C.1.1. System Prerequisites

The following host prerequisites are required for deploying the system in a local development environment or dedicated server:

- **Operating System:** GNU/Linux (tested on Debian/Ubuntu distributions) or macOS. Fully compatible with Windows via WSL2.
- **Runtime Environment:** Python 3.11 or higher.
- **Package Management:** `pip` and native virtual environment tooling (`python3-venv`).
- **Web Browser:** Modern web browser (Chrome, Firefox, or Edge) to interact with user interfaces.
- **Local Inference Engine (Optional for on-premise execution):** Ollama (v0.3.0 or higher) with GPU acceleration (CUDA / Metal) or CPU fallback.

### C.1.2. Source Code Acquisition and Environment Setup

1. **Clone the repository and enter the root directory:**

```
git clone https://github.com/JesusSolisOrtega/agentic-deployer.git
cd agentic-deployer
```

2. **Create and activate the Python virtual environment:**

```
python3 -m venv .venv
source .venv/bin/activate
```

3. **Install production and quality assurance dependencies:**

```
pip install --upgrade pip
pip install -r requirements.txt
```

4. **Install browser binaries for end-to-end UI testing (Playwright):**

```
playwright install chromium
```

### C.1.3. Environment Variable Configuration

The system is configured through a `.env` environment file. A preconfigured baseline template (`.env.example`) is provided:

```
cp .env.example .env
```

Key configurable environment variables include:

- `LLM_PROVIDER`: Cognitive inference provider (`ollama`, `gemini`, `openai`, or `fake`).
- `OLLAMA_HOST`: Listening address of the local Ollama daemon (defaults to `http://localhost:11434`).
- `OLLAMA_MODEL`: Identifier of the local model to load (defaults to `qwen2.5:7b` or `qwen2.5-coder:7b`).
- `OPENAI_API_KEY` / `GEMINI_API_KEY`: Authentication credentials when selecting cloud providers.
- `BACKEND_URL`: Base URL of the REST middleware (defaults to `http://localhost:8000`).
- `DATABASE_PATH`: Filesystem path to the SQLite persistence database (`agentic_deployer.db`).

*(Note: The executable script `scripts/install.sh` automates the above steps and triggers local model retrieval on compatible systems).*

### C.1.4. Installation Verification

Prior to launching production services, executing the automated test suite is strongly recommended:

```
pytest -v
```

Passing the unit and integration test suites validates that domain invariants, the FSM state machine, and security policy validators are fully functional.

---

## C.2. Subsystem Execution

The decoupled architecture requires the concurrent execution of two independent services:

### C.2.1. Operations Backend and MCP Server (FastAPI)

In a primary terminal with the active virtual environment, start the application server:

```
uvicorn app.infrastructure.main:app --host 0.0.0.0 --port 8000 --reload
```

- **HITL Operations Dashboard:** http://localhost:8000/frontend/ (with automatic redirection from http://localhost:8000)
- **Interactive OpenAPI (Swagger) Documentation:** http://localhost:8000/docs

### C.2.2. Agent Conversational Interface (Streamlit)

In a secondary independent terminal with the active virtual environment, launch the chat client:

- **On-Premise Mode (Data Sovereignty with Ollama):**

```
# In an auxiliary terminal, launch the model server if not already running:
ollama serve

# Launch the agentic interface connected to the local model:
LLM_PROVIDER=ollama OLLAMA_MODEL=qwen2.5:7b streamlit run app/agent_layer/chat_app.py
```

- **Rapid Demonstration Mode (Deterministic, zero GPU and API key overhead):**

```
streamlit run app/agent_layer/chat_app.py
```

- **Web entry point:** http://localhost:8501

---

# C.3. User Manual and HITL Operation

## C.3.1. Formulating Natural Language Requests

1. The researcher accesses the conversational web interface on port `8501`.
2. In the bottom text input (or via quick suggestion buttons), the user inputs a service request in natural language. Valid examples:
    - *"I need a website for the 'ai-congress-2025' conference with medium traffic."*
    - *"Deploy a WordPress instance for the computer science department."*
    - *"Deploy a Python REST API to publish research datasets."*
    - *"I need a PostgreSQL database with 10 GB of storage capacity."*
    - *"I want to decommission the old-web service."*
3. **Dialogue and parameter negotiation:** If the request lacks mandatory attributes or violates institutional policies, the agent reasons over validator observations and prompts the user with guided questions to converge on compliant parameters.
4. **Intent registration:** Upon resolving valid parameters, the agent invokes the corresponding MCP tool and persists the deployment intent in quarantine status (`PENDING_APPROVAL`), returning the unique deployment tracking ID to the user (e.g., `120a9ea1`).

## C.3.2. Technical Supervision via the HITL Dashboard

1. The institutional IT operator accesses the Approval Dashboard at http://localhost:8000/frontend/.
2. Under **"Pending Requests"**, the operator inspects the technical profile generated by the agent:
    - Allocated compute quotas (CPU and memory limits).
    - Certified container image sourced from institutional registries and exposed service ports.
    - Generated declarative Kubernetes manifest and security policy validation findings.
3. **Operator decision:**

- **Approve:** Clicking the confirmation button transitions the FSM to `DEPLOYED`, executing deployment simulation against OpenShift/Kubernetes and recording an immutable audit trail.
- **Reject:** If administrative discrepancies or quota violations are detected, clicking **Reject** transitions the request to the immutable terminal state `REJECTED`.

## C.3.3. Asynchronous Feedback and Status Tracking

The researcher can inspect request status at any time without full page reloads:

1. **Within the chat interface (Streamlit):** Under the bottom expander **"📬 My Pending Requests"**, clicking **"🔄 Refresh"**, or querying the conversational assistant directly:

   > *"What is the status of request 120a9ea1?"*

2. **Via the REST traceability endpoint:**

```
curl -X GET "http://localhost:8000/hitl/status/120a9ea1"
```

The system responds with the current FSM lifecycle state (`PENDING_APPROVAL`, `DEPLOYED`, or `REJECTED`) along with container specifications, guaranteeing end-to-end operational traceability.